

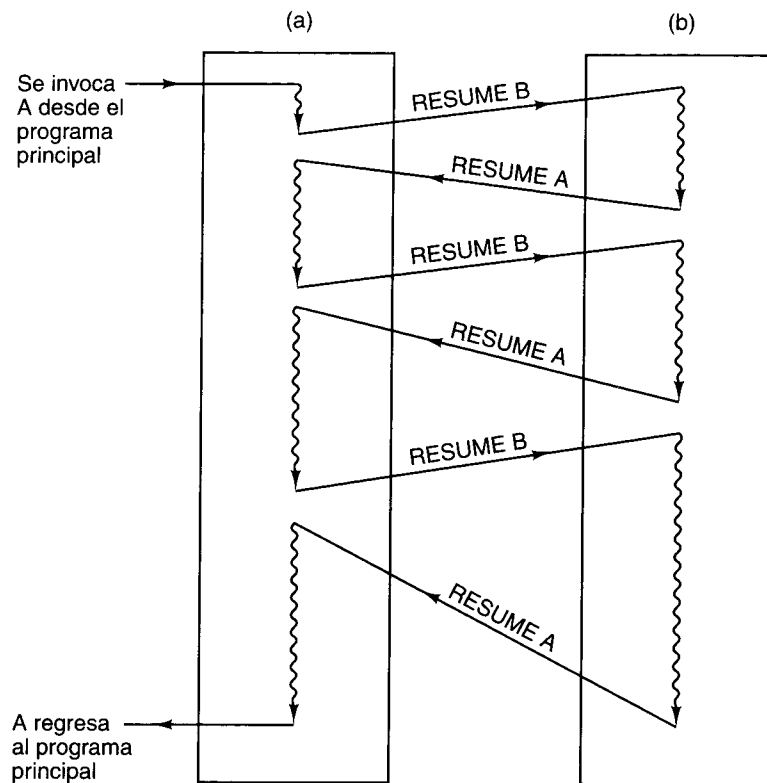


Figura 5-43. Cuando se reanuda una corrutina, la ejecución inicia en el enunciado donde se suspendió en la ocasión anterior, no al principio.

A veces resulta útil tener dos procedimientos, A y B, cada uno de los cuales invoca al otro como procedimiento, como se muestra en la figura 5-43. Cuando B regresa a A, salta al enunciado que sigue a la llamada a B, igual que antes. Cuando A transfiere el control a B, pasa no al principio (excepto la primera vez) sino al enunciado que sigue al “retorno” más reciente, es decir, la llamada más reciente de A. Dos procedimientos que funcionan de esta forma se llaman **corrutinas**.

Un uso común de las corrutinas es simular un procesamiento en paralelo con una sola CPU. Cada corrutina se ejecuta en seudoparalelo con las otras, como si tuviera su propia CPU. Este estilo de programación facilita la implementación de algunas aplicaciones, y también es útil para probar software que después se ejecutará en un multiprocesador.

Ni la llamada **CALL** normal ni la instrucción **RETURN** normal sirven para llamar corrutinas, ya que la dirección a la que se saltará proviene de la pila como en un retorno pero, a diferencia de un retorno, la llamada a la corrutina misma coloca una dirección de retorno en algún lugar para el regreso subsecuente a ella. Sería bueno contar con una instrucción que intercambiara el tope de la pila y el contador de programa. En detalle, esta instrucción primero sacaría la antigua dirección de retorno de la pila y la colocaría en un registro interno, luego metería el

contenido del contador de programa en la pila, y por último copiaría el registro interno en el contador de programa. Dado que se saca una palabra de la pila y se mete una palabra en ella, el apuntador de pila no cambia. Esta instrucción casi nunca existe, por lo que en la generalidad de los casos se tiene que simular con varias instrucciones.

5.6.4 Trampas

Una **trampa** es una especie de llamada a procedimiento automática iniciada por alguna condición causada por el programa, casi siempre una condición importante pero que raras veces ocurre. Un buen ejemplo es un desbordamiento. En muchas computadoras, si el resultado de una operación aritmética excede el número más grande que se puede representar, ocurre una trampa, lo que significa que el flujo de control se conmuta a alguna posición de memoria fija en lugar de continuar en sucesión. En esa posición fija hay una ramificación a un procedimiento llamado **manejador de trampas** que realiza alguna acción apropiada, como exhibir un mensaje de error. Si el resultado de una operación está dentro del intervalo, no ocurre ninguna trampa.

La característica fundamental de una trampa es que es iniciada por alguna condición especial causada por el programa mismo y detectada por el hardware o el microprograma. Un método alternativo para manejar el desbordamiento es tener un registro de un bit que se enciende cada vez que ocurre un desbordamiento. Un programador que quiera verificar si hubo o no desbordamiento deberá incluir una instrucción explícita “saltar si el bit de desbordamiento está encendido” después de cada instrucción aritmética. Esto es lento y desperdicia espacio. Las trampas ahorran tiempo y memoria en comparación con las verificaciones explícitas controladas por el programador.

La trampa podría implementarse con una prueba explícita efectuada por el microprograma (o el hardware). Si se detecta un desbordamiento, la dirección de la trampa se carga en el contador de programa. Lo que en un nivel es una trampa podría estar bajo el control del programa en un nivel más bajo. Hacer que el microprograma realice la prueba ahorra tiempo en comparación con una prueba del programador, porque se puede traslapar fácilmente con alguna otra cosa. También se ahorra memoria, porque sólo necesita ocurrir en un lugar, digamos el ciclo principal del microprograma, sin importar cuántas instrucciones aritméticas haya en el programa principal.

Algunas de las condiciones comunes que pueden causar trampas son el desbordamiento de punto flotante, el subdesbordamiento de punto flotante, el desbordamiento de enteros, una violación de la protección, un código de operación no definido, un desbordamiento de la pila, un intento por iniciar un dispositivo de E/S inexistente, un intento por traer una palabra de dirección impar, y una división entre cero.

5.6.5 Interrupciones

Las **interrupciones** son cambios en el flujo de control causados no por el programa en ejecución sino por otra cosa, casi siempre relacionada con E/S. Por ejemplo, un programa podría ordenar al disco que comience a transferir información, y que genere una interrupción apenas

haya terminado la transferencia. Al igual que la trampa, la interrupción detiene el programa en ejecución y transfiere el control a un manejador de interrupciones, el cual realiza alguna acción apropiada. Cuando el manejador de interrupciones termina, devuelve el control al programa interrumpido. El proceso interrumpido se debe reiniciar en el mismo estado exactamente en el que estaba cuando ocurrió la interrupción, lo que implica restaurar todos los registros internos a su estado previo a la interrupción.

La diferencia fundamental entre las trampas y las interrupciones es ésta: las *trampas* son sincrónicas con el programa y las *interrupciones* son asincrónicas. Si el programa se vuelve a ejecutar un millón de veces con las mismas entradas, las trampas volverán a ocurrir en el mismo punto en cada ocasión pero las interrupciones podrían variar, dependiendo, por ejemplo, del momento exacto en que una persona sentada ante una terminal pulsa el retorno de carro. La razón de que las trampas sean reproducibles y las interrupciones no lo sean es que las primeras son causadas directamente por el programa, mientras que las segundas son, si acaso, indirectamente causadas por el programa.

Para ver cómo funcionan en realidad las interrupciones, consideremos un ejemplo común: una computadora quiere enviar una línea de caracteres a una terminal. El software del sistema primero reúne en un buffer todos los caracteres que se van a escribir en la terminal, inicializa una variable global *apunt* que apunte al principio del buffer, y asigna a una segunda variable global *cuenta* el número de caracteres que se escribirán. Luego, el software verifica si la terminal está lista y, si así es, le envía el primer carácter (por ejemplo, usando registros como los de la figura 5-30). Después de iniciar la E/S, la CPU queda libre para ejecutar otro programa o hacer alguna otra cosa.

En algún momento, el carácter se exhibe en la pantalla. Ahora puede iniciarse la interrupción. En una forma simplificada, los pasos son los siguientes.

ACCIONES DEL HARDWARE

1. El controlador de dispositivo habilita una línea de interrupción en el bus del sistema para iniciar la secuencia de interrupción.
2. Tan pronto como la CPU está lista para manejar la interrupción, habilita una señal de reconocimiento de interrupción en el bus.
3. Cuando el controlador de dispositivo ve que su señal de interrupción ha sido reconocida, coloca un entero pequeño en las líneas de datos para identificarse. Este número se llama **vector de interrupción**.
4. La CPU quita el vector de interrupción del bus y lo guarda temporalmente.
5. Luego la CPU almacena el contador de programa y la palabra de estado del programa (PSW) en la pila.
6. La CPU localiza un nuevo contador de programa utilizando el vector de interrupción como índice de una tabla que está en la parte baja de la memoria. Si el contador de programa tiene 4 bytes, por ejemplo, el vector de interrupción n corresponde a la dirección $4n$. Este nuevo contador de programa apunta al inicio de la rutina de

servicio de interrupción para el dispositivo que causó la interrupción. En muchos casos también se carga o modifica la PSW (por ejemplo, para inhabilitar interrupciones subsecuentes).

ACCIONES DEL SOFTWARE

7. Lo primero que hace la rutina de servicio de interrupción es guardar todos los registros para que puedan restaurarse después. Se pueden guardar en la pila o en una tabla del sistema.
8. En general, cada vector de interrupción es compartido por todos los dispositivos de un tipo dado, por lo que todavía no se sabe cuál terminal causó la interrupción. El número de terminal puede averiguarse leyendo algún registro de dispositivo.
9. Ahora puede leerse cualquier otra información que haya acerca de la interrupción, como códigos de situación.
10. Si ocurrió un error de E/S, se puede manejar aquí.
11. Se actualizan las variables globales *apunt* y *cuenta*. La primera se incrementa de modo que apunte al siguiente byte, y la segunda se decrementa para indicar que falta un byte menos que enviar a la salida. Si *cuenta* sigue siendo mayor que 0, hay más caracteres que exhibir. El carácter al que ahora apunta *apunt* se copia en el registro buffer de salida.
12. Si es necesario, se envía a la salida un código especial para indicar al dispositivo o al controlador de interrupciones que ya se procesó la interrupción.
13. Se restauran todos los registros que se guardaron.
14. Se ejecuta la instrucción **RETURN FROM INTERRUPT** (regresar de interrupción), que coloca la CPU otra vez en el modo y el estado en que estaba antes de la interrupción. La computadora continúa con lo que estaba haciendo.

Un concepto fundamental relacionado con las interrupciones es la **transparencia**. Cuando ocurre una interrupción se efectúan algunas acciones y se ejecuta cierto código, pero una vez que todo ha terminado la computadora deberá volver al mismo estado exactamente en el que estaba antes de la interrupción. Se dice que una rutina de interrupción que tiene esta propiedad es transparente. Si todas las interrupciones son transparentes es mucho más fácil entenderlas.

Si una computadora sólo tiene un dispositivo de E/S, las interrupciones siempre funcionan como acabamos de describirlas, y no tenemos más que decir al respecto. Sin embargo, una computadora grande podría tener muchos dispositivos de E/S, y varios podrían estar operando al mismo tiempo, tal vez para diferentes usuarios. Existe una probabilidad finita de que mientras se está ejecutando una rutina de interrupción, un segundo dispositivo de E/S quiera generar su interrupción.

Este problema se puede atacar de dos maneras. La primera es que todas las rutinas de interrupción inhabiliten las interrupciones subsecuentes como primer paso, aun antes de guardar los registros. Tal estrategia simplifica las cosas, ya que entonces las interrupciones se proce-

san en sucesión, pero puede causar problemas en el caso de dispositivos que no pueden tolerar mucho retraso. Por ejemplo, en una línea de comunicación de 9600 bps llegan caracteres cada 1042 μ s, estemos o no listos para recibirlos. Si el primero todavía no se ha procesado cuando llega el segundo, se podrían perder datos.

Cuando una computadora tiene dispositivos de E/S para los que el tiempo es crítico, una mejor estrategia de diseño es asignar a cada dispositivo de E/S una prioridad, alta para los dispositivos críticos y baja para los más tolerantes de retrasos. La CPU deberá tener también prioridades, generalmente determinadas por un campo de la PSW. Cuando un dispositivo con prioridad n interrumpe, la rutina de interrupción también deberá ejecutarse con prioridad n .

Mientras una rutina de interrupción con prioridad n se está ejecutando, se hará caso omiso de cualquier intento de un dispositivo con más baja prioridad de causar una interrupción, hasta que la rutina termine y la CPU regrese para ejecutar un programa de usuario (prioridad 0). En cambio, las interrupciones de dispositivos con más alta prioridad se procesan sin retraso.

Ahora que las rutinas de interrupción mismas están sujetas a interrupciones, la mejor manera de organizar la administración es asegurarse de que todas las interrupciones sean transparentes. Consideremos un sencillo ejemplo de múltiples interrupciones. Una computadora tiene tres dispositivos de E/S, una impresora, un disco y una línea RS232, con prioridades 2, 4 y 5, respectivamente. En un principio ($t = 0$) se está ejecutando un programa de usuario. En $t = 10$ ocurre una interrupción de impresora. Se pone en marcha la rutina de servicio de interrupción (ISR, *Interrupt Service Routine*) de la impresora, como se muestra en la figura 5-44.

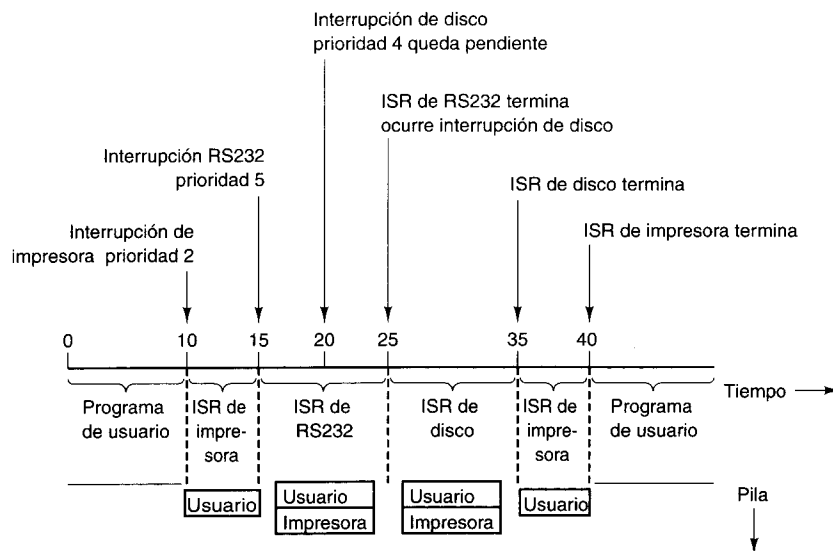


Figura 5-44. Secuencia temporal del ejemplo de múltiples interrupciones.

En $t = 15$ la línea RS232 necesita que la atiendan y genera una interrupción. Puesto que la línea RS232 tiene mayor prioridad (5) que la impresora (2), la interrupción ocurre. El estado de la máquina, que ahora está ejecutando la rutina de servicio de interrupción de la impresora, se almacena en la pila, y se inicia la rutina de servicio de interrupción de RS232.

Un poco después, en $t = 20$, el disco termina y quiere que lo atiendan. Sin embargo, su prioridad (4) es menor que la de la rutina de interrupción que se está ejecutando (5), por lo que el hardware de la CPU no reconoce la interrupción, y queda pendiente. En $t = 25$ termina la rutina de RS232, por lo que la máquina regresa al estado en el que estaba justo antes de que ocurriera la interrupción de RS232, es decir, ejecutando la rutina de servicio de interrupción de la impresora con prioridad 2. Tan pronto como la CPU cambia a prioridad 2, antes de que pueda ejecutarse siquiera una instrucción, se permite la entrada de la interrupción de disco con prioridad 4, y se ejecuta la rutina de servicio de disco. Cuando esta rutina termina, la rutina de la impresora puede continuar. Por último, en $t = 40$, todas las rutinas de servicio de interrupción han terminado y el programa de usuario continúa desde donde se quedó.

Desde el 8088, los chips de CPU de Intel han tenido dos niveles de interrupción (prioridades): enmascarables y no enmascarables. Las interrupciones no enmascarables por lo regular sólo se usan para avisar de posibles catástrofes, como errores de paridad de memoria. Todos los dispositivos de E/S usan la interrupción enmascarable.

Cuando un dispositivo de E/S emite una interrupción, la CPU usa el vector de interrupción como índice de una tabla de 256 entradas para encontrar la dirección de la rutina de servicio de interrupción. Las entradas de la tabla son descriptores de segmento de 8 bytes y la tabla puede iniciar en cualquier lugar de la memoria. Un registro global apunta a su principio.

Al tener un solo nivel de interrupción utilizable, la CPU no puede permitir que un dispositivo con mayor prioridad interrumpa una rutina de servicio de interrupción con prioridad intermedia y al mismo tiempo prohíba a un dispositivo con menor prioridad hacerlo. Para resolver este problema, las CPU de Intel ahora se usan normalmente con un controlador de interrupciones externo (por ejemplo, un 8259A). Cuando llega la primera interrupción, con una prioridad de n , la CPU se interrumpe. Si después llega una interrupción con mayor prioridad, el controlador de interrupciones interrumpe otra vez. Si la segunda interrupción es de menor prioridad, se detiene hasta que la primera termina. Para que este esquema funcione el controlador de interrupciones debe poder detectar el momento en que la rutina de servicio de interrupción en curso termina, por lo que la CPU debe enviarle un comando al terminar de procesar la interrupción en curso.

5.7 UN EJEMPLO DETALLADO: LAS TORRES DE HANOI

Ahora que hemos estudiado la ISA de tres máquinas, armemos todas las piezas y examinemos de cerca el mismo programa de ejemplo para las tres ISA. Nuestro ejemplo será el programa de las Torres de Hanoi. Ya dimos una versión en Java de este programa en la figura 5-40. En las secciones que siguen presentaremos programas en lenguaje ensamblador para el problema de las Torres de Hanoi.

Sin embargo, usaremos un pequeño truco. En vez de dar la traducción de la versión en Java, para el Pentium II y el UltraSPARC II daremos la traducción de una versión en C con objeto de evitar ciertos problemas de la E/S de Java. La única diferencia será la sustitución de la llamada a *println* en Java por el enunciado C estándar

```
printf("Pase un disco de %d a %d\n", i, j)
```

Para nuestros fines, la sintaxis de las cadenas de formato de *printf* no es importante (básicamente, la cadena se imprime literalmente excepto que %d significa imprimir el siguiente entero en formato decimal). Lo único importante aquí es que el procedimiento se invoque con tres parámetros: una cadena de formato y dos enteros.

La razón por la que usamos la versión C para el Pentium II y el UltraSPARC II es que la biblioteca de E/S de Java no está disponible en forma nativa para estas máquinas, mientras que la biblioteca de C sí lo está. Con la JVM usaremos la versión en Java. La diferencia es mínima, y sólo afecta el enunciado de impresión.

5.7.1 Las Torres de Hanoi en lenguaje ensamblador del Pentium II

La figura 5-45 presenta una posible traducción de la versión en C de las Torres de Hanoi para el Pentium II. En su mayor parte, el programa es relativamente sencillo. Se usa el registro EBP como apuntador de la pila. Las primeras dos palabras se usan para enlace, así que el primer parámetro real, *n* (o *N* aquí, ya que MASM no distingue entre mayúsculas y minúsculas), está en EBP + 8, seguido de *i* y *j* en EBP + 12 y EBP + 16, respectivamente. La variable local, *k*, está en EBP + 20.

Lo primero que hace el procedimiento es establecer el nuevo marco al final del anterior. Esto lo hace copiando ESP en el apuntador de marco, EBP. Luego compara *n* con 1, saltando a la cláusula *else* si *n* > 1. El código *then* mete tres valores en la pila: la dirección de la cadena de formato, *i* y *j*, y luego se llama a sí mismo.

Los parámetros se meten en orden inverso, lo cual es necesario para los programas en C a fin de colocar el apuntador a la cadena de formato en el tope de la pila. Puesto que *printf* tiene un número variable de parámetros, si los parámetros se metieran después *printf* no sabría qué tan abajo en la pila está la cadena de formato.

Después de la llamada, se suma 12 a ESP para eliminar los parámetros de la pila. Desde luego, los parámetros no se borran realmente de la memoria, pero el ajuste de ESP los vuelve inaccesibles a través de las operaciones de pila normales.

La cláusula *else*, que inicia en *L1*, es sencilla: primero calcula $6 - i - j$ y guarda este valor en *k*. Sean cuales sean los valores de *i* y *j*, la tercera varilla siempre es $6 - i - j$; guardarla en *k* ahorra el trabajo de recalcularla la segunda vez.

Luego, el procedimiento se llama a sí mismo tres veces, con diferentes parámetros en cada ocasión. Después de cada llamada, la pila se asea. Eso es todo.

Los procedimientos recursivos provocan confusión en mucha gente al principio, pero cuando se estudian en este nivel son de lo más sencillo. Lo único que sucede es que se meten los parámetros en la pila y se invoca el procedimiento.

5.7.2 Las Torres de Hanoi en lenguaje ensamblador del UltraSPARC II

Vamos a intentarlo otra vez, pero ahora para el UltraSPARC II. El código se presenta en la figura 5-46. Dado que el código del UltraSPARC II es muy difícil de entender, aun en comparación con otros códigos de ensamblador y aun después de mucha práctica, nos hemos tomado la libertad de definir unos cuantos símbolos al principio para hacerlo más claro.

```

.586                                     ; compilar para Pentium (no 8088, etc.)
.MODEL FLAT
PUBLIC _torres                          ; exportar 'torres'
EXTERN _printf:NEAR                    ; importar printf
.CODE
_torres: PUSH EBP                       ; guardar EBP (apuntador a marco)
        MOV EBP, ESP                   ; fijar nuevo apunt. a marco arriba de ESP
        CMP [EBP+8], 1                 ; si (n == 1)
        JNE L1                         ; ramificar si n no es 1
        MOV EAX, [EBP+16]              ; printf("...", i, j);
        PUSH EAX                       ; observe que los parámetros i, j y la cadena
        MOV EAX, [EBP+12]              ; de formato se empilan en orden inverso.
        PUSH EAX                       ; Esta es la convención de llamada de C
        PUSH OFFSET FLAT:format        ; offset flat se refiere a la direc. de format
        CALL _printf                  ; llamar a printf
        ADD ESP, 12                    ; quitar parámetros de la pila
        JMP Done                       ; ya terminamos
L1:     MOV EAX, 6                      ; iniciar k = 6 - i - j
        SUB EAX, [EBP+12]              ; EAX = 6 - i
        SUB EAX, [EBP+16]              ; EAX = 6 - i - j
        MOV [EBP+20], EAX              ; k = EAX
        PUSH EAX                       ; iniciar torres(n - 1, i, k)
        MOV EAX, [EBP+12]              ; EAX = i
        PUSH EAX                       ; apilar i
        MOV EAX, [EBP+8]               ; EAX = n
        DEC EAX                        ; EAX = n - 1
        PUSH EAX                       ; apilar n - 1
        CALL _torres                  ; llamar a torres(n - 1, i, 6 - i - j)
        ADD ESP, 12                    ; quitar parámetros de la pila
        MOV EAX, [EBP+16]              ; iniciar torres(1, i, j)
        PUSH EAX                       ; apilar j
        MOV EAX, [EBP+12]              ; EAX = i
        PUSH EAX                       ; apilar i
        PUSH 1                         ; apilar 1
        CALL _torres                  ; llamar torres(1, i, j)
        ADD ESP, 12                    ; quitar parámetros de la pila
        MOV EAX, [EBP+12]              ; iniciar torres(n - 1, 6 - i - j, i)
        PUSH EAX                       ; apilar i
        MOV EAX, [EBP+20]              ; apilar 20
        PUSH EAX                       ; apilar k
        MOV EAX, [EBP+8]               ; EAX = n
        DEC EAX                        ; EAX = n - 1
        PUSH EAX                       ; apilar n - 1
        CALL _torres                  ; llamar torres(n - 1, 6 - i - j, i)
        ADD ESP, 12                    ; ajustar apuntador de pila
        LEAVE                          ; prepararse para salir
        RET 0                          ; regresar al invocador

.DATA
format DB "Pase un disco de %d a %d\n" ; cadena de formato
END

```

Figura 5-45. Las Torres de Hanoi para el Pentium II.

Para que funcione, el programa se tiene que procesar con otro programa llamado *cpp*, el preprocesador de C, antes de ensamblarlo. Además, hemos usado minúsculas aquí porque el ensamblador de UltraSPARC II lo exige (por si algún lector desea introducir el programa y ejecutarlo).

En cuanto al algoritmo, la versión para UltraSPARC II es idéntica a la versión para el Pentium II. Ambas prueban n al principio y saltan a la cláusula *else* si $n > 1$. La complejidad de la versión UltraSPARC II se debe principalmente a ciertas propiedades de la ISA.

Por principio, el UltraSPARC II tiene que pasar la dirección de la cadena de formato a *printf*, pero la máquina no puede limitarse a poner la dirección en el registro que contiene el parámetro de salida porque no hay forma de colocar una constante de 32 bits en un registro con una sola instrucción. Se requieren dos instrucciones para hacerlo, *SETH* y *OR*.

Otra cosa que debemos notar es que no es necesario ajustar la pila después de la llamada porque la instrucción *RESTORE* del final del procedimiento ajusta la ventana de registro. La capacidad para colocar los parámetros de salida en registros y no tener que ir a la memoria aumenta de manera considerable la rapidez si la profundidad de las llamadas no es excesiva, pero en general es probable que la ventaja de contar con el mecanismo de ventanas de registros no justifique el aumento en la complejidad que implica.

Observe ahora la instrucción *NOP* que sigue a la ramificación a *Done*. Se trata de una ranura de retraso. Esta instrucción siempre se ejecutará, aunque sigue a una instrucción de ramificación condicional. El problema es que el UltraSPARC II tiene un conducto profundo, y para cuando el hardware descubre que enfrenta una ramificación, la siguiente instrucción prácticamente ya terminó de ejecutarse. Bienvenido al Mundo Maravilloso de la Programación RISC.

Esta característica tan divertida también se extiende a las llamadas a procedimientos. Observe la primera llamada a *torres* en la cláusula *else*: coloca $n - 1$ en *%o0* y en *%o1*, pero efectúa la llamada a *torres* antes de colocar el último parámetro. En el Pentium II, primero se pasan los parámetros y luego se hace la llamada. Aquí, primero se pasan algunos parámetros, luego se hace la llamada, y luego se pasa el último parámetro. Para cuando la máquina se da cuenta de que enfrenta una instrucción *CALL*, la instrucción que le sigue ya está en un nivel tan profundo del conducto que tiene que ejecutarse. Entonces, ¿por qué no usar la ranura de retraso para pasar el último parámetro? Incluso si la primera instrucción del procedimiento invocado usa ese parámetro, el valor estará en su lugar a tiempo.

Por último, en *Done*, vemos que la instrucción *RET* también tiene una ranura de retraso, la cual aprovechamos para la instrucción *RESTORE*, que incrementa *CWP* a fin de restaurar la ventana de registros a la posición que el invocador espera.

5.7.3 Las Torres de Hanoi en lenguaje ensamblador de la JVM

El código de ensamblador JVM para la versión en Java de las Torres de Hanoi se da en la figura 5-47. La solución es relativamente sencilla, con excepción de la E/S. Este programa se generó con el compilador de Java, se desensambló a lenguaje ensamblador simbólico, se editó para hacerlo más comprensible y se le añadieron comentarios.

```

#define N %i0          /* N es el parámetro de entrada 0 */
#define I %i1          /* I es el parámetro de entrada 1 */
#define J %i2          /* J es el parámetro de entrada 2 */
#define K %i0          /* K es la variable local 0 */
#define Param0 %o0     /* Param0 es el parámetro de salida 0 */
#define Param1 %o1     /* Param1 es el parámetro de salida 1 */
#define Param2 %o2     /* Param2 es el parámetro de salida 2 */
#define Scratch %l1    /* por cierto, cpp usa la convención de comentarios de C */

.proc 04
.global torres

torres: save %sp, -112, %sp
        cmp N, 1
        bne Else

        sethi %hi(format), Param0
        or Param0, %lo(format), Param0
        mov I, Param1
        call printf
        mov J, Param2
        b Done
        nop

Else:    mov 6, K
        sub K, J, K
        sub K, I, K

        add N, -1, Scratch
        mov Scratch, Param0
        mov I, Param1
        call torres
        mov K, Param2

        mov 1, Param0
        mov I, Param1
        call torres
        mov J, Param2

        mov Scratch, Param0
        mov K, Param1
        call torres
        mov J, Param2

Done:    ret
        restore

format: .asciz "Pase un disco de %d a %d\n"

! si (n == 1)
! si (n != 1), ir a Else

! printf("Pase un disco de %d a %d\n", i, j)
! Param0 = dirección de cadena de formato
! Param1 = i
! llamar printf ANTES de preparar parámetro 2 (j)
! usar ranura de retraso para preparar parámetro 2
! ya terminamos
! llenar ranura de retraso

! iniciar k = 6 - i - j
! k = 6 - j
! k = 6 - i - j

! iniciar torres(n - 1, i, k)
! Scratch = N - 1
! parámetro 1 = i
! llamar torres ANTES DE preparar parámetro 2 (k)
! usar ranura de retraso para preparar parámetro 2

! iniciar torres(1, i, j)
! parámetro 1 = i
! llamar torres ANTES DE preparar parámetro 2 (j)
! parámetro 2 = j

! iniciar torres(n - 1, k, j)
! parámetro 1 = k
! llamar torres ANTES DE preparar parámetro 2 (j)
! parámetro 2 = j

! regresar
! usar ranura de retraso para restaurar ventanas

```

Figura 5-46. Las Torres de Hanoi para el UltraSPARC II.

El compilador JVM mantiene los tres parámetros, n , i y j en las variables locales 0, 1 y 2, respectivamente. La variable local, k , se guarda en la 3. Podemos acceder a estas cuatro variables locales usando el código de operación de un byte correspondiente, como `ILOAD_0`. El resultado es que la versión binaria de este programa en JVM es muy corta (67 bytes).

Lo primero que el programa hace es meter n y la constante 1 en la pila, y luego usa `IF_ICMPNE` para compararlos. `IF_ICMPNE` es la prueba inversa de nuestra instrucción `IF_ICMPEQ` de JVM: desempila dos operandos y ramifica si son diferentes.

Si los operandos son iguales, la ejecución continúa con la siguiente instrucción. Las 13 instrucciones que siguen asignan espacio para un buffer de cadena y construyen en él una cadena que al final se pasa a `println` para imprimirla. Una vez terminada la impresión, el procedimiento regresa.

En pocas palabras, lo que estas 13 instrucciones están haciendo es asignar espacio para un buffer de cadena en el montículo y llenarlo. La instrucción `GETSTATIC` usa un índice para obtener la palabra 13 de la reserva de constantes, la cual contiene un apuntador a un descriptor del buffer de cadena. La instrucción `NEW` usa este descriptor para asignar espacio para el buffer de cadena en el montículo. Las siguientes 11 instrucciones concatenan las dos cadenas y dos enteros para formar una sola cadena en este buffer y pasarla a `println`, que la imprime.

Si n no es 1, el control pasa a `L1`. Aquí se calcula k directamente usando aritmética de pila. Luego se efectúan las tres llamadas, una tras otra.

JVM cuenta con diversas instrucciones para invocar procedimientos. Aquí el compilador usó tres distintas. Todas ellas tienen un operando de dos bytes que funciona como índice de la reserva de constantes para encontrar un apuntador a un descriptor que contiene toda la información acerca del procedimiento que se invocará. Las constantes como #10, #11, etc., son índices para la reserva de constantes.

Las diferencias entre los diferentes tipos de llamadas a procedimientos tienen que ver con diferentes optimaciones. Se usa la instrucción `INVOKESTATIC` para invocar métodos estáticos escritos por el usuario, como `torres`. En contraste, `INVOKESPECIAL` sirve para invocar métodos de inicialización, métodos privados y métodos de la superclase de la clase actual. Por último, `INVOKEVIRTUAL` se usa para llamadas internas (de biblioteca).

5.8 EL IA-64 DE INTEL

Intel está llegando rápidamente a un punto en el que se ha exprimido hasta la última gota de jugo que puede sacarse a la ISA IA-32 y a la línea de procesadores Pentium II. Los nuevos modelos todavía pueden sacar ventaja de los adelantos en la tecnología de fabricación, lo que significa transistores más pequeños (y por ende velocidades de reloj más altas). Sin embargo, encontrar nuevos trucos para hacer más rápida la implementación es cada vez más difícil, y las restricciones impuestas por la ISA IA-32 son un obstáculo cada día mayor.

La única solución real consiste en abandonar la IA-32 como línea principal de desarrollo y cambiar a una ISA totalmente distinta. De hecho, esto es lo que Intel piensa hacer. La nueva arquitectura, desarrollada en forma conjunta por Intel y Hewlett-Packard, se llama **IA-64**, y es una máquina de 64 bits de principio a fin. En los próximos años se espera la aparición de toda una serie de CPU que implementen esta arquitectura. La primera implementación, cuyo

```

ILOAD_0                // local 0 = n; apilar n
ICONST_1               // apilar 1
IF_ICMPNE L1           // si (n != 1), ir a L1

GETSTATIC #13          // n == 1; este código maneja el enunc. println
NEW #7                // asignar buffer para la cadena a construir
DUP                   // duplicar apuntador al buffer
LDC #2                // apilar apunt. a cadena "pase un disco de "
INVOKESPECIAL #10      // copiar la cadena en el buffer
ILOAD_1               // apilar i
INVOKEVIRTUAL #11      // convertir i en cadena y anexar a nuevo buffer
LDC #1                // apilar apuntador a cadena " a "
INVOKEVIRTUAL #12      // anexar esta cadena al buffer
ILOAD_2               // apilar j
INVOKEVIRTUAL #11      // convertir j en cadena y anexar a buffer
INVOKEVIRTUAL #15      // conversión en cadena
INVOKEVIRTUAL #14      // invocar println
RETURN                // regresar de torres

L1: BIPUSH 6           // parte Else: calcular k = 6 - i - j
ILOAD_1               // local 1 = i; apilar i
ISUB                  // tope de pila = 6 - i
ILOAD_2               // local 2 = j; apilar j
ISUB                  // tope de pila = 6 - i - j
ISTORE_3              // local 3 = k = 6 - i - j; la pila está vacía

ILOAD_0               // iniciar trabajos de torres(n - 1, i, k); apilar n
ICONST_1              // apilar 1
ISUB                  // tope de pila = n - 1
ILOAD_1               // apilar i
ILOAD_3               // apilar k
INVOKESTATIC #16      // invocar torres(n - 1, 1, k)

ICONST_1              // iniciar trabajos de torres(1, i, k); apilar 1
ILOAD_1               // apilar i
ILOAD_2               // apilar j
INVOKESTATIC #16      // invocar torres(1, i, j)

ILOAD_0               // iniciar trabajos de torres(n - 1, k, j); apilar n
ICONST_1              // apilar 1
ISUB                  // tope de pila = n - 1
ILOAD_3               // apilar k
ILOAD_2               // apilar j
INVOKESTATIC #16      // invocar torres(n - 1, k, j)
RETURN                // regresar de torres

```

Figura 5-47. Las Torres de Hanoi para JVM.

nombre en código es **Merced** (según el río californiano favorito de alguien), es una CPU de extremo superior, pero tarde o temprano aparecerá toda una gama de CPU, desde servidores de extremo superior hasta modelos económicos para escritorio.

Dada la importancia que para la industria de las computadoras tiene todo lo que Intel hace, vale la pena dedicar una mirada a la arquitectura IA-64 (y por implicación al Merced).

Sin embargo, dejando eso a un lado, los investigadores en este campo ya conocen bien las ideas fundamentales en que se apoya la IA-64, y bien podrían aparecer en otros diseños. De hecho, algunas de ellas ya están presentes en diversas formas en sistemas (experimentales). En las secciones que siguen examinaremos la naturaleza de los problemas que Intel enfrenta, el modelo que la IA-64 adoptó para resolverlos, el funcionamiento de algunas de las ideas fundamentales, y lo que podría suceder.

5.8.1 El problema del Pentium II

La raíz de todos los problemas es el hecho ineludible de que la IA-32 es una ISA antigua cuyas propiedades son totalmente inadecuadas para la tecnología actual. Es una ISA CISC con instrucciones de longitud variable y una multitud de formatos distintos que son difíciles de decodificar rápidamente sobre la marcha. La tecnología actual funciona de manera óptima con las ISA RISC que tienen una sola longitud de instrucción y un código de operación de longitud fija fácil de decodificar. Las instrucciones de la IA-32 pueden descomponerse en microoperaciones tipo RISC en el momento de la ejecución, pero ello requiere hardware (área de chip), consume tiempo y aumenta la complejidad del diseño. Ése es el primer *strike*.*

También, la IA-32 es una ISA de dos direcciones orientada hacia la memoria. Casi todas las instrucciones hacen referencia a la memoria, y pocos programadores dudan en hacer referencias a la memoria a diestra y siniestra. La tecnología actual favorece a las ISA de carga/almacenamiento que sólo hacen referencia a la memoria para obtener los operandos y colocarlos en registros, pero por lo demás realizan todos sus cálculos con instrucciones de tres direcciones que manejan registros. Y en vista del aumento mucho mayor de las velocidades de reloj de las CPU en comparación con las de memoria, el problema va a empeorar con el tiempo. Éste es el segundo *strike*.

Además, la IA-32 tiene un conjunto de registros pequeño e irregular. Esto no sólo hace que los compiladores parezcan contorsionistas, sino que el número tan reducido de registros de propósito general (cuatro o seis, dependiendo de cómo se cuenten ESI y EDI) obliga a verter a la memoria continuamente los resultados intermedios, lo que genera referencias a la memoria adicionales aun cuando ni siquiera se necesitan lógicamente. Ése es el tercer *strike*. La IA-32 está *out*.

Pasemos ahora a la segunda entrada (*inning*). La escasez de registros causa muchas dependencias, sobre todo dependencias WAR innecesarias porque los resultados tienen que colocarse en algún lado y no hay registros extra disponibles. Para subsanar la falta de registros, la implementación tiene que efectuar cambios de nombres internamente —un parche horrible— a registros secretos dentro del buffer de reordenamiento. Para evitar bloqueos con demasiada frecuencia cuando hay fallos de caché, las instrucciones tienen que ejecutarse en desorden. Sin embargo, la semántica de la IA-32 especifica interrupciones precisas, por lo que las instrucciones en desorden tienen que retirarse en orden. Todo esto requiere una gran cantidad de hardware muy complejo. Cuarto *strike*.

Para poder efectuar todo este trabajo rápidamente se requiere una fila de procesamiento muy grande (de 12 etapas). A su vez, la fila de procesamiento implica que las instrucciones ingresan en él 11 ciclos antes de que terminen. Por ello, es indispensable una predicción de ramificaciones muy exacta para asegurarse de que las instrucciones correctas ingresen en el

conducto. Dado que una predicción errónea obliga a vaciar la fila de procesamiento, lo cual es una operación muy costosa, los errores de predicción no tienen que ser muy frecuentes para que el desempeño sufra una degradación sustancial. Quinto *strike*.

Para aliviar los problemas que causan las predicciones erróneas, el procesador necesita efectuar ejecución especulativa, con todos los problemas que conlleva, sobre todo cuando referencias a la memoria en la rama equivocada causan una excepción. Sexto *strike*.

No vamos a jugar el partido de béisbol completo aquí, pero a estas alturas ya debe ser obvio que existe un problema real. Y ni siquiera hemos mencionado el hecho de que las direcciones de 32 bits de la IA-32 limitan los programas individuales a 4 GB de memoria, lo cual es un problema cada vez más grave en los servidores de extremo superior.

En síntesis, la situación de la IA-32 puede compararse favorablemente con el estado de la mecánica celeste justo antes de Copérnico. La teoría que entonces dominaba la astronomía era que la Tierra estaba fija e inmóvil en el espacio y que los planetas se movían a su alrededor en círculos con epiciclos. Sin embargo, a medida que mejoraban las observaciones y era posible detectar más claramente desviaciones respecto a este modelo, se añadieron epiciclos a los epiciclos hasta que el modelo completo se vino abajo por su propia complejidad interna.

Intel está en un aprieto comparable. Una fracción enorme de todos los transistores del Pentium II se dedica a descomponer instrucciones CISC, averiguar qué puede hacerse en paralelo, resolver conflictos, hacer predicciones, reparar las consecuencias de predicciones equivocadas y otras tareas contables, con lo que queda una cantidad sorprendentemente reducida de transistores para efectuar el trabajo real que el usuario solicitó. La conclusión a la que está siendo llevado inexorablemente Intel es la única conclusión razonable: tirar todo (la IA-32) a la basura y comenzar otra vez desde cero (IA-64).

5.8.2 El modelo IA-64: Computación con instrucciones explícitamente paralelas

El punto de partida para la IA-64 fue un procesador RISC de 64 bits del extremo superior (del cual el UltraSPARC II es uno de muchos ejemplos actuales). Puesto que el IA-64 se diseñó junto con Hewlett-Packard, es casi seguro que la arquitectura PA-RISC de HP haya sido la base real, pero todas las máquinas RISC de vanguardia son lo bastante parecidas como para que la descripción que dimos del UltraSPARC II aplique en muchos aspectos. El Merced es un procesador de modo dual, que puede ejecutar programas tanto IA-32 como IA-64, pero en lo que sigue nos concentraremos únicamente en la parte IA-64.

La arquitectura IA-64 es una arquitectura de carga/almacenamiento con direcciones de 64 bits y registros de 64 bits de anchura. Hay 64 registros generales que los programas IA-64 pueden usar (con registros adicionales para los programas IA-32). Todas las instrucciones tienen el mismo formato fijo: un código de operación, dos campos de registro fuente de 6 bits, un campo de registro de destino de 6 bits y otro campo de 6 bits que explicaremos más adelante. Casi todas las instrucciones aceptan dos operandos en registros, realizan algún cálculo con ellos, y colocan el resultado en el registro de destino. Hay muchas unidades funcionales que realizan diferentes operaciones en paralelo. Hasta aquí, nada fuera de lo común. Casi todas las máquinas RISC tienen una arquitectura muy parecida.

*El autor está usando aquí la terminología de un juego de béisbol. (N. del T.)

Lo que es inusitado es la idea de un haz de instrucciones relacionadas. Las instrucciones vienen en grupos de tres, llamados **haces**, como se muestra en la figura 5-48. Cada haz de 128 bits contiene tres instrucciones de formato fijo de 40 bits y una plantilla de 8 bits. Los haces pueden encadenarse mediante un bit de fin de haz, por lo que puede haber más de tres instrucciones presentes en un haz. La plantilla contiene información acerca de cuáles instrucciones se pueden ejecutar en paralelo. Este esquema, más la abundancia de registros, permite al compilador aislar bloques de instrucciones y decirle a la CPU que se pueden ejecutar en paralelo. Así, el compilador debe reordenar las instrucciones, detectar dependencias, asegurarse de que haya unidades funcionales disponibles, etc., en lugar de que lo haga el hardware. La idea es que al exponer el funcionamiento interno de la máquina y pedir a los escritores de compiladores que se aseguren de que cada haz consta de instrucciones compatibles, la tarea de planificar las instrucciones RISC se traslada del hardware (en el momento de la ejecución) al compilador (en el momento de la compilación). Por esta razón, el modelo se llama **computación con instrucciones explícitamente paralelas (EPIC, Explicitly Parallel Instruction Computing)**.

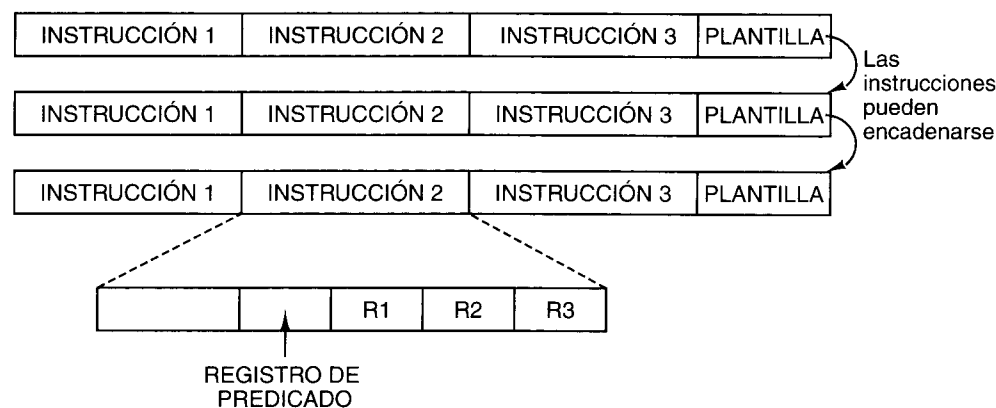


Figura 5-48. La IA-64 se basa en haces de tres instrucciones.

Planificar las instrucciones en el momento de la compilación tiene varias ventajas. Primera, puesto que ahora el compilador está haciendo todo el trabajo, el hardware puede ser mucho más sencillo, lo que ahorra millones de transistores para otras funciones útiles, como cachés de nivel 1 más grandes. Segunda, para un programa dado la planificación sólo tiene que realizarse una vez, en el momento de la compilación. Tercera, puesto que el compilador efectúa todo el trabajo, un proveedor de software puede usar un compilador que dedique horas a optimizar su programa y así beneficiar a todos los usuarios cada vez que se ejecute el programa.

La idea de haces de instrucciones puede servir para crear toda una familia de procesadores. En las CPU de extremo inferior se podría emitir un haz cada ciclo de reloj. La CPU podría esperar hasta que todas las instrucciones hayan terminado antes de emitir el siguiente haz. En las CPU del extremo alto podría ser posible emitir varios haces durante el mismo ciclo de reloj, como en los diseños superescalares actuales. Además, en esas CPU, el procesador po-

dría comenzar a emitir instrucciones de un nuevo haz antes de que terminen todas las instrucciones del haz anterior. Desde luego, tendría que verificar si los registros y la unidad funcional requeridos están disponibles, pero no tiene que verificar si otras instrucciones de su propio haz están en conflicto con la instrucción actual porque el compilador ya garantizó que no sucedería.

5.8.3 Predicación

Otra característica importante de IA-64 es el novedoso modo en que maneja las ramificaciones condicionales. Si hubiera alguna forma de deshacerse de casi todas ellas, las CPU podrían ser mucho más sencillas y rápidas. A primera vista podría parecer que es imposible deshacerse de las ramificaciones condicionales porque los programas están llenos de enunciados **if**. No obstante, la IA-64 usa una técnica llamada **predicación** que puede reducir su número considerablemente (August *et al.*, 1998; Hwu, 1998). A continuación la describiremos brevemente.

En las máquinas actuales, todas las instrucciones son incondicionales en el sentido de que cuando la CPU llega a una instrucción simplemente la ejecuta. No hay ningún debate interno del tipo de "Ejecutar o no ejecutar, ¡he ahí el dilema!". En cambio, en una arquitectura predicativa, las instrucciones contienen condiciones (predicados) que indican cuándo deben ejecutarse y cuándo no. Este cambio de paradigma, de instrucciones incondicionales a instrucciones predicativas, es lo que permite deshacerse de (muchas) ramificaciones condicionales. En lugar de tener que escoger entre una sucesión de instrucciones incondicionales y otra sucesión de instrucciones incondicionales, todas las instrucciones se fusionan en una sola sucesión de instrucciones predicativas, empleando diferentes predicados para diferentes instrucciones.

Para ver cómo funciona la predicción, comencemos con el sencillo ejemplo de la figura 5-49, que muestra la **ejecución condicional**, una precursora de la predicción. En la figura 5-49(a) vemos un enunciado **if**. En la figura 5-49(b) vemos su traducción a tres instrucciones: una comparación, una ramificación condicional y una instrucción de traslado.

if (R1 == 0)	CMP R1,0	CMOVZ R2,R3,R1
R2 = R3;	BNE L1	
	MOV R2,R3	
	L1:	
(a)	(b)	(c)

Figura 5-49. (a) Un enunciado **if**. (b) Código de ensamblador genérico para (a). (c) Una instrucción condicional.

En la figura 5-49(c) nos deshacemos de la ramificación condicional empleando una instrucción nueva, **CMOVZ**, que es un traslado condicional. Lo que esta instrucción hace es verificar si el tercer registro, **R1**, es cero. Si lo es, se copia **R3** en **R2**; si no, no se hace nada.

Una vez que tenemos una instrucción que puede copiar datos cuando algún registro es cero, no es difícil diseñar una instrucción que pueda copiar datos cuando algún registro no

sea cero, digamos **CMOVN**. Si contamos con estas dos instrucciones, ya no nos falta mucho para lograr una ejecución plenamente condicional. Imagine un enunciado **if** que tiene varias asignaciones en la parte **then** y varias asignaciones en la parte **else**. El enunciado completo se puede traducir en código que pone en cero algún registro si la condición no se cumple y le asigna otro valor si la condición se cumple. Después de la preparación de los registros, las asignaciones de la parte **then** se pueden compilar a una secuencia de instrucciones **CMOVN**, y las de la parte **else**, a una secuencia de instrucciones **CMOVZ**.

Todas estas instrucciones (la preparación de registros, las **CMOVN** y las **CMOVZ**) forman un solo bloque básico sin ramificaciones condicionales. Hasta es posible reordenar las instrucciones, sea que lo haga el compilador (que incluso podría colocar las asignaciones antes de la prueba) o durante la ejecución. El único requisito es que la condición tiene que conocerse en el momento en que hay que retirar las instrucciones condicionales (cerca del final de la fila de procesamiento). En la figura 5-50 se da un ejemplo sencillo con una parte **then** y una parte **else**.

if (R1 == 0) {	CMP R1,0	CMOVZ R2,R3,R1
R2 = R3;	BNE L1	CMOVZ R4,R5,R1
R4 = R5;	MOV R2,R3	CMOVN R6,R7,R1
} else {	MOV R4,R5	CMOVN R8,R9,R1
R6 = R7;	BR L2	
R8 = R9;	L1: MOV R6,R7	
}	MOV R8,R9	
	L2:	
(a)	(b)	(c)

Figura 5-50. (a) Un enunciado **if**. (b) Código de ensamblador genérico para (a). (c) Ejecución condicional.

Aunque aquí sólo hemos mostrado instrucciones condicionales muy sencillas (tomadas del Pentium II, de hecho), en la IA-64 todas las instrucciones son predicativas. Esto implica que se puede hacer condicional la ejecución de cada instrucción. El campo de 6 bits extra que mencionamos antes selecciona uno de los 64 registros de predicado de un bit. Así, un enunciado **if** se compila para dar código que pone en 1 uno de los registros de predicado si la condición se cumple, y lo pone en 0 si no se cumple. De forma simultánea y automática, el código asigna el valor opuesto a otro registro de predicado. Mediante la predicación, las instrucciones de máquina que forman las cláusulas **then** y **else** se fusionan en un solo flujo de instrucciones, de las cuales las primeras usan el predicado y las segundas usan su inverso.

Aunque sencillo, el ejemplo de la figura 5-51 ilustra la idea en la que se basa el uso de la predicación para eliminar ramificaciones. La instrucción **CMPEQ** compara dos registros y pone en 1 el registro de predicado **P4** si son iguales, y en 0 si son diferentes; además, asigna el valor opuesto a un registro apareado, digamos **P5**. Ahora las instrucciones de las partes **if** y **then** se pueden poner una tras otra, cada una condicionada a algún registro de predicado (que se indica entre paréntesis angulares). Se puede colocar aquí cualquier código, siempre que cada instrucción tenga el predicado correcto.

if (R1 == R2)	CMP R1,R2	CMPEQ R1,R2,P4
R3 = R4 + R5;	BNE L1	<P4> ADD R3,R4,R5
else	MOX R3,R4	<P5> SUB R6,R4,R5
R6 = R4 - R5	ADD R3,R5	
	BR L2	
	L1: MOV R6,R4	
	SUB R6,R5	
	L2:	
(a)	(b)	(c)

Figura 5-51. (a) Un enunciado **if**. (b) Código de ensamblador genérico para (a). (c) Ejecución predicativa.

En la IA-64, esta idea se lleva al extremo, con instrucciones de comparación para ajustar los registros de predicado además de instrucciones aritméticas y de otro tipo cuya ejecución depende de algún registro de predicado. Las instrucciones predicativas se pueden insertar en la fila de procesamiento una tras otra, sin paros ni problemas. Es por ello que son tan útiles.

La forma en que funciona realmente la predicación en la IA-64 es que todas las instrucciones se ejecutan realmente. Al final de la fila de procesamiento, cuando llega el momento de retirar una instrucción, se verifica si el predicado es 1; en tal caso, la instrucción se retira normalmente y sus resultados se escriben en el registro de destino. Si el predicado es 0, no se escribe el resultado y la instrucción no tiene efectos. La predicación se explica con mayor detalle en (Dulong, 1998).

5.8.4 Cargas especulativas

Otra característica de la IA-64 que acelera la ejecución es la presencia de instrucciones **LOAD** especulativas. Si un **LOAD** es especulativo y falla, no causa una excepción, sino que se detiene y enciende un bit asociado al registro que se iba a cargar, para marcar ese registro como no válido. Éste es el bit venenoso que introdujimos en el capítulo 4. Si posteriormente se usa el registro envenenado, la excepción ocurre en ese momento; de lo contrario, nunca ocurre.

La forma en que se usa normalmente la especulación es que el compilador eleva las **LOAD** a posiciones anteriores a los puntos en que se necesitan. Al iniciarse antes, pueden completarse antes de que se necesiten los resultados. En el lugar en que el compilador necesita usar el registro recién cargado, inserta una instrucción **CHECK**. Si el valor está ahí, **CHECK** actúa como **NOP** y la ejecución continúa de inmediato. Si el valor todavía no ha llegado, la siguiente instrucción deberá parar. Si ocurrió una excepción y el bit venenoso está encendido, la excepción pendiente ocurrirá en ese punto.

En síntesis, una máquina que implementa la arquitectura IA-64 obtiene su velocidad de varias fuentes. En lo fundamental, se trata de una máquina RISC de vanguardia con filas de procesamiento, de carga/almacenamiento, y de tres direcciones. Además, la IA-64 tiene un modelo de paralelismo explícito que exige al compilador determinar cuáles instrucciones se pueden ejecutar al mismo tiempo sin conflictos y agruparlas en haces. Así, la CPU podrá programar ciegamente un haz sin tener que romperse la cabeza pensando. Luego, la predica-

ción permite fusionar en un solo flujo los enunciados de ambas ramas de un enunciado if, con lo que se elimina la ramificación condicional y por ende la predicción correspondiente. Por último, las instrucciones **LOAD** especulativas permiten traer los operandos con anticipación, sin incurrir en un castigo si resulta que en realidad no se necesitaban.

5.8.5 Los pies en la tierra

Si todas estas cosas funcionan como se espera, Merced en verdad será una máquina potente. No obstante, debemos expresar ciertas reservas. En primer lugar, nunca se ha construido una máquina tan avanzada, al menos no para un mercado masivo. La historia muestra que incluso los planes mejor fraguados a veces se frustran por diversas razones.

En segundo lugar, va a ser necesario escribir compiladores para la IA-64. Escribir un buen compilador para esta arquitectura no será fácil. Además, durante los últimos 30 años el Santo Grial de las investigaciones en programación paralela ha sido la extracción automática de paralelismo de programas secuenciales. Estos trabajos no han tenido mucho éxito. Si un programa no tiene mucho paralelismo inherente, o si el compilador no puede extraerlo, todos los haces de la IA-64 serán cortos, y no se ganará mucho.

En tercer lugar, todo lo que dijimos en las secciones anteriores supone que el sistema operativo es cabalmente de 64 bits. Windows 95 y Windows 98 ciertamente no lo son y probablemente nunca lo serán. Esto implica que todo mundo va a tener que cambiar a Windows NT o a alguna versión de UNIX. Semejante transición podría ser dolorosa. Después de todo, a 10 años de la introducción del 386 de 32 bits, Windows 95 aún contenía una buena cantidad de código para 16 bits, y nunca usó el hardware de segmentación que ha estado incluido en todas las CPU de Intel desde hace más de una década (veremos la segmentación en el capítulo 6). No sabemos cuánto tardarán los sistemas operativos en convertirse en sistemas de 64 bits cabales.

En cuarto lugar, mucha gente va a juzgar a la IA-64 por lo bien que ejecute viejos juegos de 16 bits para MS-DOS. Cuando salió el Pentium Pro, fue criticado en la prensa porque no ejecutaba los programas viejos de 16 bits más rápidamente que el Pentium normal. Puesto que los programas viejos de 16 bits (o incluso de 32 bits) ni siquiera usarán las nuevas características de las CPU IA-64, todos los registros de predicado del mundo de nada servirán. Para observar mejoras, la gente tendrá que comprar modernizaciones de todo su software, compiladas con los nuevos compiladores en modo IA-64. Muchos de esos usuarios se resistirán a efectuar el desembolso.

Por último, podría haber alternativas de otros fabricantes (incluido Intel) que también ofrezcan alto desempeño empleando arquitecturas RISC más convencionales, tal vez con más instrucciones condicionales en una forma más débil de predicación. Hasta las versiones más rápidas de la línea Pentium II misma podrían ser importantes rivales de la IA-64. Es probable que pasen muchos años antes de que la IA-64 alcance el tipo de dominio del mercado que la IA-32 tiene ahora, y podría no suceder nunca.

5.9 RESUMEN

El nivel de arquitectura del conjunto de instrucciones es lo que la mayoría de las personas conoce como “lenguaje de máquina”. En este nivel, la máquina tiene una memoria orientada a bytes o palabras que consiste en algunas decenas de megabytes, e instrucciones como **MOVE**, **ADD** y **BEQ**.

Casi todas las computadoras modernas tienen una memoria organizada en forma de una secuencia de bytes, los cuales se agrupan en palabras de 4 u 8 bytes cada una. También es normal contar con entre 8 y 32 registros, cada uno de los cuales puede contener una palabra. En algunas máquinas (como el Pentium II), las referencias a palabras de memoria no tienen que estar alineadas respecto a las fronteras naturales de la memoria, mientras que en otras (como el UltraSPARC II) es obligatorio.

Las instrucciones generalmente tienen uno, dos o tres operandos, que se direccionan empleando modos de direccionamiento directo, de registro, indexado, etc. Algunas máquinas tienen un gran número de modos de direccionamiento complejos. Por lo regular se cuenta con instrucciones para trasladar datos, efectuar operaciones diádicas o monádicas que incluyen operaciones aritméticas y booleanas, ramificaciones, llamadas a procedimientos, y ciclos, y a veces para E/S. Una instrucción representativa traslada una palabra de la memoria a un registro (o viceversa), suma, resta, multiplica o divide dos registros o un registro y una palabra de memoria, o compara dos valores que están en registros o en la memoria. No es inusitado que una computadora tenga más de 200 instrucciones en su repertorio.

El flujo de control en el nivel 2 se implementa utilizando diversas primitivas que incluyen ramificaciones, llamadas a procedimientos, llamadas a corrutinas, trampas e interrupciones. Las ramificaciones sirven para terminar una secuencia de instrucciones e iniciar una nueva. Los procedimientos se usan como mecanismo de abstracción, a fin de aislar una parte del programa en una unidad e invocarla desde varios puntos. Las corrutinas permiten a dos hilos de control operar simultáneamente. Las trampas sirven para avisar de situaciones excepcionales, como un desbordamiento aritmético. Las interrupciones permiten efectuar E/S en paralelo con los cálculos principales, ya que la CPU recibe una señal tan pronto como se completa la E/S.

Examinamos una buena solución recursiva del divertido problema de las Torres de Hanoi.

Por último, la arquitectura IA-64 usa el modelo de computación EPIC que facilita la explotación del paralelismo por parte de los programas. Además, usa predicación y cargas especulativas para aumentar su velocidad. Esto podría representar un adelanto importante respecto al Pentium II, pero relega una buena parte del trabajo de paralelización al compilador.

PROBLEMAS

1. En el Pentium II las instrucciones pueden contener cualquier número de bytes, par o impar. En el UltraSPARC II todas las instrucciones contienen un número entero de palabras, es decir, un número par de bytes. Cite una ventaja del esquema del Pentium II.

2. Diseñe un código de operación expansible que permita codificar lo siguiente en una instrucción de 36 bits:
 - 7 instrucciones con dos direcciones de 15 bits y un número de registro de 3 bits.
 - 500 instrucciones con una dirección de 15 bits y un número de registro de 3 bits.
 - 50 instrucciones sin direcciones ni registros.
3. ¿Es posible diseñar un código de operación expansible que permita codificar lo siguiente en una instrucción de 12 bits? La especificación de un registro requiere 3 bits.
 - 4 instrucciones con 3 registros
 - 255 instrucciones con un registro
 - 16 instrucciones con cero registros.
4. Cierta máquina tiene instrucciones de 16 bits y direcciones de 6 bits. Algunas instrucciones tienen una dirección y otras tienen dos. Si hay n instrucciones de dos direcciones, ¿cuántas instrucciones de una dirección puede haber como máximo?
5. Dados los valores de memoria que siguen y una máquina de una dirección con acumulador, ¿qué valores cargan en el acumulador las instrucciones siguientes?
 - la palabra 20 contiene 40
 - la palabra 30 contiene 50
 - la palabra 40 contiene 60
 - la palabra 50 contiene 70
 - a. LOAD IMMEDIATE 20
 - b. LOAD DIRECT 20
 - c. LOAD INDIRECT 20
 - d. LOAD IMMEDIATE 30
 - e. LOAD DIRECT 30
 - f. LOAD INDIRECT 30
6. Compare las máquinas de 0, 1, 2 y 3 direcciones escribiendo programas que calculen

$$X = (A + B \times C) / (D - E \times F)$$

para cada una de las cuatro máquinas. Las instrucciones que se pueden usar son:

0 direcciones	1 dirección	2 direcciones	3 direcciones
PUSH M	LOAD M	MOV (X = Y)	MOV (X = Y)
POP M	STORE M	ADD (X = X+Y)	ADD (X = Y+Z)
ADD	ADD M	SUB (X = X-Y)	SUB (X = Y-Z)
SUB	SUB M	MUL (X = X*Y)	MUL (X = Y*Z)
MUL	MUL M	DIV (X = X/Y)	DIV (X = Y/Z)
DIV	DIV M		

M es una dirección de memoria de 16 bits, y X , Y y Z son direcciones de 16 bits o bien registros de 4 bits. La máquina de cero direcciones usa una pila, la máquina de una dirección usa un acumulador, y las otras dos tienen 16 registros e instrucciones que operan con todas las combinaciones posibles de localidades de memoria y registros. SUB X, Y resta Y a X , y SUB X, Y, Z resta Z a Y y coloca el resultado en X . Suponiendo códigos de operación e instrucciones cuyas longitudes son múltiplos de 4 bits, ¿cuántos bits necesita cada máquina para calcular X ?

7. Diseñe un mecanismo de direccionamiento que permita especificar en un campo de 6 bits un conjunto arbitrario de 64 direcciones, no necesariamente contiguas, de un espacio de direcciones grande.
8. Sugiera una desventaja del código automodificante que no se haya mencionado en el texto.
9. Convierta las siguientes fórmulas de notación infija a notación polaca inversa.
 - a. $A+B+C+D+E$
 - b. $(A+B) \times (C+D)+E$
 - c. $(A \times B) + (C \times D) + E$
 - d. $(A+B) \times (((C-D \times E) / F) / G) \times H$
10. Convierta las siguientes fórmulas de notación polaca inversa a notación infija.
 - a. $AB+C+D \times$
 - b. $AB/CD/+$
 - c. $ABCDE+ \times \times /$
 - d. $ABCDE \times F/+G-H/\times +$
11. ¿Cuáles de los siguientes pares de fórmulas en notación polaca inversa son matemáticamente equivalentes?
 - a. $AB+C+$ y $ABC++$
 - b. $AB-C-$ y $ABC--$
 - c. $AB \times C+$ y $ABC+\times$
12. Escriba tres fórmulas en notación polaca inversa que no puedan convertirse a notación infija.
13. Convierta las siguientes fórmulas booleanas infijas a notación polaca inversa.
 - a. $(A \text{ AND } B) \text{ OR } C$
 - b. $(A \text{ OR } B) \text{ AND } (A \text{ OR } C)$
 - c. $(A \text{ AND } B) \text{ OR } (C \text{ AND } D)$
14. Convierta la siguiente fórmula infija a notación polaca inversa y genere código JVM para evaluarla.

$$(2 \times 3 + 4) - (4 / 2 + 1)$$
15. La instrucción de lenguaje ensamblador

MOV REG, ADDR

 significa "cargar un registro de la memoria" en el Pentium II. En cambio, si queremos cargar un registro de la memoria en el UltraSPARC II escribimos

LOAD ADDR, REG

 ¿Por qué es diferente el orden de los operandos?
16. ¿Cuántos registros tiene la máquina cuyos formatos de instrucción se dan en la figura 5-24?
17. En la figura 5-24, el bit 23 sirve para distinguir el uso del formato 1 y el formato 2, pero ningún bit indica el uso del formato 3. ¿Cómo sabe el hardware cuándo debe usarlo?
18. En programación es común que un programa necesite determinar dónde está una variable X respecto al intervalo de A a B . Si se contara con una instrucción de tres direcciones con operandos A , B y X , ¿cuántos bits de código de condición tendría que ajustar esta instrucción?

19. El Pentium II tiene un bit de código de condición que sigue la pista al acarreo de salida del bit 3 después de una operación aritmética. ¿De qué sirve?
20. El UltraSPARC II no tiene ninguna instrucción para cargar un número de 32 bits en un registro. En vez de ello, normalmente se usa una secuencia de dos instrucciones, **SETHI** y **ADD**. ¿Hay más de una forma de cargar un número de 32 bits en un registro? Comente su respuesta.
21. Uno de sus amigos irrumpe en su habitación a las 3 A.M., jadeando, para contarle a usted la brillante idea que se le acaba de ocurrir: una instrucción con dos códigos de operación. ¿A dónde debe enviar a su amigo, a la oficina de patentes o al psiquiatra?
22. Las pruebas de la forma


```
if (n == 0) ...
if (i > j) ...
if (k < 4) ...
```

 son comunes en programación. Genere una instrucción que realice tales pruebas de forma eficiente. ¿Qué campos contiene su instrucción?
23. Para el número binario de 16 bits 1001 0101 1100 0011, muestre el efecto de:
 - a. Un desplazamiento de 4 bits a la derecha rellenando con ceros.
 - b. Un desplazamiento de 4 bits a la derecha extendiendo el signo.
 - c. Un desplazamiento de 4 bits a la izquierda.
 - d. Una rotación de 4 bits a la izquierda.
 - e. Una rotación de 4 bits a la derecha.
24. ¿Cómo puede borrar una palabra de memoria en una instrucción que no cuenta con una instrucción CLR?
25. Calcule la expresión booleana $(A \text{ AND } B) \text{ OR } C$ para


```
A = 1101 0000 1010 1101
B = 1111 1111 0000 1111
C = 0000 0000 0010 0000
```
26. Diseñe una forma de intercambiar dos variables A y B sin usar una tercera variable o registro. *Sugerencia:* piense en la operación **OR EXCLUSIVO**.
27. En cierta computadora es posible trasladar un número de un registro a otro, desplazar cada uno a la izquierda diferentes números de bits, y sumar los resultados en menos tiempo del que tarda una multiplicación. ¿En qué condiciones es útil esta sucesión de instrucciones para calcular “constante $\times$ variable”?
28. Las diferentes máquinas tienen diferentes densidades de instrucciones (número de bytes requeridos para realizar cierto cálculo). Traduzca cada uno de los siguientes fragmentos de código en Java a lenguaje ensamblador de Pentium II, lenguaje ensamblador de UltraSPARC II, y JVM. Luego determine cuántos bytes requiere la expresión en cada máquina. Suponga que i y j son variables locales en la memoria, pero por lo demás haga los supuestos más optimistas en todos los casos.
 - a. $i = 3$;
 - b. $i = j$;
 - c. $i = j - 1$;

29. Las instrucciones de ciclos que vimos en el texto servían para manejar ciclos **for**. Diseñe una instrucción que sirva para manejar ciclos **while** comunes.
30. Suponga que los monjes de Hanoi pueden mover un disco por minuto (no tienen prisa por terminar el trabajo porque las oportunidades de empleo para personas con esta habilidad en particular son limitadas en Hanoi). ¿Cuánto tardarán en resolver el problema con los 64 discos? Expresé su resultado en años.
31. ¿Por qué los dispositivos de E/S colocan el vector de interrupción en el bus? Sería posible guardar esa información en una tabla en la memoria en vez de colocarla en el bus?
32. Una computadora usa DMA para leer de su disco. El disco tiene 64 sectores de 512 bytes en cada pista. El tiempo de rotación del disco es de 16 ms. El bus tiene 16 bits de anchura, y las transferencias de bus tardan 500 ns cada una. Una instrucción de CPU representativa requiere dos ciclos de bus. ¿Cuánto frena el DMA a la CPU?
33. ¿Por qué se asignan prioridades a las rutinas de servicio de interrupción pero no a los procedimientos normales?
34. La arquitectura IA-64 contiene un número inusualmente alto de registros (64). ¿La decisión de tener tantos registros tiene que ver con el uso de la predicación? Si es así, ¿en qué sentido? Si no, ¿por qué hay tantos?
35. En el texto se explica el concepto de instrucciones **LOAD** especulativas. Sin embargo, no se mencionan instrucciones **STORE** especulativas. ¿Por qué no? ¿Son en lo esencial iguales a las instrucciones **LOAD** especulativas o hay alguna otra razón para no mencionarlas?
36. Cuando se desea conectar dos redes de área local, se inserta entre ellas una computadora llamada puente, conectada a ambas redes. Cada paquete que se transmite por cualquiera de las redes causa una interrupción en el puente, para que éste vea si tiene que reenviar el paquete o no. Suponga que se requieren 250 μ s por paquete para manejar la interrupción e inspeccionar el paquete, pero su reenvío, si es necesario, corre por cuenta de hardware de DMA y es “gratuito” para la CPU. Si todos los paquetes son de 1K bytes, ¿qué tasa de datos máxima puede tolerarse en cada red sin que el puente pierda paquetes?
37. En la figura 5-41 el apuntador de marco apunta a la primera variable local. ¿Qué información necesita el programa para regresar de un procedimiento?
38. Escriba una subrutina en lenguaje ensamblador que convierta un entero binario con signo a ASCII.
39. Escriba una subrutina en lenguaje ensamblador que convierta una fórmula infija a notación polaca inversa.
40. El de las Torres de Hanoi no es el único procedimiento recursivo favorito de los computólogos. Otro gran favorito es $n!$, donde $n! = n(n-1)!$ sujeto a la condición limitante de que $0! = 1$. Escriba un procedimiento en su lenguaje ensamblador favorito para calcular $n!$.
41. Si no está convencido de que la recursión a veces es indispensable, trate de programar las Torres de Hanoi sin usar recursión y sin simular la solución recursiva manteniendo una pila en un arreglo. Conviene advertir que probablemente no logre encontrar la solución.

6

EL NIVEL DE MÁQUINA DE SISTEMA OPERATIVO

El tema de este libro es que una computadora moderna consiste en una serie de niveles, cada uno de los cuales añade funcionalidad al nivel que está abajo. Ya vimos el nivel de lógica digital, el nivel de microarquitectura y el nivel de conjunto de instrucciones. Ha llegado el momento de ascender al siguiente nivel, el ámbito del sistema operativo.

Un **sistema operativo** es un programa que, desde el punto de vista del programador, añade varias instrucciones y funciones nuevas, más allá de lo que el nivel ISA proporciona. Normalmente, el sistema operativo se implementa casi totalmente en software, pero no existe una razón teórica para no colocarlo en hardware, como se hace normalmente con los microprogramas (cuando están presentes). Usaremos el acrónimo **OSM** (*Operating System Machine*) para referirnos al nivel que el sistema operativo implementa, el nivel de **máquina de sistema operativo**, que se muestra en la figura 6-1.

Aunque tanto el nivel OSM como el nivel ISA son abstractos (en el sentido de que no son el verdadero nivel de hardware), existe una diferencia importante entre ellos. El conjunto de instrucciones del nivel OSM es el conjunto completo de instrucciones que pueden usar los programadores de aplicaciones; contiene casi todas las instrucciones del nivel ISA, así como el conjunto de instrucciones nuevas que el sistema operativo añade. Estas nuevas instrucciones se denominan **llamadas al sistema**. Una llamada al sistema invoca un servicio predefinido del sistema operativo, o sea, una de sus instrucciones. Una llamada al sistema típica solicita leer datos de un archivo. Usaremos el tipo de letra helvética en minúsculas para distinguir las llamadas al sistema.

El nivel OSM siempre se interpreta. Cuando un programa de usuario ejecuta una instrucción OSM, como leer datos de un archivo, el sistema operativo ejecuta esta instrucción paso

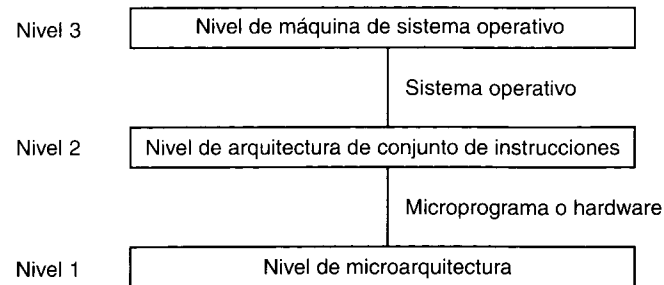


Figura 6-1. Ubicación del nivel de máquina de sistema operativo.

por paso, igual que un microprograma ejecutaría una instrucción **ADD** paso por paso. Sin embargo, cuando un programa ejecuta una instrucción del nivel ISA, el nivel de microarquitectura subyacente se encarga de llevarla a cabo directamente, sin ayuda del sistema operativo.

En este libro sólo podemos proporcionar una muy breve introducción al tema de los sistemas operativos. Nos concentraremos en tres importantes temas. El primero es la memoria virtual, una técnica que muchos sistemas operativos usan para aparentar que la máquina tiene más memoria de la que en realidad tiene. El segundo es la E/S de archivos, un concepto de nivel más alto que las instrucciones de E/S que estudiamos en el capítulo anterior. El tercer y último tema es el de procesamiento en paralelo: cómo varios procesos pueden ejecutarse, comunicarse y sincronizarse. El concepto de proceso es muy importante, y lo describiremos con detalle más adelante. Por ahora, podemos pensar en un proceso como un programa en ejecución y toda su información de estado (memoria, registros, contador de programa, situación de E/S, etc.). Después de explicar estos principios en general, veremos cómo se aplican a los sistemas operativos de dos de nuestras máquinas de ejemplo, el Pentium II (Windows NT) y el UltraSPARC II (UNIX). Puesto que el picoJava II normalmente se usa para sistemas incorporados, no tiene un sistema operativo con todas las de la ley.

6.1 MEMORIA VIRTUAL

En los albores de la computación, las memorias eran pequeñas y costosas. El IBM 650, la principal computadora científica de su época (fines de la década de los cincuenta), sólo tenía 2000 palabras de memoria. Uno de los primeros compiladores de ALGOL 60 se escribió para una máquina que sólo tenía 1024 palabras de memoria. Uno de los primeros sistemas de tiempo compartido trabajaba de forma muy satisfactoria en una PDP-1 con un tamaño total de memoria de 4096 palabras de 18 bits para el sistema operativo y los programas de usuario combinados. En esos días el programador dedicaba gran parte de su tiempo a lograr que sus programas cupieran en la diminuta memoria. En muchos casos era necesario usar un algoritmo que trabajaba mucho más despacio que otro algoritmo mejor, simplemente porque el algoritmo mejor era demasiado grande; es decir, un programa que usaba el algoritmo mejor no cabía en la memoria de la computadora.

La solución tradicional a este problema era usar memoria secundaria, como un disco. El programador dividía el programa en varios fragmentos, llamados **superposiciones**, cada uno de los cuales cabía en la memoria. Para ejecutar el programa, se leía la primera superposición y se ejecutaba durante un rato. Cuando terminaba, leía la siguiente superposición y la invocaba, y así. El programador tenía la responsabilidad de dividir el programa en superposiciones, decidir en qué lugar de la memoria secundaria se debía guardar cada superposición, encargarse del traslado de superposiciones entre la memoria principal y la memoria secundaria, y en general administrar el proceso de superponer sin ayuda de la computadora.

Aunque se usó ampliamente durante muchos años, esta técnica implicaba mucho trabajo invertido en la gestión de superposiciones. En 1961 un grupo de investigadores de Manchester, Inglaterra, propuso un método para realizar automáticamente el proceso de superponer, sin que el programador siquiera supiera qué estaba ocurriendo (Fotheringham, 1961). Este método, ahora conocido como **memoria virtual**, tenía la ventaja obvia de liberar al programador de una gran cantidad de molestas tareas de contabilidad, y se usó por primera vez en los años sesenta en varias computadoras, casi todas asociadas a proyectos de investigación sobre diseño de sistemas de cómputo. Para cuando comenzó la década de los setenta casi todas las computadoras contaban con memoria virtual. Ahora hasta las computadoras de un solo chip, incluidos el Pentium II y el UltraSPARC II, tienen sistemas de memoria virtual muy sofisticados, que examinaremos en una sección posterior del capítulo.

6.1.1 Paginación

La idea sugerida por el grupo de Manchester fue la de separar los conceptos de espacio de direcciones y posiciones de memoria. Consideremos, por ejemplo, una computadora representativa de esa época, con un campo de dirección de 16 bits en sus instrucciones y 4096 palabras de memoria. Un programa para esta computadora podía direccionar 65536 palabras de memoria. La razón es que existen 65536 (2^{16}) direcciones de 16 bits, cada una de las cuales corresponde a una palabra de memoria distinta. Cabe señalar que el número de palabras direccionables depende únicamente del número de bits que tiene una dirección y nada tiene que ver con el número de palabras de memoria con que se cuenta realmente. El **espacio de direcciones** de esta computadora consiste en los números 0, 1, 2, ..., 65535, porque ése es el conjunto de posibles direcciones. Sin embargo, la computadora bien podría tener menos de 65535 palabras de memoria.

Antes de que se inventara la memoria virtual, la gente habría distinguido entre las direcciones por debajo de 4096 y aquellas iguales o mayores que 4096. Aunque pocas veces se decía explícitamente, estas dos partes se consideraban como el espacio de direcciones útiles y el espacio de direcciones inútiles, respectivamente (las direcciones mayores que 4095 eran inútiles porque no correspondían a posiciones de memoria reales). La gente no distinguía entre espacio de direcciones y direcciones de memoria, porque el hardware exigía una correspondencia uno a uno entre ellos.

La idea de separar el espacio de direcciones y las direcciones de memoria es la siguiente. En cualquier momento dado, es posible acceder directamente a 4096 palabras de memoria,

pero no es necesario que correspondan a las direcciones de memoria 0 a 4095. Podríamos, por ejemplo, “decirle” a la computadora que, en adelante, cada vez que se haga referencia a la dirección 4096, se usará la palabra de memoria que está en la dirección 0. Cada vez que se haga referencia a la dirección 4097, se usará la palabra de memoria que está en la dirección 1; cada vez que se haga referencia a la dirección 8191, se usará la palabra de memoria que está en la dirección 4095, y así. En otras palabras, habremos definido una correspondencia o “mapeo” del espacio de direcciones a las direcciones de memoria reales, como se muestra en la figura 6-2.

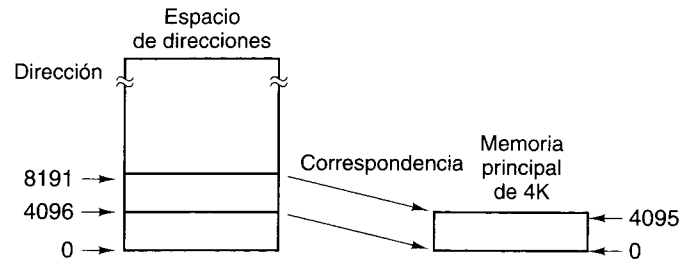


Figura 6-2. Mapeo en el que las direcciones virtuales 4096 a 8191 se hacen corresponder con las direcciones 0 a 4095 de la memoria principal.

En términos de hacer corresponder las direcciones del espacio de direcciones con las posiciones de memoria reales, una máquina de 4K sin memoria virtual simplemente tiene una correspondencia fija entre las direcciones 0 a 4095 y las 4096 palabras de memoria. Una pregunta interesante es: “¿Qué sucede si un programa ramifica a una dirección entre 8192 y 12287?” En una máquina sin memoria virtual, el programa causaría una trampa de error que imprimiría un mensaje debidamente grosero como “Referencia a memoria inexistente” y terminaría el programa. En una máquina con memoria virtual, ocurriría la siguiente sucesión de pasos:

1. El contenido de la memoria principal se guardaría en el disco.
2. Se localizarían en el disco las palabras 8192 a 12287.
3. Se cargarían en la memoria principal las palabras 8192 a 12287.
4. El mapa de direcciones se modificaría para hacer corresponder las direcciones 8192 a 12287 con las posiciones de memoria 0 a 4095.
5. La ejecución continuaría como si nada fuera de lo normal hubiera ocurrido.

Esta técnica de superposición automática se denomina **paginación**, y los fragmentos de programa que se leen del disco se llaman **páginas**.

Puede usarse una forma más sofisticada de hacer corresponder las direcciones del espacio de direcciones con las direcciones de memoria real. Para destacar la diferencia, llamaremos a las direcciones a las que el programa puede hacer referencia **espacio de direcciones virtual**, y a las direcciones de memoria reales, en hardware, **espacio de direcciones físico**. Un **mapa de memoria** o **tabla de páginas** relaciona las direcciones virtuales con las direc-

ciones físicas. Suponemos que hay suficiente espacio en el disco para guardar todo el espacio de direcciones virtual (o al menos la porción de ese espacio que se está usando).

Los programas se escriben como si hubiera suficiente memoria principal para todo el espacio de direcciones virtual, aunque no sea así. Los programas pueden cargar valores de, o guardarlos en, cualquier palabra del espacio de direcciones virtual, o saltar a cualquier instrucción situada en cualquier punto del espacio de direcciones virtual, sin tener en cuenta el hecho de que en realidad no hay suficiente memoria física. De hecho, el programador puede escribir programas sin siquiera saber si existe o no memoria virtual. La computadora simplemente parece tener una memoria grande.

Este punto es crucial y se contrastará posteriormente con la segmentación, en la que el programador debe tener conocimiento de la existencia de segmentos. Hacemos hincapié una vez más en que la paginación ofrece al programador la ilusión de una memoria principal grande, continua y lineal, del mismo tamaño que el espacio de direcciones virtual. En realidad, la memoria principal disponible podría ser menor (o mayor) que el espacio de direcciones virtual. La simulación de esta memoria grande por paginación no puede ser detectada por el programa (a menos que se efectúen pruebas de cronometría). Cada vez que se hace referencia a una dirección, parece estar presente la palabra de instrucción o datos correcta. Puesto que el programador puede programar como si no existiera la paginación, se dice que el mecanismo de paginación es **transparente**.

No es la primera vez que nos topamos con la idea de que un programador pueda usar alguna característica inexistente sin preocuparse por su funcionamiento. El conjunto de instrucciones del nivel ISA a menudo incluye una instrucción **MUL**, a pesar de que la microarquitectura subyacente no cuenta con un dispositivo de multiplicación en el hardware. La ilusión de que la máquina puede multiplicar casi siempre se mantiene con microcódigo. Así mismo, la máquina virtual que el sistema operativo proporciona puede dar la ilusión de que todas las direcciones virtuales están respaldadas por memoria real, aunque no sea así. Sólo los escritores de sistemas operativos (y estudiantes de tales sistemas) tienen que saber cómo se mantiene la ilusión.

6.1.2 Implementación de la paginación

Un requisito indispensable para una memoria virtual es un disco en el cual pueda guardarse todo el programa y todos los datos. En lo conceptual, es más sencillo pensar que la copia del programa que está en el disco es el original y que los fragmentos que se traen a la memoria principal de vez en cuando son copias, y no lo contrario. Desde luego, es importante mantener actualizado el original. Cuando se modifica la copia que está en la memoria principal, los cambios deben reflejarse en el original (tarde o temprano).

El espacio de direcciones se divide en varias páginas de tamaño uniforme. Hoy día son comunes tamaños de página entre 512 y 64K bytes por página, aunque de vez en cuando se usan tamaños de hasta 4 MB. El tamaño de página siempre es una potencia de 2. El espacio de direcciones físico se divide en fragmentos de la misma manera, y cada uno es del mismo tamaño de una página, de modo que cada fragmento de la memoria principal pueda contener exactamente una página. Estos fragmentos de la memoria principal en los que entran las

páginas se llaman **marcos de página**. En la figura 6-2 la memoria principal contiene sólo un marco de página. En los diseños prácticos por lo regular contiene miles de marcos.

La figura 6-3(a) ilustra una posible forma de dividir los primeros 64K de un espacio de direcciones virtual: en páginas de 4K. (Tome nota de que estamos hablando de 64K y 4K de direcciones aquí. Una dirección podría ser un byte pero también podría ser una palabra en una computadora en la que palabras consecutivas tienen direcciones consecutivas.) La memoria virtual de la figura 6-3 se implementaría con una tabla de páginas que tiene tantas entradas como páginas caben en el espacio de direcciones virtual. Para simplificar, sólo hemos mostrado aquí las primeras 16 entradas. Cuando el programa trata de hacer referencia a una palabra que está en los primeros 64K de su espacio de direcciones virtual, sea para traer instrucciones, traer datos o guardar datos, primero genera una dirección virtual entre 0 y 65532 (suponiendo que las direcciones de palabra deben ser divisibles entre 4). Se puede usar indexación, direccionamiento indirecto o cualquier otra de las técnicas acostumbradas para generar la dirección.

Página Direcciones virtuales	
15	61440 – 65535
14	57344 – 61439
13	53248 – 57343
12	49152 – 53247
11	45056 – 49151
10	40960 – 45055
9	36864 – 40959
8	32768 – 36863
7	28672 – 32767
6	24576 – 28671
5	20480 – 24575
4	16384 – 20479
3	12288 – 16383
2	8192 – 12287
1	4096 – 8191
0	0 – 4095

32K bajos de la memoria principal

Marco de página	Direcciones físicas
7	28672 – 32767
6	24576 – 28671
5	20480 – 24575
4	16384 – 20479
3	12288 – 16383
2	8192 – 12287
1	4096 – 8191
0	0 – 4095

(a)

(b)

Figura 6-3. (a) Los primeros 64K del espacio de direcciones virtual divididos en 16 páginas, cada una de las cuales tiene 4K. (b) Memoria principal de 32K dividida en ocho marcos de página de 4K cada uno.

La figura 6-3(b) muestra una memoria física que consiste en ocho marcos de página de 4K. Esta memoria podría estar limitada a 32K porque (1) eso es todo lo que la máquina tiene (un procesador incorporado en una lavadora de ropa o un horno de microondas tal vez no necesite más), o (2) el resto de la memoria se asignó a otros programas.

Considere ahora cómo puede hacerse corresponder una dirección virtual de 32 bits con una dirección física en la memoria principal. Después de todo, lo único que la memoria entiende son direcciones de la memoria principal, no direcciones virtuales, así que eso es lo que debemos darle. Toda computadora con memoria virtual tiene un dispositivo para efectuar el mapeo de virtual a físico, llamado **unidad de gestión de memoria (MMU, Memory Management Unit)**. La MMU podría estar en el chip de la CPU o en un chip aparte que funcione en estrecha colaboración con la CPU. Puesto que nuestra MMU de ejemplo establece una correspondencia entre una dirección virtual de 32 bits y una dirección física de 15 bits, necesita un registro de entrada de 32 bits y un registro de salida de 15 bits.

Para ver cómo funciona la MMU, considere el ejemplo de la figura 6-4. Cuando la MMU recibe una dirección virtual de 32 bits, la divide en un número de página virtual, de 20 bits, y una distancia dentro de la página, de 12 bits (porque en nuestro ejemplo las páginas son de 4K). El número de página virtual se usa como índice de la tabla de páginas para encontrar la entrada que corresponde a la página a la que se hizo referencia. En la figura 6-4 el número de página virtual es 3, por lo que se selecciona la entrada 3 de la tabla de páginas, como se muestra.

Lo primero que la MMU hace con la entrada de la tabla de páginas es verificar si la página a la que se hizo referencia está actualmente en la memoria principal. Después de todo, con 2²⁰ páginas virtuales y sólo 8 marcos de página, no todas las páginas virtuales pueden estar en la memoria a la vez. La MMU efectúa esta verificación examinando el **bit presente/ausente** de la entrada de la tabla de páginas. En nuestro ejemplo el bit es 1, lo que implica que la página actualmente está en la memoria.

El siguiente paso es tomar el valor de marco de página de la entrada seleccionada (6 en este caso) y copiarlo en los tres bits superiores del registro de salida de 15 bits. Se requieren tres bits porque hay ocho marcos de página en la memoria física. Al mismo tiempo que se realiza esta operación, los 12 bits de orden bajo de la dirección virtual (el campo de distancia dentro de la página) se copian en los 12 bits de orden bajo del registro de salida, como se muestra. Esta dirección de 15 bits se envía entonces a caché o a la memoria para buscarla.

La figura 6-5 muestra una posible correspondencia entre páginas virtuales y marcos de página físicos. La página virtual 0 está en el marco de página 1. La página virtual 1 está en el marco de página 0. La página virtual 2 no está en la memoria principal. La página virtual 3 está en el marco de página 2. La página virtual 4 no está en la memoria principal. La página virtual 5 está en el marco de página 6, etcétera.

6.1.3 Paginación por demanda y modelo de conjunto de trabajo

En la explicación anterior se supuso que la página virtual a la que se hizo referencia estaba en la memoria principal. Sin embargo, tal supuesto no siempre se cumplirá porque no hay suficiente espacio en la memoria principal para todas las páginas virtuales. Cuando se hace una referencia a una dirección que está en una página que no está presente en la memoria principal, ocurre un **fallo de página**. En tal caso, el sistema operativo necesita leer la página requerida del disco, introducir en la tabla de páginas su nueva ubicación en la memoria física, y luego repetir la instrucción que causó el fallo.

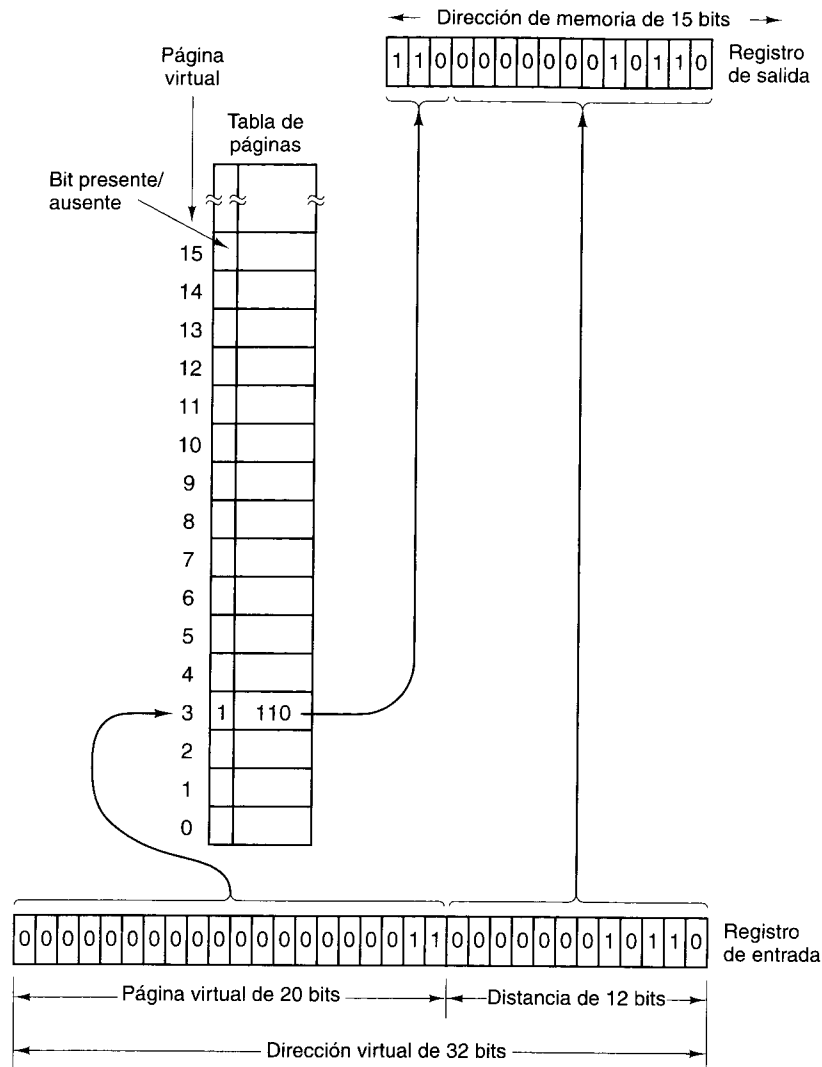


Figura 6-4. Formación de una dirección de memoria principal a partir de una dirección virtual.

Es posible iniciar la ejecución de un programa en una máquina con memoria virtual aunque ninguna parte del programa esté en la memoria principal. Simplemente hay que ajustar la tabla de páginas de modo que indique que todas y cada una de las páginas virtuales está en la memoria secundaria y no en la memoria principal. Cuando la CPU trate de traer la primera instrucción, de inmediato causará un fallo de página, lo que hará que la página que contiene la primera instrucción se cargue en la memoria y se introduzca en la tabla de páginas. Luego podrá iniciarse la primera instrucción. Si dicha instrucción tiene dos direcciones,

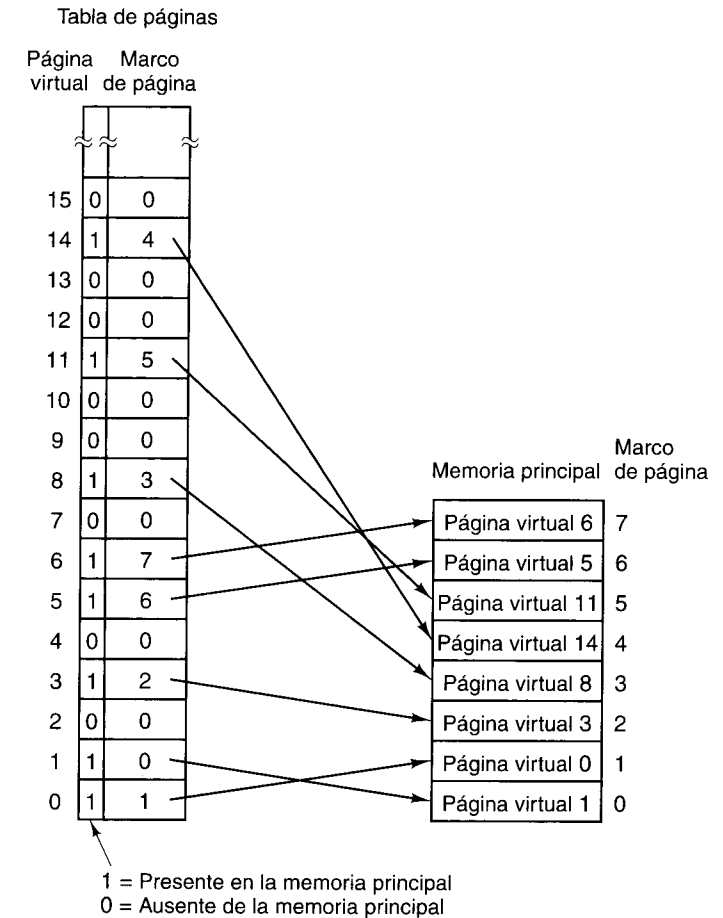


Figura 6-5. Una posible correspondencia entre las primeras 16 páginas virtuales y una memoria principal con ocho marcos de página.

las dos están en diferentes páginas y ambas páginas son diferentes de la página en la que estaba la instrucción, ocurrirán otros dos fallos de página, y se traerán dos páginas más antes de que la instrucción por fin pueda ejecutarse. La siguiente instrucción podría causar más fallos de página, y así sucesivamente.

Este método de operar una memoria virtual se llama **paginación por demanda**, por analogía con el conocido algoritmo de alimentación por demanda para los bebés: si el bebé llora, hay que alimentarlo (en lugar de alimentarlo según un programa estricto). En la paginación por demanda, sólo se traen páginas cuando se solicita realmente una página, no por adelantado.

La pregunta de si debe usarse o no paginación por demanda sólo es pertinente cuando se inicia un programa. Una vez que el programa ha estado trabajando durante cierto tiempo, las páginas requeridas ya se habrán reunido en la memoria principal. Si la computadora es de

tiempo compartido y los procesos se intercambian a disco después de ejecutarse durante 100 ms (digamos), cada programa se reiniciará muchas veces durante su ejecución. Puesto que el mapa de memoria es único para cada programa, y cambia cuando se conmutan los programas, como en un sistema de tiempo compartido, la pregunta se vuelve una cuestión crítica.

La estrategia alternativa se basa en la observación de que la mayor parte de los programas no hace referencia a su espacio de direcciones de manera uniforme, sino que las referencias tienden a concentrarse en un número reducido de páginas. Una referencia a la memoria podría traer una instrucción, podría traer datos o podría guardar datos. En cualquier instante t existe un conjunto formado por todas las páginas utilizadas por las k referencias a memoria más recientes. Denning (1968) lo llamó **conjunto de trabajo**.

Dado que el conjunto de trabajo suele variar lentamente con el tiempo, es posible hacer una conjetura razonable respecto a cuáles páginas se necesitarán cuando se reinicie el programa, con base en el conjunto de trabajo que tenía cuando se suspendió la última vez. Estas páginas podrían cargarse por adelantado antes de iniciar el programa (suponiendo que caben).

6.1.4 Política de reemplazo de páginas

Idealmente, el conjunto de páginas que un programa está usando activa e intensivamente, llamado **conjunto de trabajo**, se puede mantener en la memoria a fin de reducir los fallos de página. Sin embargo, los programadores casi nunca saben cuáles páginas están en el conjunto de trabajo, por lo que el sistema operativo debe descubrir dicho conjunto dinámicamente. Cuando un programa hace referencia a una página que no está en la memoria principal, la página requerida debe traerse del disco. Sin embargo, para que quepa generalmente es necesario enviar alguna otra página de vuelta al disco. Se necesita un algoritmo que decida cuál página debe quitarse.

Probablemente no es una buena idea escoger al azar la página que se quitará. Si la página que contiene la instrucción que causó el fallo es la que se escoge, ocurrirá otro fallo de página tan pronto como se intente traer la siguiente instrucción. Casi todos los sistemas operativos tratan de predecir cuál de las páginas de la memoria es la menos útil en el sentido de que su ausencia tendría el efecto adverso menos sensible sobre el programa en ejecución. Una forma de hacerlo es predecir cuándo ocurrirá la siguiente referencia a cada página y quitar la página cuya siguiente referencia predicha es más lejana en el futuro. En otras palabras, en lugar de expulsar una página que se necesitará en poco tiempo, se trata de seleccionar una que no se necesitará durante un buen tiempo.

Un algoritmo muy utilizado expulsa la página que se usó menos recientemente porque la probabilidad *a priori* de que no esté en el conjunto de trabajo vigente es alta. Éste es el algoritmo **LRU (menos recientemente utilizada, Least Recently Used)**. Aunque este algoritmo normalmente funciona bien, existen situaciones patológicas, como la que se describirá a continuación, en las que falla lamentablemente.

Imagine un programa que ejecuta un ciclo grande que se extiende sobre nueve páginas virtuales en una máquina que tiene espacio sólo para ocho páginas en la memoria física. Una vez que el programa llega a la página 7, el estado de la memoria principal es el que se muestra

en la figura 6-6(a). En algún momento se intenta traer una instrucción de la página virtual 8, lo que causa un fallo de página. Se debe tomar una decisión acerca de cuál página se expulsará. El algoritmo LRU escoge la página virtual 0 porque es la que se usó menos recientemente. Se quita la página virtual 0 y se trae la página virtual 8 para sustituirla, y la situación es la que se muestra en la figura 6-6(b).

Página virtual 7	Página virtual 7	Página virtual 7
Página virtual 6	Página virtual 6	Página virtual 6
Página virtual 5	Página virtual 5	Página virtual 5
Página virtual 4	Página virtual 4	Página virtual 4
Página virtual 3	Página virtual 3	Página virtual 3
Página virtual 2	Página virtual 2	Página virtual 2
Página virtual 1	Página virtual 1	Página virtual 0
Página virtual 0	Página virtual 8	Página virtual 8

(a)

(b)

(c)

Figura 6-6. Fracaso del algoritmo LRU.

Después de ejecutar las instrucciones de la página virtual 8, el programa salta al principio del ciclo, a la página virtual 0. Este paso causa otro fallo de página. La página virtual 0, que acaba de expulsarse, se tiene que volver a traer. El algoritmo LRU escoge la página 1 para expulsión, lo cual lleva a la situación de la figura 6-6(c). El programa continuará en la página 0 durante un rato, y luego tratará de traer una instrucción de la página virtual 1, causando un fallo de página. Hay que volver a traer la página 1 y para ello se expulsará la página 2.

Ya deberá ser obvio que el algoritmo LRU está tomando la peor decisión en cada ocasión (otros algoritmos también fallan en condiciones similares). Sin embargo, si la memoria principal disponible es mayor que el tamaño del conjunto de trabajo, el algoritmo LRU sí tiende a minimizar el número de fallos de página.

Otro algoritmo de reemplazo de páginas es el de **primero en entrar, primero en salir (FIFO, First-In First-Out)**. FIFO expulsa la página que se cargó menos recientemente, sin importar cuándo fue la última vez que se hizo referencia a ella. Cada marco de página tiene un contador. Inicialmente, todos los contadores contienen 0. Después de manejarse cada fallo de página, se incrementa en 1 el contador de cada página que actualmente está en la memoria, y el contador de la página que se acaba de traer se pone en 0. Cuando se hace necesario escoger una página para expulsarla, se escoge aquella cuyo contador es el más alto. Esto indica que la página correspondiente ha sido testigo del mayor número de fallos de página, y por tanto que se cargó antes que cualquiera de las otras páginas que están en la memoria y tiene (con suerte) la mayor probabilidad *a priori* de no necesitarse más.

Si el conjunto de trabajo es mayor que el número de marcos de página disponibles, ningún algoritmo que no sea un oráculo dará buenos resultados, y los fallos de página serán

frecuentes. Un programa que genera fallos de página de forma continua y frecuente está **hiperpaginando**. Sobra decir que la hiperpaginación es una condición indeseable en cualquier sistema. Si un programa usa una gran cantidad de espacio virtual de direcciones pero tiene un conjunto de trabajo pequeño que cambia lentamente y cabe en la memoria principal disponible, no causará problemas. Esta observación se cumple aunque, a lo largo de su existencia, el programa use cientos de veces más palabras de memoria virtual que las palabras de memoria principal que tiene la máquina.

Si una página que está a punto de ser expulsada no se ha modificado desde que se leyó (lo cual es muy probable si la página contiene programa, no datos), no es necesario volver a escribirla en el disco, pues ya existe ahí una copia exacta. Si la página se modificó después de leerse, la copia del disco ya no es exacta y habrá que reescribir la página.

Si hay una forma de saber si una página no ha cambiado desde que se leyó (página limpia) o si se escribió después en ella (página sucia), podrá evitarse la reescritura de páginas limpias y se ahorrará mucho tiempo. Muchas computadoras tienen un bit por página, en la MMU, que se pone en 0 cuando la página se carga, y que el microprograma o el hardware pone en 1 cada vez que se escribe en ella (es decir, se ensucia). Si el sistema operativo examina este bit, sabrá si la página está limpia o sucia, y por ende si es necesario reescribirla o no.

6.1.5 Tamaño de página y fragmentación

Si por casualidad el programa y los datos del usuario llenan exactamente un número entero de páginas, no se desperdiciará espacio cuando estén en la memoria. Pero si no sucede así, quedará cierto espacio sin utilizar en la última página. Por ejemplo, si el programa y los datos necesitan 26,000 bytes en una máquina que tiene 4096 bytes por página, las primeras seis páginas estarán llenas, para un total de $6 \times 4096 = 24,576$ bytes, y la última página contendrá $26,000 - 24,576 = 1424$ bytes. Puesto que en cada página caben 4096 bytes, se desperdiciarán 2672 bytes. Cada vez que la séptima página esté presente en la memoria, esos bytes ocuparán espacio en la memoria principal pero no servirán de nada. El problema de estos bytes desperdiciados se denomina **fragmentación interna** (porque el espacio desperdiciado está adentro de una página).

Si el tamaño de página es de n bytes, la cantidad media de espacio desperdiciado por fragmentación interna en la última página de un programa será de $n/2$ bytes, situación que sugiere usar páginas pequeñas para minimizar el desperdicio. Por otra parte, un tamaño de página pequeño implica muchas páginas, además de una tabla de páginas grande. Si la tabla de páginas se mantiene en hardware, y es grande, se necesitarán más registros para guardarla, y el costo de la computadora aumentará. Además, se requerirá más tiempo para cargar y guardar estos registros cada vez que se inicie o detenga un programa.

Además, las páginas pequeñas utilizan de forma ineficiente el ancho de banda del disco. Si suponemos que hay que esperar 10 ms para poder iniciar una transferencia (retraso por búsqueda + latencia rotacional), las transferencias grandes serán más eficientes que las pequeñas. Con una tasa de transferencia de 10 MB/s, transferir 8 KB tarda sólo 0.7 ms más que transferir 1 KB (10.1 ms *versus* 10.8 ms).

Sin embargo, las páginas pequeñas tienen la ventaja adicional de que si el conjunto de trabajo consiste en un gran número de regiones pequeñas discretas en el espacio de direcciones virtual podría haber menos hiperpaginación con un tamaño de página pequeño que con uno grande. Por ejemplo, considere una matriz de $10,000 \times 10,000$, A que se almacena con $A[1, 1]$, $A[2, 1]$, $A[3, 1]$, etc., en palabras de 8 bytes consecutivas. Este almacenamiento ordenado por columna implica que los elementos de la fila 1, $A[1, 1]$, $A[1, 2]$, $A[1, 3]$, etc., comenzarán en posiciones separadas 80,000 bytes una de la siguiente. Un programa que realice cálculos extensos con todos los elementos de esta fila usaría 10,000 regiones, cada una separada de la siguiente por 79,992 bytes. Si el tamaño de página fuera de 8 KB, se requeriría una memoria total de 80 MB para contener todas las páginas en uso.

Por otra parte, si el tamaño de página es de 1 KB, se requerirían sólo 10 MB de RAM para contener todas las páginas. Si la memoria disponible fuera de 32 MB, el programa hiperpaginaría si las páginas fueran de 8 KB, pero no si fueran de 1 KB.

6.1.6 Segmentación

La memoria virtual de la que hemos hablado aquí es unidimensional porque las direcciones virtuales van de la 0 a alguna dirección máxima, una tras otra. Para muchos problemas, tener dos o más espacios de direcciones distintos podría ser mucho mejor que tener sólo uno. Por ejemplo, un compilador podría tener muchas tablas que se van creando a medida que avanza la compilación y que incluyen:

1. La tabla de símbolos, que contiene los nombres y atributos de las variables.
2. El texto fuente que se está guardando para el listado impreso.
3. Una tabla que contiene todas las constantes enteras y de punto flotante empleadas.
4. El árbol de análisis sintáctico del programa.
5. La pila empleada para llamadas a procedimientos dentro del compilador.

Las primeras cuatro tablas crecen continuamente a medida que la compilación avanza. La última crece y se encoge de forma impredecible durante la compilación. En una memoria unidimensional, habría que asignar a estas tablas trozos contiguos del espacio de direcciones virtual, como en la figura 6-7.

Considere lo que sucede si un programa tiene un número excepcionalmente grande de variables. El trozo de espacio de direcciones asignado a la tabla de símbolos podría llenarse, aunque haya mucho espacio libre en las otras tablas. Desde luego, el compilador podría limitarse a emitir un mensaje diciendo que la compilación no puede continuar porque hay demasiadas variables, pero hacerlo no sería justo si queda espacio sin ocupar en las demás tablas.

Otra posibilidad es que el compilador actúe como Robin Hood y tome espacio libre de las tablas que tienen mucho para dárselo a las tablas que tienen poco. Esto es factible, pero sería análogo a gestionar sus propias superposiciones: una lata en el mejor de los casos y un exceso de trabajo tedioso y fútil en el peor de los casos.

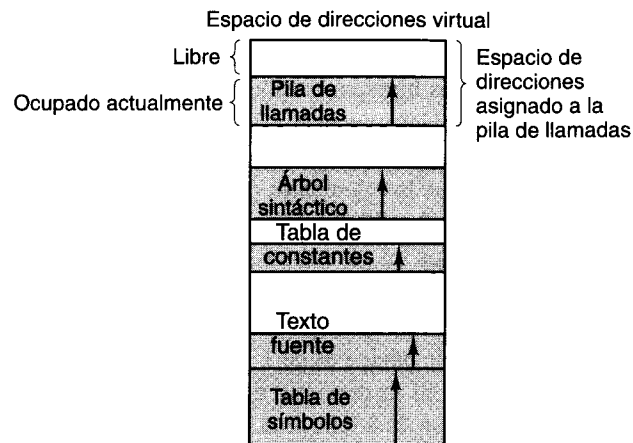


Figura 6-7. En un espacio de direcciones unidimensional con tablas que crecen, una tabla podría chocar con otra.

Lo que realmente se necesita es una forma de evitar al programador el trabajo de administrar las tablas en expansión y contracción, del mismo modo que la memoria virtual elimina la preocupación por tener que organizar el programa en superposiciones.

Una solución sencilla es proporcionar muchos espacios de direcciones totalmente independientes, llamados **segmentos**. Cada segmento consiste en una sucesión lineal de direcciones, de la 0 a alguna dirección máxima. La longitud de cada segmento puede ser cualquiera desde 0 hasta el máximo permitido. Diferentes segmentos pueden tener diferentes longitudes, y generalmente así es. Además, la longitud de los segmentos podría cambiar durante la ejecución. La longitud de un segmento de pila podría incrementarse cada vez que algo se mete en la pila y reducirse cada vez que algo se desempila.

Puesto que cada segmento constituye un espacio de direcciones individual, los diferentes segmentos pueden crecer o encogerse de manera independiente, sin afectar a los otros. Si una pila en cierto segmento necesita más espacio de direcciones para crecer, puede tenerlo, porque no hay nada más en su espacio de direcciones con lo que pueda chocar. Desde luego, un segmento podría llenarse, pero los segmentos suelen ser muy grandes, y esto casi nunca sucede. Para especificar una dirección en esta memoria bidimensional segmentada, el programa debe proporcionar una dirección con dos partes: un número de segmento y una dirección dentro de ese segmento. La figura 6-8 ilustra una memoria segmentada que se usa para las tablas de compilador antes mencionadas.

Hacemos hincapié en que un segmento es una entidad lógica, de cuya existencia el programador es consciente, y que usa como una sola entidad lógica. Un segmento podría contener un procedimiento, un arreglo, una pila o una colección de variables escalares, pero por lo regular no contiene una mezcla de diferentes tipos.

Las memorias segmentadas tienen otras ventajas además de simplificar el manejo de estructuras de datos que crecen o se encogen. Si cada procedimiento ocupa un segmento distinto, y la dirección 0 es su dirección de inicio, se simplifica considerablemente la vincu-

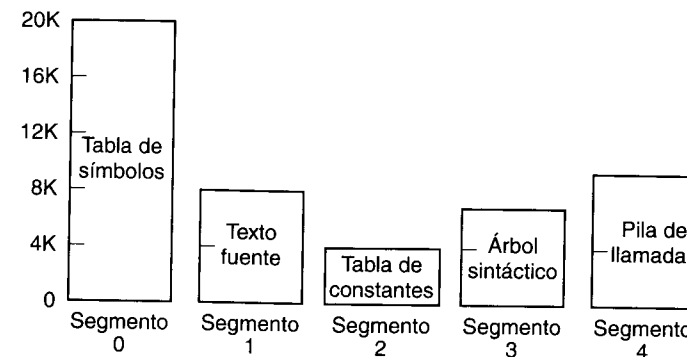


Figura 6-8. Una memoria segmentada permite a cada tabla crecer o encogerse con independencia de las otras tablas.

lación o ligado (*linking*) de procedimientos que se compilan por separado. Una vez que se han compilado y vinculado todos los procedimientos que constituyen un programa, una llamada al procedimiento en el segmento n usará la dirección de dos partes ($n, 0$) para direccionar la palabra 0 (el punto de ingreso).

Si el procedimiento que está en el segmento n se modifica y recompila posteriormente, no será necesario modificar ningún otro procedimiento (porque no se ha cambiado ninguna dirección de inicio), aunque la nueva versión sea más grande que la antigua. Con una memoria unidimensional, los procedimientos normalmente se empaquetan muy juntos, sin espacio de direcciones entre ellos. Por tanto, la modificación del tamaño de un procedimiento puede afectar la dirección de inicio de otros procedimientos no relacionados. Esto, a su vez, requiere modificar todos los procedimientos que invocan cualquiera de los procedimientos desplazados, a fin de incorporar sus nuevas direcciones de inicio. Si un programa contiene cientos de procedimientos, este proceso puede ser costoso.

La segmentación también facilita compartir procedimientos o datos entre varios programas. Si una computadora tiene varios programas ejecutándose en paralelo (sea procesamiento en paralelo verdadero o simulado), y todos usan ciertos procedimientos de biblioteca, sería un desperdicio de memoria principal proporcionar a cada uno su copia privada. Si se hace que cada procedimiento esté en un segmento distinto, es fácil compartirlos y se elimina la necesidad de tener más de una copia física de cualquier procedimiento compartido en la memoria principal. El resultado es un ahorro de memoria.

Puesto que cada segmento forma una entidad lógica de la que el programador tiene conocimiento, como un procedimiento, un arreglo o una pila, los diferentes segmentos pueden tener diferentes tipos de protección. Un segmento de procedimiento podría especificarse como "sólo para ejecución", lo que impediría que alguien lo lea o escriba en él. Un arreglo de punto flotante podría especificarse como de lectura/escritura pero no para ejecución; con ello se detectaría cualquier intento por saltar a él. Este tipo de protección suele ser útil para detectar errores de programación.

Es importante entender por qué la protección tiene sentido en una memoria segmentada pero no en una memoria paginada unidimensional (es decir, lineal). En una memoria

segmentada el usuario sabe qué hay en cada segmento. Normalmente, un segmento no contiene un procedimiento y una pila, por ejemplo, sino sólo uno de los dos. Puesto que cada segmento contiene un solo tipo de objeto, el segmento puede tener la protección apropiada para ese tipo específico. En la figura 6-9 se comparan la paginación y la segmentación.

Consideración	Paginación	Segmentación
¿El programador necesita saber que existe?	No	Sí
¿Cuántos espacios de direcciones lineales hay?	1	Muchos
¿El espacio de direcciones virtual puede exceder el tamaño de la memoria?	Sí	Sí
¿Es fácil manejar tablas con tamaño variable?	No	Sí
¿Por qué se inventó esta técnica?	Para simular memorias grandes	Para tener muchos espacios de direcciones

Figura 6-9. Comparación de paginación y segmentación.

El contenido de una página es, en cierto sentido, accidental. El programador ni siquiera se da cuenta de que está ocurriendo paginación. Aunque sería posible incluir unos cuantos bits en cada entrada de la tabla de páginas para especificar el acceso permitido, si el programador quiere aprovechar esta capacidad tendría que saber en qué parte del espacio de direcciones están las fronteras de las páginas. Lo malo de tal idea es que la paginación se inventó precisamente para hacer innecesario ese tipo de administración. Puesto que el usuario de una memoria segmentada tiene la ilusión de que todos los segmentos están en la memoria principal todo el tiempo, pueden direccionarse sin preocuparse por la administración de su superposición.

6.1.7 Implementación de la segmentación

Hay dos formas de implementar la segmentación: intercambio y paginación. En el primer esquema, cierto conjunto de segmentos está en la memoria en un momento dado. Si se hace referencia a un segmento que no está en la memoria, se trae ese segmento. Si no hay espacio para él, será preciso escribir primero en el disco uno o más segmentos (a menos que ya exista ahí una copia limpia, en cuyo caso puede abandonarse la copia que está en la memoria). En cierto sentido, el intercambio de segmentos es parecido a la paginación por demanda: los segmentos entran y salen conforme se van necesitando.

Sin embargo, la implementación de la segmentación difiere de la de la paginación en un aspecto fundamental: las páginas tienen un tamaño fijo y los segmentos no. La figura 6-10(a) muestra un ejemplo de memoria física que inicialmente contiene cinco segmentos. Considere ahora qué sucede si el segmento 1 se expulsa y se coloca en su lugar el segmento 7, que es más pequeño. Llegamos a la configuración de memoria de la figura 6-10(b). Entre el segmento 7 y el segmento 2 hay un área desocupada, es decir, un hueco. Luego el segmento 4 es sustituido por el segmento 5, como en la figura 6-10(c), y el segmento 3 es sustituido por el segmento 6, como en la figura 6-10(d). Después de un rato de funcionar el sistema, la memo-

ria estará dividida en varios trozos, algunos con segmentos y otros con huecos. Este fenómeno se llama **fragmentación externa** (porque se desperdicia espacio que está fuera de los segmentos, en los huecos que hay entre ellos). La fragmentación externa también se conoce como **cuadriculado**.

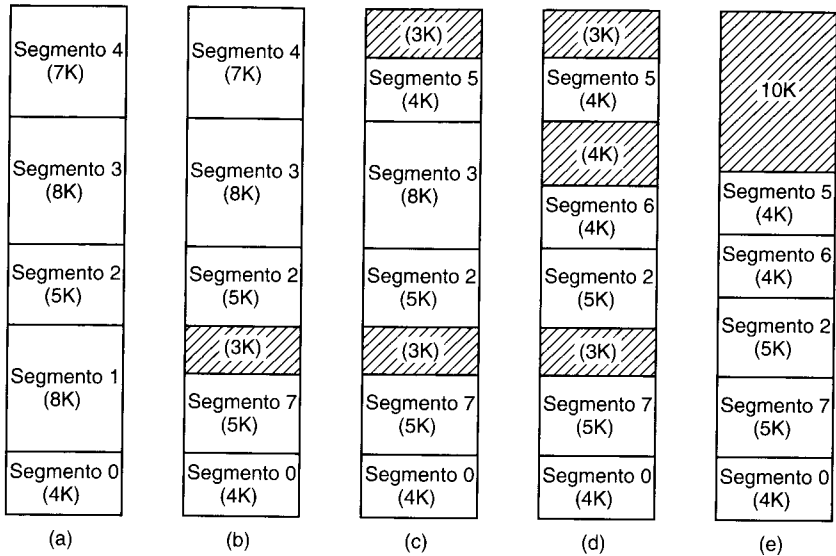


Figura 6-10. (a)-(d) Aparición de fragmentación externa. (e) Eliminación de la fragmentación externa por compactación.

Considere qué sucedería si el programa hiciera referencia al segmento 3 cuando la memoria está experimentando fragmentación externa, como en la figura 6-10(d). El espacio total que ocupan los huecos es de 10K, más que suficiente para el segmento 3, pero como el espacio está distribuido en pequeños fragmentos inútiles no podemos cargar simplemente el segmento 3; será necesario quitar primero algún otro segmento.

Una forma de evitar fragmentación externa es la siguiente: cada vez que aparezca un hueco, los segmentos que están después del hueco se desplazan para acercarlos a la posición de memoria 0 y así eliminar ese hueco y dejar un hueco grande al final. Como alternativa, podríamos esperar hasta que la fragmentación externa sea grave (por ejemplo, más de cierto porcentaje de la memoria total desperdiciado en huecos) antes de realizar la compactación (eliminación de huecos). La figura 6-10(e) muestra cómo quedaría la memoria de la figura 6-10(d) después de una compactación. La intención de compactar la memoria es reunir todos los pequeños huecos inútiles en un solo hueco grande, en el que será posible colocar uno o más segmentos. La compactación tiene la desventaja obvia de que se desperdicia cierto tiempo para llevarla a cabo. En general, no se puede compactar después de que se crea cada hueco, pues se perdería demasiado tiempo.

Si el tiempo requerido para compactar la memoria es demasiado largo, se requiere un algoritmo para determinar cuál hueco deberá usarse para un segmento dado. La gestión de

huecos requiere mantener una lista de las direcciones y tamaños de todos los huecos. Un algoritmo muy popular, llamado **de mejor ajuste**, escoge el hueco más pequeño en el que cabe el segmento requerido. La idea es equiparar los segmentos con los huecos y así evitar gastar un trozo de un agujero grande, que podría necesitarse después para un segmento grande.

Otro algoritmo muy utilizado, llamado **de primer ajuste**, explora circularmente la lista de agujeros y escoge el primer agujero que es lo bastante grande como para albergar el segmento. Esto obviamente toma menos tiempo que examinar toda la lista hasta encontrar el mejor ajuste. Resulta sorprendente que el de primer ajuste es también un mejor algoritmo en términos de desempeño global que el de mejor ajuste, porque este último tiende a generar muchos huecos pequeños totalmente inútiles (Knuth, 1997).

El primero y mejor ajuste tiende a reducir el tamaño medio de los huecos. Cada vez que un segmento se coloca en un hueco mayor que él, lo que sucede casi siempre (los ajustes exactos son raros), el agujero se divide en dos partes. Una parte la ocupa el segmento y la otra es el nuevo hueco. Éste siempre es más pequeño que el hueco viejo. A menos que exista un proceso compensador que vuelva a crear huecos grandes a partir de huecos pequeños, tanto el primer ajuste como el mejor ajuste tarde o temprano llenarán la memoria con huecos pequeños e inútiles.

Uno de esos procesos compensadores es el siguiente. Cada vez que un segmento se saca de la memoria y uno de sus vecinos inmediatos, o ambos, son huecos, no segmentos, los huecos adyacentes se pueden juntar en un hueco grande. Si el segmento 5 se quitara de la figura 6-10(d), los dos huecos que lo flanquean y los 4K utilizados por el segmento se fusionarían en un solo hueco de 11K.

Al principio de esta sección dijimos que hay dos formas de implementar la segmentación: intercambio y paginación. Nuestra explicación hasta ahora se ha centrado en el intercambio. Con este esquema, segmentos enteros se transfieren entre la memoria y el disco por demanda. La otra forma de implementar la segmentación es dividir cada segmento en páginas de tamaño fijo y paginarlas por demanda. En este esquema, algunas de las páginas de un segmento podrían estar en la memoria y otras en disco. Para paginar un segmento se requiere una tabla de páginas individual para cada segmento. Puesto que un segmento no es más que un espacio de direcciones lineal, todas las técnicas que hemos visto hasta ahora para la paginación se aplican a cada segmento. La única novedad aquí es que cada segmento tiene su propia tabla de páginas.

Uno de los primeros sistemas operativos que combinaron la segmentación con la paginación fue **MULTICS (Sistema de Información y Cómputo Multiplexado, MULTiplexed Information and Computing Service)**, que inicialmente fue un proyecto conjunto de MIT, Bell Labs y General Electric (Corbató y Vyssotsky, 1995; y Organick, 1972). Las direcciones de MULTICS tenían dos partes: un número de segmento y una dirección dentro del segmento. Había un segmento de descriptores para cada proceso, que contenía un descriptor para cada segmento. Cuando se presentaba una dirección virtual al hardware, se usaba el número de segmento como índice del segmento de descriptores para localizar el descriptor del segmento accesado, como se muestra en la figura 6-11. El descriptor apuntaba a la tabla de páginas, la que permitía paginar cada segmento de la forma acostumbrada. Para mejorar el desempeño, las combinaciones segmento/página de uso más reciente se guardaban en una **memoria asociativa** de 16 entradas en hardware que permitía buscarlas rápidamente. Aun-

que hace mucho que MULTICS desapareció, su espíritu sigue vivo porque la memoria virtual de todas las CPU de Intel a partir del 386 ha seguido de cerca su modelo.

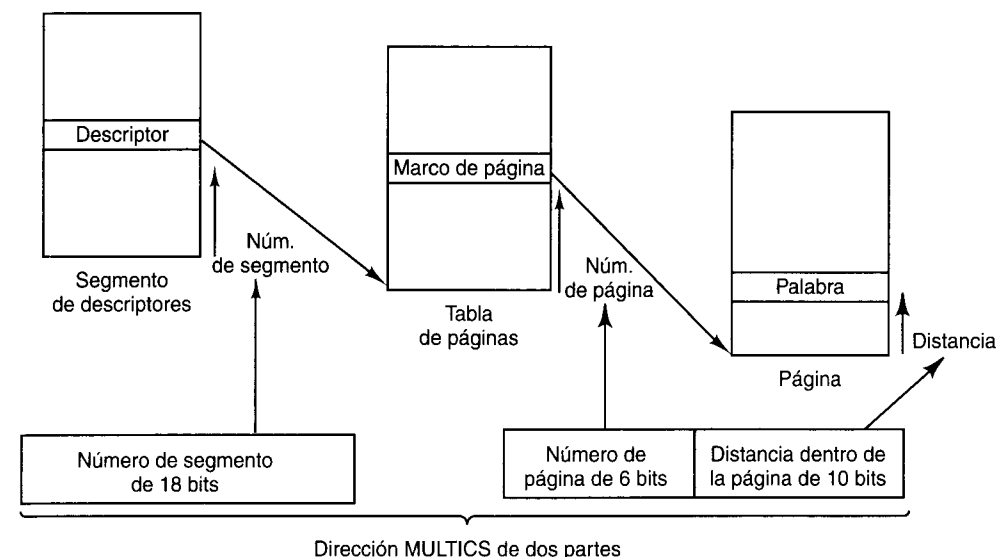


Figura 6-11. Conversión de una dirección MULTICS de dos partes en una dirección de memoria principal.

6.1.8 Memoria virtual en el Pentium II

El Pentium II tiene un sistema de memoria virtual avanzado que maneja paginación por demanda, segmentación pura y segmentación con paginación. El corazón de la memoria virtual del Pentium II consiste de dos tablas: la **tabla de descriptores locales (LDT, Local Descriptor Table)** y la **tabla de descriptores globales (GDT, Global Descriptor Table)**. Cada programa tiene su propia LDT, pero hay una sola GDT compartida por todos los programas de la computadora. La LDT describe los segmentos locales de cada programa, incluidos los de código, datos, pila, etc., mientras que la GDT describe los segmentos del sistema, incluido el sistema operativo mismo.

Como explicamos en el capítulo 5, para que un programa de Pentium II accese a un segmento, primero debe cargar un selector de ese segmento en uno de sus registros de segmento. Durante la ejecución, CS contiene el selector del segmento de código, DS contiene el selector del segmento de datos, etc. Cada selector es un número de 16 bits, como se muestra en la figura 6-12.

Uno de los bits del selector indica si el segmento es local o global (es decir, si está en la LDT o en la GDT). Otros 13 bits especifican el número de entrada de la LDT o GDT, así que estas tablas sólo pueden contener 8K (2^{13}) descriptores de segmentos. Los otros dos bits

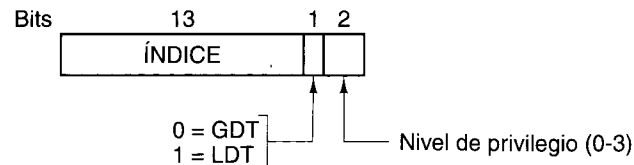


Figura 6-12. Selector de Pentium II.

tienen que ver con la protección y se describirán después. El descriptor 0 no es válido y su uso causa una trampa. Puede cargarse sin peligro en un registro de segmento para indicar que ese registro no está disponible, pero causa una trampa si se usa.

Cuando un selector se carga en un registro de segmento, el descriptor correspondiente se trae de la LDT o GDT y se guarda en registros internos de la MMU, para poder acceder a él rápidamente. Un descriptor consiste en 8 bytes e incluye la dirección base del segmento, su tamaño y otra información, como se muestra en la figura 6-13.

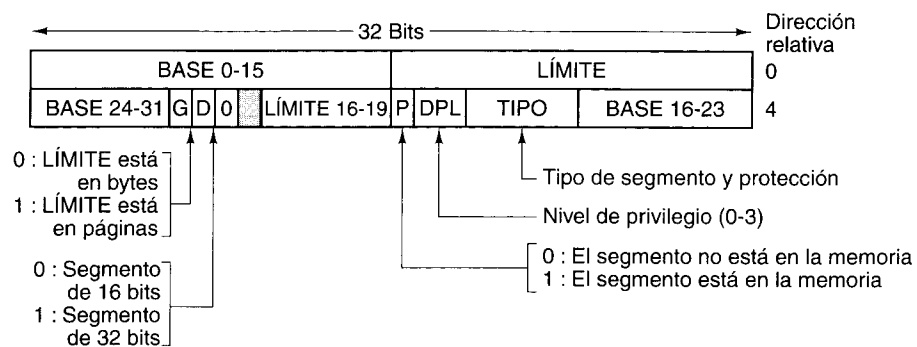


Figura 6-13. Descriptor de segmento de código de Pentium II. Los segmentos de datos son un poco diferentes.

El formato del selector se escogió ingeniosamente a modo de facilitar la localización del descriptor. Primero se selecciona la LDT o bien la GDT, con base en el bit 2 del selector. Luego el selector se copia en un registro de borrador de la MMU y los tres bits de orden bajo se ponen en 0, con lo que el número de selector de 13 bits se multiplica efectivamente por 8. Por último, se le suma la dirección de la tabla LDT o GDT (que se guarda en registros internos de la MMU) para dar un apuntador directo al descriptor. Por ejemplo, el selector 72 se refiere a la entrada 9 de la GDT, que se encuentra en la dirección $GDT + 72$.

Sigamos los pasos por los que un par (selector, distancia) se convierte en una dirección física. Tan pronto como el hardware sabe cuál registro de segmento se está usando, puede encontrar el descriptor completo que corresponde a ese selector en sus registros internos. Si el segmento no existe (selector 0) o no está en la memoria (P es 0), ocurre una trampa. El primer caso es un error de programación; el segundo obliga al sistema operativo a obtener el segmento.

Luego, el sistema verifica si la distancia rebasa el final del segmento, en cuyo caso ocurre otra trampa. Lógicamente, debería haber un campo de 32 bits en el descriptor que diera el

tamaño del segmento, pero sólo se cuenta con 20 bits, así que se emplea un esquema distinto. Si el campo G (Granularidad) es 0, el campo $LÍMITE$ da el tamaño exacto del segmento, hasta 1 MB; si es 1, $LÍMITE$ da el tamaño del segmento en páginas en lugar de bytes. El tamaño de página del Pentium II nunca es menor que 4 KB, por lo que 20 bits son suficientes para segmentos de hasta 2^{32} bytes.

Suponiendo que el segmento está en la memoria y la distancia no se sale del intervalo, el Pentium II suma el campo de 32 bits $BASE$ del descriptor a la distancia para formar una **dirección lineal**, como se muestra en la figura 6-14. El campo $BASE$ se descompone en tres fragmentos que se distribuyen por todo el descriptor por razones de compatibilidad con el 80286, en el que $BASE$ sólo tiene 24 bits. Así pues, el campo $BASE$ permite que un segmento inicie en cualquier punto dentro del espacio de direcciones lineal de 32 bits.

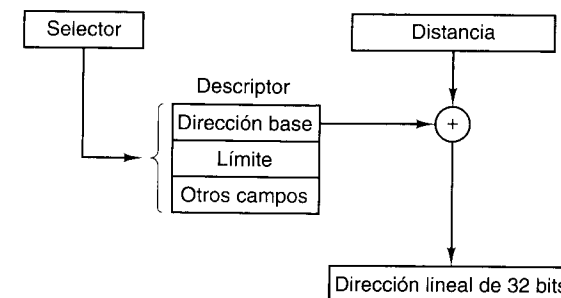


Figura 6-14. Conversión de un par (selector, distancia) en una dirección lineal.

Si se inhabilita la paginación (con un bit en un registro de control global), la dirección lineal se interpreta como la dirección física y se envía a la memoria para efectuar la lectura o escritura. Así, con la paginación desactivada, tenemos un esquema de segmentación pura, y la dirección base de cada segmento está dada en su descriptor. Por cierto, se permite el traslado de segmentos, tal vez porque sería demasiado problemático y tardado verificar que todos sean disjuntos.

Por otra parte, si la paginación está habilitada la dirección lineal se interpreta como una dirección virtual y se transforma en una dirección física mediante tablas de páginas, prácticamente igual que en nuestros ejemplos. La única complicación es que con una dirección virtual de 32 bits y páginas de 4 K, un segmento podría contener un millón de páginas, por lo que se usa un mapeo de dos niveles para reducir el tamaño de la tabla de páginas cuando los segmentos son pequeños.

Cada programa en ejecución tiene un **directorio de páginas** que consta de 1024 entradas de 32 bits y se encuentra en una dirección a la que un registro global apunta. Cada entrada de este directorio apunta a una tabla de páginas que también contiene 1024 entradas de 32 bits. Las entradas de la tabla de páginas apuntan a marcos de página. El esquema se muestra en la figura 6-15.

En la figura 6-15(a) vemos una dirección lineal desglosada en tres campos: **DIRECTORIO**, **PÁGINA** y **DISTANCIA**. El campo **DIRECTORIO** se usa primero como índice del directorio de páginas para localizar un apuntador a la tabla de páginas apropiada. Luego se

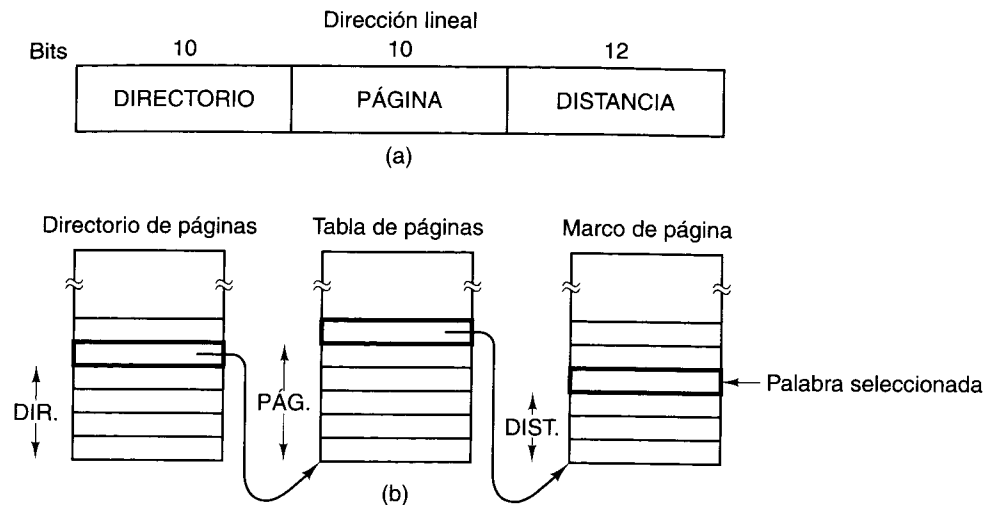


Figura 6-15. Transformación de una dirección lineal en una dirección física.

usa el campo **PÁGINA** como índice de la tabla de páginas para encontrar la dirección física del marco de página. Por último, **DISTANCIA** se suma a la dirección del marco de página para obtener la dirección física del byte o palabra direccionado.

Las entradas de la tabla de páginas tienen 32 bits cada una, de los cuales 20 contienen un número de marco de página. Los demás bits controlan el acceso e indican si la página está limpia o no. El hardware ajusta estos bits que son utilizados por el sistema operativo, los bits de protección y otros bits utilitarios.

Cada tabla de páginas tiene entradas para 1024 marcos de página de 4K, por lo que una sola tabla de páginas maneja 4 MB de memoria. Un segmento menor que 4M tendrá un directorio de páginas con una sola entrada, un apuntador a su única tabla de páginas. Así, el gasto extra en el caso de un segmento corto es de sólo dos páginas, en lugar del millón de páginas que se necesitaría en una tabla de páginas de un solo nivel.

A fin de evitar referencias repetidas a la memoria, la MMU del Pentium II cuenta con hardware especial que busca las combinaciones **DIRECTORIO-PÁGINA** utilizadas más recientemente y las transforma en la dirección física del marco de página correspondiente. Sólo se llevan a cabo los pasos de la figura 6-15 si la combinación actual no se usó recientemente.

Si nos ponemos a pensar un poco, veremos que si se usa paginación en realidad no tiene caso que el campo **BASE** del descriptor sea distinto de cero. Para lo único que sirve **BASE** es para desplazarse un poco dentro del directorio de páginas a fin de usar una entrada en ese punto en lugar de al principio. La verdadera razón para incluir **BASE** es hacer posible la segmentación pura (no paginada), y mantener la compatibilidad con el viejo 80286, que no tenía paginación.

También vale la pena señalar que si una aplicación dada no necesita segmentación y se conforma con un solo espacio de direcciones paginado de 32 bits, es fácil satisfacerla. Se asigna

a todos los registros de segmento el mismo selector, cuyo descriptor tiene **BASE = 0** y **LÍMITE** igual al máximo. Entonces, la distancia de la instrucción será la dirección lineal, empleando un solo espacio de direcciones; esto es, de hecho, paginación tradicional.

Con esto terminamos nuestro tratamiento de la memoria virtual en el Pentium II. Sin embargo, vale la pena hablar un poco de protección, ya que este tema está íntimamente relacionado con la memoria virtual. El Pentium II maneja cuatro niveles de protección, siendo el nivel 0 el que goza de más privilegios, y el nivel 3, el que menos. Dichos niveles se muestran en la figura 6-16. En un instante dado, un programa en ejecución está en cierto nivel, indicado por un campo de dos bits en su **palabra de estado del programa (PSW)**, un registro en hardware que contiene los códigos de condición y varios otros bits de estado. Además, cada segmento del sistema pertenece a un nivel dado.

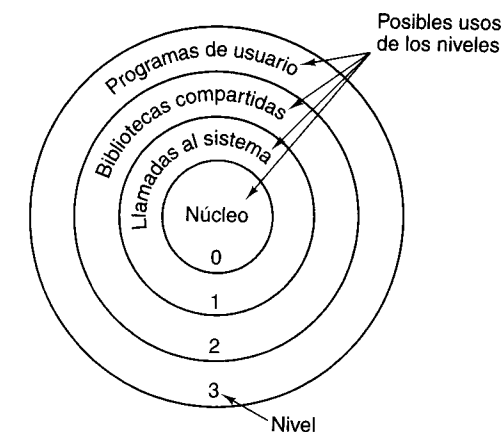


Figura 6-16. Protección en el Pentium II.

En tanto un programa se limite a usar segmentos que estén en su mismo nivel, todo funcionará de maravilla. Se permiten los intentos de acceder a datos que están en un nivel más alto, pero los intentos de acceder a datos en un nivel inferior están prohibidos y causan trampas. Los intentos por invocar procedimientos de otro nivel (más alto o más bajo) están permitidos, pero de forma muy controlada. Para efectuar una llamada internivel, la instrucción **CALL** debe contener un selector en lugar de una dirección. Este selector designa un descriptor llamado **puerta de llamada**, que da la dirección del procedimiento que se desea invocar. Así, no es posible saltar a la mitad de un segmento de código arbitrario en un nivel distinto; sólo pueden usarse los puntos de ingreso oficiales.

En la figura 6-16 se sugiere un posible uso de este mecanismo. En el nivel 0 está el núcleo del sistema operativo, que se encarga de la E/S, la gestión de memoria y otras tareas cruciales. En el nivel 1 está el manejador de llamadas al sistema. Los programas de usuario pueden invocar procedimientos de este nivel para solicitar que se efectúen llamadas al sistema, pero sólo es posible invocar una lista específica y protegida de procedimientos. El nivel 2 contiene procedimientos de biblioteca, posiblemente compartidos entre muchos programas

en ejecución. Los programas de usuario pueden invocar estos procedimientos, pero no modificarlos. Por último, los programas de usuario se ejecutan en el nivel 3, que es el que menos protección tiene. Al igual que el esquema de gestión de memoria del Pentium II, el sistema de protección se basa en el de MULTICS.

Las trampas e interrupciones usan un mecanismo similar a las puertas de llamada; también hacen referencia a descriptores, no a direcciones absolutas, y dichos descriptores apuntan a procedimientos específicos que deben ejecutarse. El campo TIPO de la figura 6-13 distingue entre segmentos de código, segmentos de datos y los diversos tipos de puertas.

6.1.9 Memoria virtual en el UltraSPARC II

El UltraSPARC II es una máquina de 64 bits y maneja una memoria virtual paginada basada en una dirección virtual de 64 bits. Sin embargo, por razones de ingeniería y costos, los programas no pueden usar todo el espacio de direcciones virtual de 64 bits. Sólo se reconocen 44 bits, por lo que los programas no pueden exceder 1.8×10^{13} bytes. La memoria virtual permitida se divide en dos zonas de 2^{43} bytes cada una, una en la parte superior del espacio de direcciones virtual y una en la parte inferior. En medio hay un hueco que contiene direcciones que no pueden usarse; un intento por usarlas causa un fallo de página.

La memoria física máxima en un UltraSPARC II es de 2^{41} bytes, que equivale a unos 2200 GB, lo bastante grande para casi todas las aplicaciones ordinarias. Se reconocen cuatro tamaños de página: 8 KB, 64 KB, 512 KB y 4 MB. En la figura 6-17 se ilustran las correspondencias implícitas en estos cuatro tamaños de página.

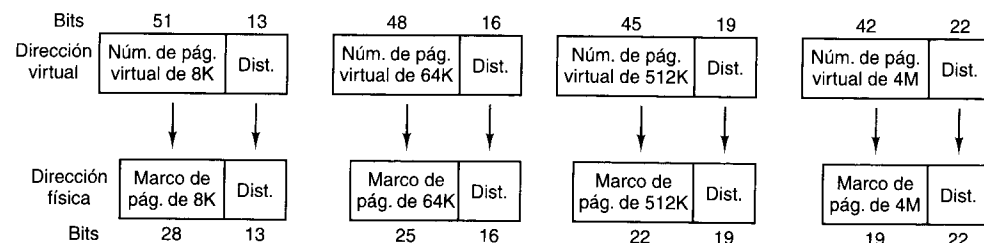


Figura 6-17. Transformaciones virtual-física en el UltraSPARC II.

Por lo extremadamente grande de espacio de direcciones virtual, una tabla de páginas sencilla como la del Pentium II no sería práctica. En vez de ello, la MMU del UltraSPARC II adopta una estrategia muy diferente: contiene una tabla en hardware llamada **buffer de consulta para traducción (TLB, Translation Lookaside Buffer)** que transforma números de página virtual en números de marco de página físico. Para el tamaño de página de 8K hay 2^{31} números de página virtual, o sea, más de dos millones. Es evidente que no todos pueden estar en el mapa.

En vez de ello, el TLB sólo contiene los números de página virtual más recientemente usados. Se sigue la pista por separado a las páginas de instrucciones y a las de datos, y el TLB guarda los 64 números de página virtual más recientemente usados de cada categoría. Cada entrada del TLB contiene un número de página virtual y el número de marco de página físico correspondiente. Cuando se presentan a la MMU un número de proceso, llamado **contexto**, y

una dirección virtual dentro de ese contexto, la MMU emplea circuitos especiales para comparar simultáneamente el número de página virtual contenido en la dirección virtual con todas las entradas del TLB. Si se encuentra una concordancia, el número de marco de página contenido en esa entrada del TLB se combina con la distancia tomada de la dirección virtual para formar una dirección física de 41 bits y producir algunas banderas, como los bits de protección. El TLB se ilustra en la figura 6-18(a).

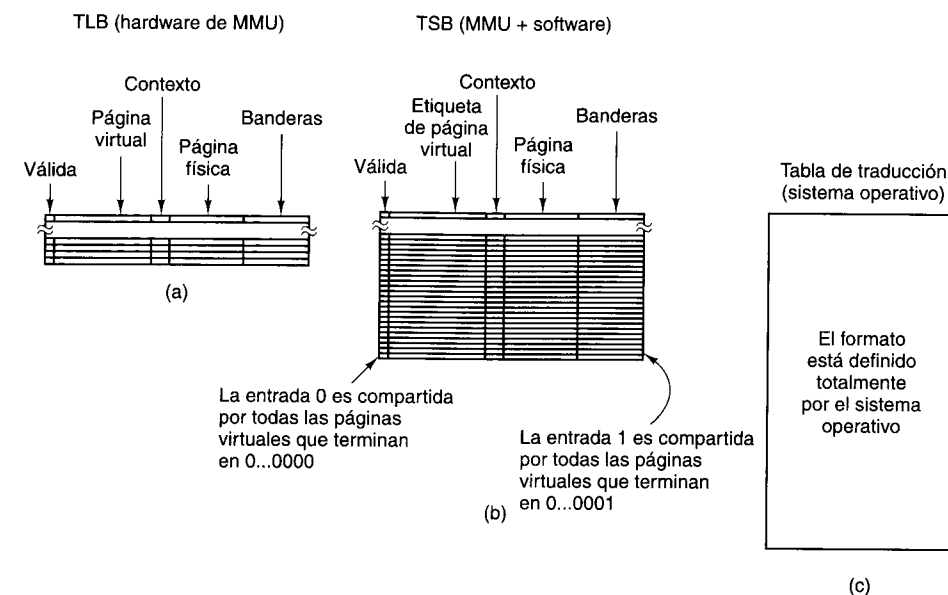


Figura 6-18. Estructuras de datos empleadas para traducir direcciones virtuales en el UltraSPARC. (a) TLB. (b) TSB. (c) Tabla de traducción.

Si ninguna de las entradas de la TLB concuerda, ocurre un **fallo de TLB**, que causa un salto al sistema operativo a través de una trampa. El manejo del fallo corresponde al sistema operativo. Cabe señalar que un fallo de TLB es muy diferente de un fallo de página. Puede ocurrir un fallo de TLB aunque la página a la que se hizo referencia esté en la memoria. En teoría, el sistema operativo puede hacer lo que quiera para cargar una nueva entrada de TLB para la página virtual requerida. Sin embargo, para acelerar esta operación crucial, se puede contar con cierta ayuda del hardware si es que el software coopera.

En particular, se espera que el sistema operativo mantenga una caché en software con las entradas de TLB más utilizadas en una tabla llamada **buffer de almacenamiento para traducción (TSB, Translation Storage Buffer)**. Esta tabla se organiza como caché de páginas virtuales con correspondencia directa. Cada entrada de TSB de 16 bytes se refiere a una página virtual y contiene un bit de validez, el número de contexto, la etiqueta de dirección virtual, el número de página física y algunos bits que sirven como banderas. Si el tamaño del caché es de, digamos, 8192 entradas, todas las páginas virtuales cuyos 13 bits de orden bajo sean 000000000000 competirán por la entrada 0 del TSB. Así mismo, todas las páginas virtuales cuyos bits de orden bajo sean 000000000001 competirán por la entrada 1 del TSB,

como se muestra en la figura 6-18(b). El tamaño del TSB lo determina el software y se comunica a la MMU por medio de registros especiales a los que sólo el sistema operativo tiene acceso.

Cuando ocurre un fallo de TLB, el sistema operativo verifica si la entrada correspondiente en el TSB contiene la página virtual requerida. La MMU ayuda aquí calculando la dirección de esta entrada y colocándola en un registro interno MMU accesible para el sistema operativo. Si ocurre un acierto de caché en el TSB, se borrará alguna entrada del TLB y la entrada de TSB requerida se copiará en el TLB. El hardware también ayuda a escoger cuál entrada del TLB se borrará utilizando un algoritmo LRU de un bit.

Si la búsqueda en el TSB no corre con suerte y la página virtual a la que se hizo referencia no está en la caché, el sistema operativo usa otra tabla para localizar la información acerca de la página, que podría o no estar en la memoria principal. La tabla que se usa para esta consulta de último recurso se llama **tabla de traducción**. Puesto que el hardware no ayuda con las consultas a esta tabla, el sistema operativo está en libertad de usar el formato que quiera. Por ejemplo, puede usar *hashing* dividiendo el número de página virtual entre algún número primo p , y usar el residuo como índice de una tabla de apuntadores, cada uno de los cuales apunta a una lista enlazada de entradas de página virtual que dan p por *hashing*. Cabe señalar que estas entradas no son las páginas, sino entradas de TSB. El resultado de buscar una página en la tabla de traducción podría ser localizar la página en la memoria, en cuyo caso se actualizará la entrada de TSB de la caché en software, pero también podría ser el descubrimiento de que la página no está en la memoria, en cuyo caso se inicia la acción estándar de fallo de página.

Resulta interesante comparar los esquemas de paginación del Pentium II y el UltraSPARC II. El Pentium II maneja segmentación pura, paginación pura y segmentos paginados. El UltraSPARC II sólo tiene paginación. El Pentium II también usa hardware para recorrer la tabla de páginas y volver a cargar el TLB en caso de un fallo de TLB. El UltraSPARC II simplemente transfiere el control al sistema operativo cuando hay un fallo de TLB.

La razón primordial de esta diferencia es que el Pentium II usa segmentos de 32 bits, y tales segmentos tan pequeños (sólo un millón de páginas) pueden manejarse con tablas de páginas convencionales. En teoría, el Pentium II tendría problemas si un programa usara miles de segmentos, pero dado que ninguna versión de Windows ni de UNIX reconoce más de un segmento por proceso, el problema no se presenta. En contraste, el UltraSPARC II es una máquina de 64 bits y puede tener hasta 2000 millones de páginas, por lo que las tablas de páginas convencionales no funcionan. En el futuro, todas las máquinas tendrán espacios de direcciones virtuales de 64 bits, por lo que esquemas como el del UltraSPARC II se convertirán en la norma. En (Jacob y Mudge, 1998b) se hace una comparación del Pentium II, el UltraSPARC II y otros cuatro esquemas de memoria virtual.

6.1.10 Memoria virtual y uso de cachés

Aunque a primera vista no parece haber relación entre la memoria virtual (paginada por demanda) y el uso de cachés, en lo conceptual son muy similares. Con memoria virtual todo el programa se mantiene en disco y se divide en páginas de tamaño fijo. Un subconjunto de

estas páginas está en la memoria principal. Si el programa usa casi todo el tiempo las páginas que están en la memoria, habrá pocos fallos de página y el programa trabajará rápidamente. Con cachés, todo el programa se mantiene en la memoria principal y se divide en bloques de caché de tamaño fijo. Un subconjunto de estos bloques está en el caché. Si el programa usa casi todo el tiempo los bloques que están en el caché, habrá pocos fallos de caché y el programa trabajará rápidamente. En lo conceptual, las dos cosas son idénticas, excepto que operan en diferentes niveles de la jerarquía.

Desde luego, la memoria virtual y el uso de cachés también tienen ciertas diferencias. Una de ellas es que los fallos de caché son manejados por el hardware, mientras que los fallos de página son manejados por el sistema operativo. Además, los bloques de caché suelen ser mucho más pequeños que las páginas (por ejemplo, 64 bytes *versus* 8 KB). Además, la correspondencia entre páginas virtuales y marcos de página es diferente, y las tablas de páginas se organizan usando como índice los bits de orden alto de la dirección virtual, mientras que las cachés usan como índice los bits de orden bajo de la dirección de memoria. Pese a esto, es importante darse cuenta de que éstas son diferencias de implementación. El concepto subyacente es muy similar.

6.2 INSTRUCCIONES DE E/S VIRTUALES

El conjunto de instrucciones en el nivel ISA es totalmente distinto del conjunto de instrucciones de la microarquitectura. Tanto las operaciones que pueden efectuarse como los formatos de las instrucciones son muy diferentes en los dos niveles. La existencia de unas cuantas instrucciones que son iguales en ambos niveles es en esencia accidental.

En contraste, el conjunto de instrucciones del nivel OSM contiene casi todas las instrucciones del nivel ISA, con la adición de unas cuantas instrucciones nuevas pero importantes y la eliminación de unas cuantas instrucciones que podrían ser perjudiciales. La entrada/salida es una de las áreas en la que los dos niveles difieren considerablemente. La razón de esta diferencia es sencilla: un usuario que puede ejecutar las verdaderas instrucciones de E/S del nivel ISA podría leer datos confidenciales almacenados en cualquier lugar del sistema, escribir en las terminales de otros usuarios y, en general, causar muchos estropicios además de ser una amenaza para la seguridad del sistema. Además, los programadores normales que están en sus cabales no quieren efectuar ellos mismos la E/S en el nivel ISA porque ello es en extremo tedioso y complejo. Se efectúa asignando valores a campos y bits en diversos registros de dispositivo, esperando hasta que la operación se lleva a cabo, y verificando después qué sucedió. Como ejemplo de esto último, los discos por lo regular tienen bits de registros de dispositivo que detectan los siguientes errores, entre muchos otros:

1. El brazo del disco no realizó correctamente la búsqueda.
2. Se especificó memoria inexistente como buffer.
3. La E/S de disco inició antes de que terminara la anterior.
4. Error de temporización al leer.
5. Se direccionó un disco inexistente.

6. Se direccionó un cilindro inexistente.
7. Se direccionó un sector inexistente.
8. Error de suma de verificación al leer.
9. Error de verificación de escritura después de operación de escritura.

Cuando ocurre uno de estos errores, se enciende el bit correspondiente de un registro de dispositivo. Pocos usuarios quieren ocuparse de seguir la pista a todos estos bits de error y a una gran cantidad de información de estado adicional.

6.2.1 Archivos

Una forma de organizar la E/S virtual es usar una abstracción llamada **archivo**. En su forma más sencilla, un archivo consiste en una secuencia de bytes que se escriben en un dispositivo de E/S. Si el dispositivo de E/S es un dispositivo de almacenamiento, como un disco, el archivo se podrá leer posteriormente; claro que si no se trata de un dispositivo de almacenamiento (por ejemplo, una impresora), no podrá leerse. Un disco puede contener muchos archivos, cada uno con un tipo de datos específico; por ejemplo, una imagen, una hoja de cálculo o el texto del capítulo de un libro. Los diferentes archivos tienen diferentes longitudes y otras propiedades. La abstracción de archivo permite organizar la E/S virtual de una forma sencilla.

Para el sistema operativo, un archivo normalmente es sólo una secuencia de bytes, como dijimos antes. Cualquier estructura adicional es asunto de los programas de aplicación. La E/S con archivos se efectúa con llamadas al sistema para abrir, leer, escribir y cerrar archivos. Antes de leer un archivo, es preciso abrirlo. El proceso de abrir un archivo permite al sistema operativo localizar el archivo en el disco y traer a la memoria la información necesaria para acceder a él.

Una vez que se ha abierto un archivo, puede leerse. La llamada al sistema **read** debe tener, como mínimo, los siguientes parámetros:

1. Una indicación de cuál archivo abierto debe leerse.
2. Un apuntador a un buffer en la memoria en el que se colocarán los datos.
3. El número de bytes que se leerán.

La llamada **read** coloca los datos solicitados en el buffer, y por lo regular devuelve el número de bytes que se leyeron realmente, que podría ser menor que el número solicitado (no es posible leer 2000 bytes de un archivo de 1000 bytes).

Todo archivo abierto tiene asociado un apuntador que indica cuál byte se leerá a continuación. Después de un **read**, el apuntador se incrementa en el número de bytes leídos, por lo que **reads** consecutivos leen bloques consecutivos de datos del archivo. Por lo regular hay una forma de asignar a este apuntador un valor específico para que los programas puedan acceder aleatoriamente a cualquier parte del archivo. Cuando un programa termina de leer un archivo, puede cerrarlo para informar al sistema operativo que ya no usará el archivo y que puede liberar el espacio en tablas que se estaban usando para contener la información acerca del archivo.

Los sistemas operativos de *mainframes* tienen una idea más sofisticada de lo que es un archivo. Ahí, un archivo puede ser una sucesión de **registros lógicos**, cada uno con una estructura bien definida. Por ejemplo, un registro lógico podría ser una estructura de datos que consiste en cinco elementos: dos cadenas de caracteres, "Nombre" y "Supervisor"; dos enteros, "Departamento" y "Oficina" y un booleano, "SexoEsFemenino". Algunos sistemas operativos distinguen entre archivos en los que todos los registros tienen la misma estructura y los que contienen una mezcla de diferentes tipos de registros.

La instrucción de entrada virtual básica lee el siguiente registro del archivo especificado y lo coloca en la memoria principal a partir de una dirección especificada, como se ilustra en la figura 6-19. Para realizar esta operación, hay que indicar a la instrucción virtual cuál archivo debe leer y en qué lugar de la memoria debe colocar el registro. Es común que haya opciones para leer un registro en particular, especificado sea por su posición en el archivo o por su clave.

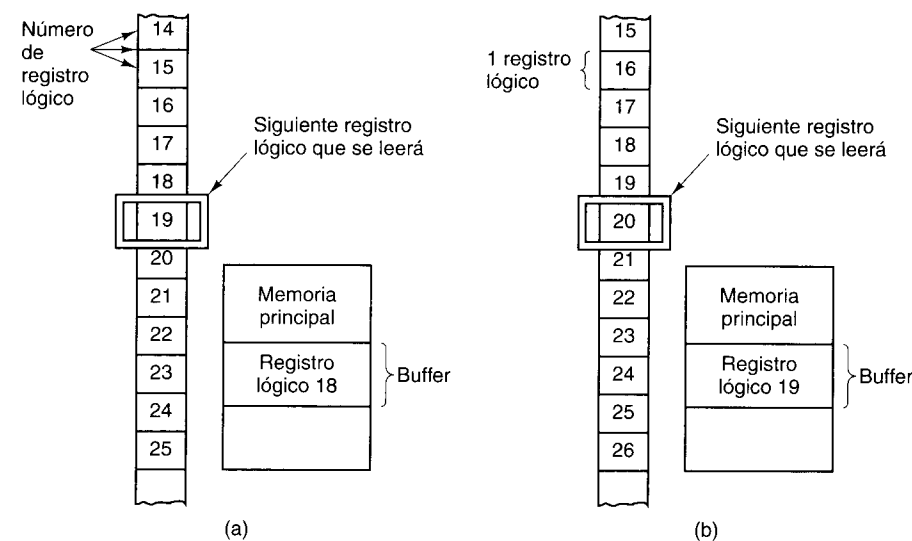


Figura 6-19. Lectura de un archivo que consiste en registros lógicos. (a) Antes de leer el registro 19. (b) Después de leer el registro 19.

La instrucción de salida virtual básica escribe un registro lógico de la memoria en un archivo. Instrucciones **write** consecutivas producen registros lógicos consecutivos en el archivo.

6.2.2 Implementación de instrucciones de E/S virtuales

Para entender cómo se implementan las instrucciones de E/S virtuales, es necesario examinar la organización y almacenamiento de los archivos. Una cuestión básica que debe resolverse con todos los sistemas de archivos es la asignación de almacenamiento. La unidad de asignación puede ser un solo sector de disco, pero es común que consista en un bloque de sectores consecutivos.

Otra propiedad fundamental de una implementación de sistema de archivos es si un archivo se almacena en unidades de asignación consecutivas o no. La figura 6-20 muestra un disco sencillo con una superficie que consiste en cinco pistas de 12 sectores cada una. La figura 6-20(a) muestra un esquema de asignación en el que el sector es la unidad básica de asignación de espacio y un archivo consiste en sectores consecutivos. Es común usar la asignación consecutiva de bloques de archivo en los CD-ROM. La figura 6-20(b) muestra un esquema de asignación en el que el sector es la unidad básica de asignación pero en el que un archivo no tiene que ocupar sectores consecutivos. Este esquema es la norma en los discos duros.

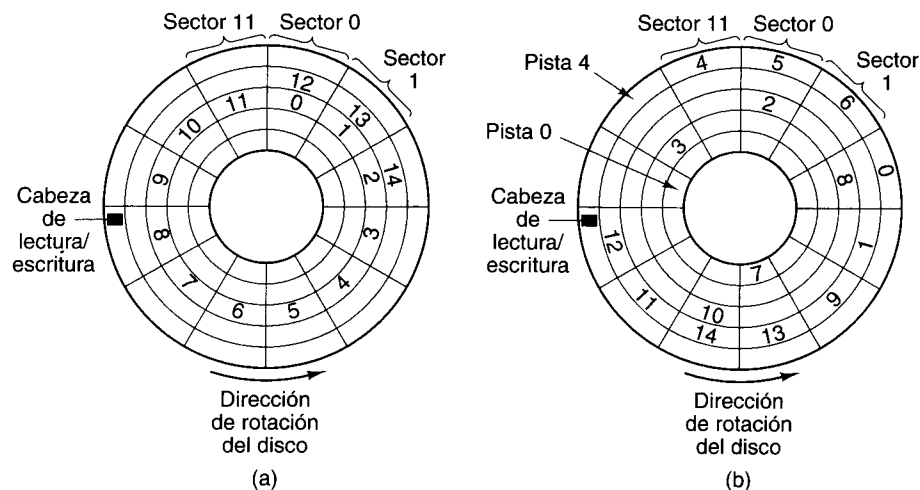


Figura 6-20. Estrategias de asignación de disco. (a) Un archivo en sectores consecutivos. (b) Un archivo en sectores no consecutivos.

Existe una diferencia importante entre la forma como un programador de aplicaciones ve un archivo y la forma en que lo ve el sistema operativo. El programador ve el archivo como una secuencia lineal de bytes o palabras lógicas. El sistema operativo ve el archivo como una colección ordenada, aunque no necesariamente consecutiva, de unidades de asignación en el disco.

Para que el sistema operativo entregue el byte o el registro lógico n de algún archivo cuando se le solicite, debe tener algún método para localizar los datos. Si el archivo se asigna consecutivamente, el sistema operativo sólo tiene que conocer la posición del principio del archivo para poder calcular la posición del byte o registro lógico requerido.

Si el archivo no se asigna consecutivamente, no es posible calcular la posición de un byte o registro lógico arbitrario sólo a partir de la posición del principio del archivo. Para localizar cualquier byte o registro lógico se necesita una tabla llamada **índice de archivos** que da las unidades de asignación y sus direcciones de disco reales. El índice de archivos se puede organizar como una lista de direcciones de bloques de disco (como en UNIX) o como lista de registros lógicos, dando la dirección en disco y la distancia para cada uno. A veces cada registro lógico tiene una **clave** y los programas pueden referirse a un registro por su clave, en lugar de por su número de registro lógico. En este caso se requiere la segunda organización, en la que

cada entrada contiene no sólo la ubicación del registro en el disco, sino también su clave. Esta organización es común en las *mainframes*.

Un método alternativo para localizar las unidades de asignación de un archivo es organizar el archivo como lista enlazada. Cada unidad de asignación contiene la dirección de su sucesora. Una forma de implementar este esquema de forma eficiente es guardar en la memoria principal la tabla con las direcciones de todas las sucesoras. Por ejemplo, en el caso de un disco con unidades de asignación de 64K, el sistema operativo podría tener una tabla en la memoria con 64K entradas, cada una de las cuales da el índice de su sucesora. Por ejemplo, si un archivo ocupara las unidades de asignación 4, 52 y 19, la entrada 4 de la tabla contendría un 52, la entrada 52 contendría un 19 y la entrada 19 contendría un código especial (digamos 0 o -1) para indicar el fin del archivo. Los sistemas de archivos empleados por MS-DOS y Windows 95 y Windows 98 funcionan así. Windows NT reconoce este sistema de archivos pero además tiene su propio sistema de archivos nativo que funciona de forma más parecida al de UNIX.

Hasta ahora hemos hablado de archivos asignados consecutiva y no consecutivamente, pero no hemos explicado por qué se usan ambos tipos. La administración de bloques de los archivos asignados consecutivamente es más sencilla, pero cuando no se conoce por adelantado el tamaño máximo del archivo casi nunca es posible usar esta técnica. Si un archivo iniciara en el sector j y pudiera crecer hacia sectores consecutivos, podría toparse con otro archivo en el sector k y no tener más espacio para expandirse. Si el archivo no se asigna consecutivamente, esta situación no representa un problema, porque los bloques subsecuentes se pueden colocar en cualquier lugar del disco. Si un disco contiene varios archivos que están creciendo, y no se conoce el tamaño final de ninguno de ellos, almacenarlos como archivos consecutivos será casi imposible. A veces es posible cambiar de lugar un archivo existente, pero siempre es costoso.

Por otra parte, si se conoce por adelantado el tamaño máximo de todos los archivos, como sucede cuando se graba un CD-ROM, el programa de grabación puede preasignar una serie de sectores cuya longitud es exactamente igual a la de cada archivo. Así pues, si se van a colocar archivos con longitudes de 1200, 700, 2000 y 900 sectores en un CD-ROM, simplemente se inician en los sectores 0, 1200, 1900 y 3900, respectivamente (si nos olvidamos de la tabla de contenido). Encontrar cualquier parte de cualquier archivo es fácil si se conoce el primer sector del archivo.

Para asignar espacio de disco a un archivo, el sistema operativo debe saber cuáles bloques están disponibles y en cuáles ya están almacenados otros archivos. En el caso de un CD-ROM, el cálculo se efectúa de una vez por todas antes de la grabación, pero en el caso de un disco duro los archivos entran y salen constantemente. Un método consiste en mantener una lista de todos los huecos, donde un hueco es cualquier cantidad de unidades de asignación contiguas. Esta lista se llama **lista libre**. En la figura 6-21(a) se ilustra la lista libre del disco de la figura 6-20(b).

Un método alternativo consiste en mantener un mapa de bits, con un bit por unidad de asignación, como se muestra en la figura 6-21(b). Un bit 1 indica que la unidad de asignación ya está ocupada, y un bit 0 indica que está disponible.

El primer método tiene la ventaja de que facilita la localización de un hueco de cierto tamaño, pero tiene la desventaja de que el tamaño de la lista es variable. A medida que se

Pista	Sector	Número de sectores en el hueco
0	0	5
0	6	6
1	0	10
1	11	1
2	1	1
2	3	3
2	7	5
3	0	3
3	9	3
4	3	8

(a)

	Sector											
Pista	0	1	2	3	4	5	6	7	8	9	10	11
0	0	0	0	0	0	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1	0
2	1	0	1	0	0	0	1	0	0	0	0	0
3	0	0	0	1	1	1	1	1	1	0	0	0
4	1	1	1	0	0	0	0	0	0	0	0	1

(b)

(a)

(b)

Figura 6-21. Dos formas de llevar la contabilidad de los sectores disponibles.
(a) Una lista libre. (b) Un mapa de bits.

crean y destruyen archivos, la longitud de la lista fluctúa, característica que es indeseable. El mapa de bits tiene la ventaja de tener un tamaño constante. Además, para modificar la situación de una unidad de asignación, de disponible a ocupada, sólo requiere modificar un bit. Por otra parte, no es fácil encontrar un bloque de un tamaño dado. Ambos métodos requieren que, cuando se asigna o borra cualquier archivo del disco, la lista o tabla de asignación se actualice.

Antes de abandonar el tema de la implementación de sistemas de archivos, vale la pena comentar algo acerca del tamaño de la unidad de asignación. En este sentido hay varios factores importantes. Primero, el tiempo de búsqueda y el retraso rotacional dominan los accesos a disco. Habiendo invertido 10 ms en llegar al principio de una unidad de asignación, es mucho mejor leer 8 KB (en aproximadamente 1 ms) que 1 KB (en aproximadamente 0.125 ms), ya que leer 8 KB como ocho unidades de 1 KB requeriría ocho búsquedas. La eficiencia de transferencia es un argumento a favor de tener unidades de asignación grandes.

Otro argumento a favor es que tener unidades de asignación pequeñas también implica tener muchas de ellas. Esto, a su vez, implica que los índices de archivos o las tablas de lista enlazada en la memoria serán grandes. De hecho, la razón por la que MS-DOS tuvo que cambiar a unidades de asignación de varios sectores fue el hecho de que las direcciones en disco se guardaban como números de 16 bits. Cuando los discos comenzaron a tener más de 64K sectores, la única forma de representarlos era usar unidades de asignación más grandes, para que el número de unidades de asignación no excediera 64K. La primera versión de Windows 95 tenía el mismo problema, pero una versión subsecuente usó números de 32 bits. Windows 98 reconoce ambos tamaños.

Por otra parte, un argumento en favor de tener unidades de asignación pequeñas es el hecho de que pocos archivos ocupan un número entero de unidades de asignación. Por tanto, se desperdiciará algo de espacio en la última unidad de asignación de casi todos los archivos. Si el archivo es mucho mayor que la unidad de asignación, el espacio desperdiciado medio será la mitad de una unidad de asignación. Cuanto mayor sea la unidad de asignación, mayor será la cantidad de espacio desperdiciado. Si en promedio los archivos son mucho más pequeños

que la unidad de asignación, se desperdiciará la mayor parte del espacio de disco. Por ejemplo, en una partición de disco de 2 GB de MS-DOS o Windows 95 versión 1, las unidades de asignación son de 32 KB, por lo que un archivo de 100 caracteres desperdicia 32,668 bytes de espacio de disco. La eficiencia de almacenamiento es un argumento en favor de usar unidades de asignación pequeñas. En la actualidad, la eficiencia de transferencia suele ser el factor más importante, por lo que hay una tendencia de los tamaños de bloque a aumentar con el tiempo.

6.2.3 Instrucciones para gestión de directorios

En los albores de la computación, la gente guardaba sus programas y datos en tarjetas perforadas en archiveros en sus oficinas. A medida que el tamaño y el número de los programas y datos creció, esta situación se volvió cada vez más indeseable y llevó finalmente a la idea de usar la memoria secundaria de la computadora (por ejemplo, los discos) como lugar de almacenamiento para los programas y datos. La información a la que la computadora puede acceder directamente sin necesidad de intervención humana se dice que está **en línea**, en contraste con la información **fuera de línea**, que requiere intervención humana (por ejemplo, insertar un CD-ROM) para que la computadora pueda acceder a ella.

La información en línea se guarda en archivos, y los programas pueden acceder a ella usando las instrucciones de E/S de archivos que acabamos de explicar. Sin embargo, se necesitan instrucciones adicionales para seguir la pista a toda la información almacenada en línea, reunirlos en unidades convenientes y protegerla contra el uso no autorizado.

La forma en que un sistema operativo acostumbra organizar los archivos en línea es agrupándolos en **directorios**. En la figura 6-22 se muestra un ejemplo de organización de directorios. Se incluyen llamadas al sistema para efectuar al menos las siguientes funciones:

1. Crear un archivo e introducirlo en un directorio.
2. Borrar un archivo de un directorio.
3. Cambiar el nombre de un archivo.
4. Modificar la situación de protección de un archivo.

Se usan diversos esquemas de protección. Una posibilidad es que el dueño de cada archivo especifique una contraseña secreta para cada archivo. Al intentar acceder a un archivo, un programa debe proporcionar la contraseña, que el sistema operativo revisa para ver si es correcta antes de permitir el acceso. Otro método de protección es que el dueño de cada archivo proporcione una lista explícita de las personas cuyos programas pueden acceder a ese archivo.

Todos los sistemas operativos modernos permiten a los usuarios mantener más de un directorio de archivos. Por lo regular, cada directorio es en sí un archivo y, como tal, puede listarse en otro directorio, lo que da pie a un árbol de directorios. Tener varios directorios es útil sobre todo para los programadores que trabajan en varios proyectos. Así, pueden agrupar en un directorio todos los archivos relacionados con un proyecto. Mientras trabajan en ese proyecto, no se distraerán por la presencia de archivos que nada tienen que ver. Los directorios también son una forma cómoda de compartir archivos con los miembros de un grupo.

Archivo 0	Nombre: Patio
Archivo 1	Longitud: 1840
Archivo 2	Tipo: Anatidae dataram
Archivo 3	Fecha creado: Marzo 16 de 1066
Archivo 4	Último acceso: Septiembre 1 de 1492
Archivo 5	Último cambio: Julio 4 de 1776
Archivo 6	Total de accesos: 144
Archivo 7	Bloque 0: Pista 4 Sector 6
Archivo 8	Bloque 1: Pista 19 Sector 9
Archivo 9	Bloque 2: Pista 11 Sector 2
Archivo 10	Bloque 3: Pista 77 Sector 0

Figura 6-22. (a) Directorio de archivos de usuario. (b) Contenido de una entrada típica de un directorio de archivos.

6.3 INSTRUCCIONES VIRTUALES PARA PROCESAMIENTO EN PARALELO

Algunos cálculos se pueden programar de forma más conveniente para dos o más procesos cooperativos que se ejecutan en paralelo (es decir, simultáneamente en diferentes procesadores), que para un solo proceso. Otros cálculos se pueden dividir en fragmentos, que entonces se pueden ejecutar en paralelo a fin de reducir el tiempo requerido para el cálculo total. Para que varios procesos trabajen juntos de forma paralela se requieren ciertas instrucciones virtuales, que analizaremos en las secciones que siguen.

Las leyes de la física proporcionan una justificación más del interés actual en el procesamiento en paralelo. Según la teoría de la relatividad especial de Einstein, es imposible transmitir señales eléctricas a una velocidad mayor que la de la luz, que es de casi 1 pie/ns en el vacío, y menos en alambre de cobre o fibra óptica. Este límite tiene importantes implicaciones para la organización de las computadoras. Por ejemplo, si una CPU necesita datos de una memoria principal que está a 1 pie de distancia, la solicitud tardará al menos 1 ns en llegar a la memoria y la respuesta tardará otro nanosegundo en llegar a la CPU. Por consiguiente, las computadoras que quieran operar con tiempos de menos de un nanosegundo tendrán que ser muy pequeñas. Una estrategia alternativa para acelerar las computadoras es construir máquinas con muchas CPU. Una computadora con mil CPU de 1 ns podría tener la misma potencia de cómputo que una CPU con un tiempo de ciclo de 0.001 ns, pero la construcción de la primera podría ser mucho más fácil y económica.

En una computadora con más de una CPU, cada uno de varios procesos cooperativos puede asignarse a su propia CPU, para que los procesos puedan avanzar simultáneamente. Si sólo se cuenta con un procesador, el efecto del procesamiento en paralelo puede simularse haciendo que el procesador ejecute cada proceso por turno durante un tiempo corto. En otras palabras, varios procesos pueden compartir el procesador.

La figura 6-23 muestra la diferencia entre el verdadero procesamiento en paralelo, con más de un procesador físico, y el procesamiento paralelo simulado, con un solo procesador físico. Aun cuando se simula el procesamiento paralelo, es útil pensar que cada proceso tiene su propio procesador virtual dedicado. En el caso simulado surgen los mismos problemas de comunicación que se presentan cuando hay un verdadero procesamiento paralelo.

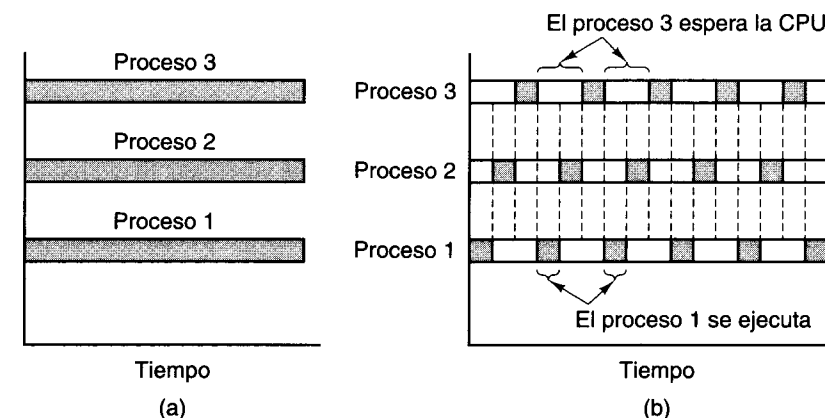


Figura 6-23. (a) Verdadero procesamiento paralelo con múltiples CPU. (b) Procesamiento paralelo simulado por conmutación de una CPU entre tres procesos.

6.3.1 Creación de procesos

Cuando se va a ejecutar un programa, debe operar como parte de algún proceso. Este proceso, como todos los demás, se caracteriza por un estado y un espacio de direcciones a través del cual se puede acceder al programa y los datos. El estado incluye como mínimo el contador de programa, una palabra de estado del programa, un apuntador de pila y los registros generales.

Casi todos los sistemas operativos modernos permiten crear y terminar procesos dinámicamente. Para aprovechar plenamente esta característica y lograr el procesamiento en paralelo, se requiere una llamada al sistema para crear un proceso nuevo. Esta llamada al sistema podría limitarse a crear un clon del proceso creador, o podría permitir al proceso creador especificar el estado inicial del nuevo proceso, incluido su programa, datos y dirección de inicio.

En algunos casos, el proceso creador (padre) mantiene un control parcial o incluso total del proceso creado (hijo). Con este fin, se incluyen instrucciones virtuales para que un padre pare, reinicie, examine y termine sus hijos. En otros casos, el padre tiene menos control sobre sus hijos. Una vez que se ha creado un proceso, el padre no tiene manera de detenerlo, reiniciarlo, examinarlo o terminarlo forzosamente. En tal caso, los procesos se ejecutan de forma independiente uno del otro.

6.3.2 Condiciones de competencia

En muchos casos los procesos paralelos necesitan comunicarse y sincronizarse para poder efectuar su trabajo. En esta sección examinaremos la sincronización de procesos y explicaremos algunas de las dificultades con la ayuda de un ejemplo detallado. En la sección que sigue se presentará una solución a estas dificultades.

Considere una situación que consiste en dos procesos independientes, el 1 y el 2, que se comunican por medio de un buffer compartido en la memoria principal. Para simplificar, llamaremos al proceso 1 el **productor** y al proceso 2 el **consumidor**. El productor calcula números primos y los coloca en el buffer, uno por uno. El consumidor los saca del buffer uno por uno y los imprime.

Estos dos procesos se ejecutan en paralelo con diferente velocidad. Si el productor descubre que el buffer está lleno, se “duerme”; es decir, suspende temporalmente su operación en espera de una señal del consumidor. Más adelante, una vez que el consumidor ha sacado un número del buffer, envía una señal al productor para despertarlo; es decir, reiniciarlo. Así mismo, si el consumidor descubre que el buffer está vacío, se duerme. Una vez que el productor ha colocado un número en el buffer vacío, despierta al consumidor dormido.

En este ejemplo usaremos un buffer circular para la comunicación entre procesos. Los apuntadores *in* y *out* se usarán como sigue: *in* apunta a la siguiente palabra desocupada (donde el productor pondrá el siguiente número primo) y *out* apunta al siguiente número que el consumidor quitará. Si *in* = *out*, el buffer está vacío, como se muestra en la figura 6-24(a). Una vez que el productor ha generado algunos primos, la situación es la que se muestra en la figura 6-24(b). La figura 6-24(c) ilustra el buffer después de que el consumidor ha sacado algunos de estos números primos para imprimirlos. La figura 6-24(d)-(f) muestra el efecto de una actividad de buffer constante. El tope del buffer es lógicamente contiguo a su base; es decir, el buffer da la vuelta. Si hubo una ráfaga repentina de entradas e *in* dio la vuelta hasta estar sólo una palabra detrás de *out* (por ejemplo, *in* = 52 y *out* = 53), el buffer está lleno. La última palabra no se usa; si se usara, no habría forma de saber si *in* = *out* significa que el buffer está lleno o que está vacío.

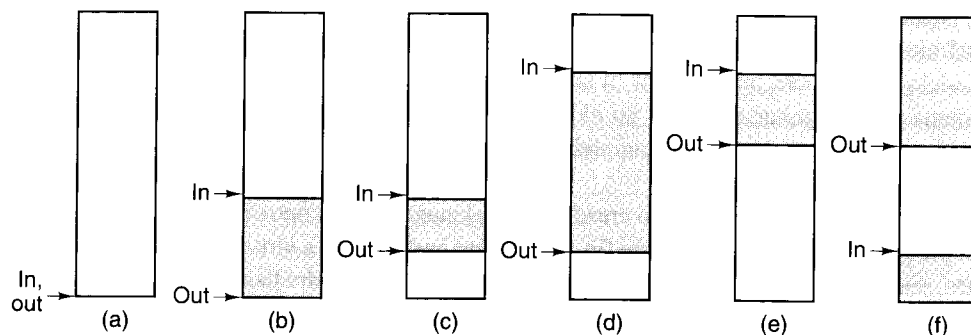


Figura 6-24. Uso de un buffer circular.

La figura 6-25 muestra una forma sencilla de implementar el problema del productor y el consumidor en Java. Esta solución usa tres clases, *m*, *productor* y *consumidor*. La clase *m* (“main”), contiene algunas definiciones de constantes, los apuntadores *in* y *out*, y el buffer mismo, que en este ejemplo contiene 100 números primos: *buffer*[0] a *buffer*[99].

Esta solución usa enlaces de Java para simular procesos paralelos. Con esta solución tenemos una clase *productor* y una clase *consumidor*, que se ejemplifican en las variables *p* y *c*, respectivamente. Ambas clases se derivan de una clase base *Thread* (enlace), que tiene un método *run* (ejecutar). La clase *run* contiene el código del enlace. Cuando se invoca el método *start* de un objeto derivado de *Thread*, se inicia un nuevo enlace.

Cada enlace es como un proceso, excepto que todos los enlaces dentro de un solo programa Java se ejecutan en el mismo espacio de direcciones. Esta característica es conveniente para que todos compartan un buffer común. Si la computadora tiene dos o más CPU, cada enlace se puede planificar en una CPU distinta, y se logra un verdadero paralelismo. Si sólo hay una CPU, los enlaces se ejecutan por tiempo compartido en ella. Seguiremos refiriéndonos al productor y al consumidor como procesos (ya que lo que realmente nos interesa son los procesos paralelos), aunque Java sólo reconozca enlaces paralelos y no procesos verdaderamente paralelos.

La función *siguiente* permite incrementar fácilmente a *in* y *out*, sin tener que escribir código que verifique la condición de continuidad del tope al fondo en cada ocasión. Si el parámetro de *siguiente* es 98 o menos, se devuelve el siguiente entero mayor, pero si el parámetro es 99 quiere decir que llegamos al final del buffer, y se devuelve 0.

Necesitamos un mecanismo que permita a cualquiera de los procesos dormirse cuando no pueda continuar. Los diseñadores de Java entendieron la necesidad de una capacidad así e incluyeron los métodos *suspend* (suspender, para dormirse) y *resume* (reanudar, para despertarse) en la clase *Thread* desde la primera versión de Java. Estos métodos se usan en la figura 6-25.

Ahora llegamos al código para el productor y el consumidor. Primero, el productor genera un primo nuevo en P1. Observe el uso de *m.MAX_PRIMO* aquí. El prefijo *m.* es necesario para indicar que estamos hablando del *MAX_PRIMO* definido en la clase *m*. Por la misma razón necesitamos anteponer este prefijo también a *in*, *out*, *buffer* y *siguiente*.

Luego el productor verifica (en P2) si *in* está una posición atrás de *out*. Si así es (por ejemplo, si *in* = 62 y *out* = 63), quiere decir que el buffer está lleno y el productor se duerme llamando a *suspend* en P2. Si el buffer no está lleno, el nuevo primo se inserta en el buffer (P3) e *in* se incrementa (P4). Si el nuevo valor de *in* está 1 adelante de *out* (P5) (por ejemplo, si *in* = 17 y *out* = 16), quiere decir que *in* y *out* deben haber sido iguales antes de que *in* se incrementara. El productor concluye que el buffer estaba vacío y que el consumidor estaba, y todavía está, durmiendo. Por tanto, el productor invoca *resume* para despertar al consumidor (P5). Por último, el productor comienza a buscar el siguiente número primo.

Estructuralmente, el programa del consumidor es similar. Primero se efectúa una prueba (C1) para ver si el buffer está vacío. Si así es, el consumidor no tiene nada que hacer, por lo que se duerme. Si el buffer no está vacío, el consumidor saca el siguiente número que se imprimirá (C2) e incrementa *out* (C3). Si *out* está dos posiciones adelante de *in* en este momento (C4), debió haber estado una posición adelante de *in* antes de que éste se incrementara.


```

public class m {
    final public static int TAM_BUFFER = 100;           // buffer va de 0 a 99
    final public static long MAX_PRIMO = 1000000000000L; // parar aquí
    public static int in = 0, out = 0;                 // apuntadores a los datos
    public static long buffer[] = new long[TAM_BUFFER]; // los primos se guardan aquí
    public static productor p;                         // nombre del productor
    public static consumidor c;                       // nombre del consumidor

    public static void main(String args[] ) {           // clase principal
        p = new productor( );                          // crear el productor
        c = new consumidor( );                         // crear el consumidor
        p.start( );                                    // iniciar el productor
        c.start( );                                    // iniciar el consumidor
    }

    // Función utilitaria para incrementar circularmente in y out
    public static int siguiente(int k) {if (k < TAM_BUFFER - 1) return (k + 1); else return(0);}
}

clas productor extends Thread {                       // clase de productor
    public void run( ) {                               // código de productor
        long primo = 2;                               // variable de borrador

        while (primo < m.MAX_PRIMO) {
            primo = sig_primo(primo);                  // enunciado P1
            if (m.siguiente(m.in) == m.out) suspend(); // enunciado P2
            m.buffer[m.in] = primo;                    // enunciado P3
            m.in = m.siguiente(m.in);                  // enunciado P4
            if (m.siguiente(m.out) == m.in) m.c.resume(); // enunciado P5
        }

        private long sig_primo(long primo){...}        // función que calcula el siguiente primo
    }

class consumidor extends Thread {                    // clase consumidor
    public void run( ) {                              // código del consumidor
        long omirp = 2;                               // variable de borrador

        while (omirp < m.MAX_PRIMO){
            if (m.in == m.out) suspend( );             // enunciado C1
            omirp = m.buffer[m.out];                   // enunciado C2
            m.out = m.siguiente(m.out);                 // enunciado C3
            if (m.out == m.siguiente(m.siguiente(m.in))) // enunciado C4
                m.p.resume( );
            System.out.println(omirp);                 // enunciado C5
        }
    }
}

```

Figura 6-25. Procesamiento en paralelo con condición de competencia fatal.

Puesto que $in = out - 1$ es la condición “buffer lleno”, el productor debe haber estado durmiendo, así que el consumidor lo despertará con *resume*. Por último, se imprime el número (C5) y el ciclo se repite.

Lamentablemente, este diseño contiene un defecto fatal, como se muestra en la figura 6-26. Recuerde que los dos procesos se ejecutan de forma asincrónica y a diferentes velocidades, que podrían ser variables. Considere el caso en que sólo queda un número en el buffer, en la entrada 21, e $in = 22$ y $out = 21$, como en la figura 6-26(a). El productor está en el enunciado P1 buscando un número primo y el consumidor está ocupado en C5 imprimiendo el número que está en la posición 20. El consumidor termina de imprimir el número, realiza la prueba en C1, y saca el último número del buffer en C2; luego incrementa *out*. En este momento, tanto *in* como *out* tienen el valor 22. El consumidor imprime el número y pasa a C1, donde trae a *in* y *out* de la memoria para compararlos, como se muestra en la figura 6-26(b).

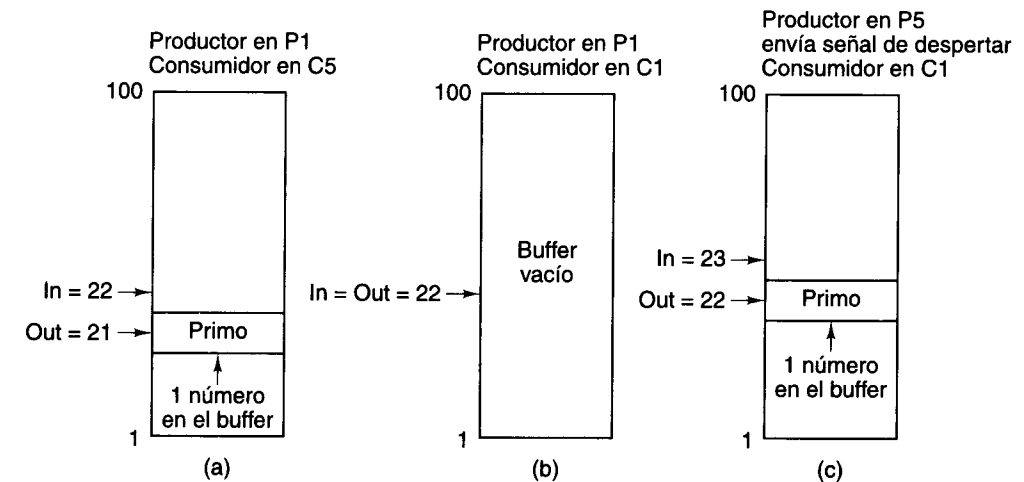


Figura 6-26. Fracaso del mecanismo de comunicación productor-consumidor.

En este momento preciso, después de que el consumidor ha traído a *in* y *out* pero antes de haberlos comparado, el productor encuentra el siguiente primo, lo coloca en el buffer P3 e incrementa *in* en P4. Ahora $in = 23$ y $out = 22$. En P5 el productor descubre que $in = siguiente(out)$. En otras palabras, *in* es 1 más que *out*, lo que indica que hay un elemento en el buffer. Por ello, el productor concluye (incorrectamente) que el consumidor está durmiendo, así que le envía una señal de despertar (es decir, invoca *resume*), como se muestra en la figura 6-26(c). Claro que el consumidor todavía está despierto, por lo que la señal de despertar se pierde. El productor comienza a buscar el siguiente número primo.

En este momento el consumidor continúa. El consumidor ya había traído *in* y *out* de la memoria antes de que el productor colocara el último número en el buffer. Como ahora tanto

in como *out* tienen el valor 22, el consumidor se duerme. Ahora el productor encuentra otro número primo, examina los apuntadores y encuentra que *in* = 24 y *out* = 22, por lo que supone que hay dos números en el buffer (lo cual es cierto) y que el consumidor está despierto (lo cual es falso). El productor sigue repitiendo el ciclo. En algún momento, el buffer se llena y el productor se duerme. Ahora ambos procesos están durmiendo y seguirán haciéndolo indefinidamente.

El problema aquí es que entre el momento en que el consumidor trajo a *in* y *out* y el momento en que se durmió, el productor llegó, descubrió que *in* = *out* + 1, supuso que el consumidor estaba dormido (aunque todavía no lo estaba), y envió una señal de despertar que se perdió porque el consumidor todavía estaba despierto. Este problema se denomina **condición de competencia** porque el éxito del método depende de quién gana la competencia por probar *in* y *out* después de que *out* se incrementa.

El problema de las condiciones de competencia se conoce bien. De hecho, es tan grave que varios años después de la introducción de Java Sun modificó la clase *Thread* y recomendó no usar las llamadas *suspend* y *resume* porque daban pie a condiciones de competencia muy a menudo. La solución que se ofreció fue una basada en el lenguaje, pero como aquí estamos estudiando sistemas operativos analizaremos una solución distinta, que muchos sistemas operativos manejan, entre ellos UNIX y NT.

6.3.3 Sincronización de procesos con semáforos

La condición de competencia se puede resolver de por lo menos dos maneras. Una solución consiste en equipar cada proceso con un “bit de despertar pendiente”. Siempre que se envía una señal de despertar a un proceso que todavía está en ejecución, se enciende su bit de despertar pendiente. Si un proceso se duerme y el bit de despertar pendiente está encendido, de inmediato se le reinicia y se apaga el bit de despertar pendiente. De hecho, lo que hace este bit es guardar la señal de despertar superflua para usarla en el futuro.

Aunque este método resuelve la condición de competencia cuando sólo hay dos procesos, falla en el caso general de *n* procesos en comunicación porque podría ser necesario guardar hasta *n* - 1 señales de despertar. Desde luego, se podría equipar a cada proceso con *n* - 1 bits de despertar pendiente para que pudiera contar hasta *n* - 1 en el sistema unario, pero se trata de una solución un tanto torpe.

Dijkstra (1968b) propuso una solución más general al problema de sincronizar procesos paralelos. En algún lugar de la memoria hay dos variables enteras no negativas llamadas **semáforos**. El sistema operativo proporciona dos llamadas al sistema que operan con semáforos, *up* y *down*. *Up* suma 1 a un semáforo y *down* resta 1 a un semáforo.

Si se realiza una operación *down* con un semáforo cuyo valor es mayor que 0, el semáforo se decrementa en 1 y el proceso que llamó a *down* continúa. En cambio, si el semáforo es 0, *down* no puede llevarse a cabo; el proceso que lo llamó se duerme y permanece dormido hasta que el otro proceso realiza *up* con ese semáforo.

La instrucción *up* verifica si el semáforo es 0. Si así es y el otro proceso está durmiendo por él, el semáforo se incrementa en 1. Entonces, el proceso dormido puede finalizar la operación *down* que lo suspendió, el semáforo se vuelve a poner en 0 y los dos procesos pueden

continuar. Una instrucción *up* con un semáforo distinto de cero simplemente lo incrementa en 1. En esencia, un semáforo es un contador para guardar señales de despertar que se usarán posteriormente, a fin de que no se pierdan. Una propiedad indispensable de las instrucciones de semáforo es que, una vez que un proceso ha iniciado una instrucción con un semáforo, ningún otro proceso podrá acceder al semáforo hasta que el primero haya finalizado la instrucción o bien se haya suspendido tratando de ejecutar *down* con un 0. La figura 6-27 resume las propiedades fundamentales de las llamadas al sistema *up* y *down*.

Instrucción	Semáforo = 0	Semáforo > 0
Up	Semáforo = semáforo + 1 si el otro proceso se detuvo tratando de llevar a cabo una instrucción <i>down</i> con este semáforo, ahora puede finalizar la <i>down</i> y continuar su ejecución	Semáforo = semáforo + 1;
Down	El proceso se detiene hasta que el otro proceso hace <i>up</i> con este semáforo	Semáforo = semáforo - 1

Figura 6-27. Efecto de una operación de semáforo.

Como se dijo antes, Java tiene una solución basada en el lenguaje para manejar condiciones de competencia, y ahora estamos hablando de sistemas operativos. Por tanto, necesitamos una forma de expresar el uso de semáforos en Java, aunque no forme parte del lenguaje ni de las clases estándar. Lo haremos suponiendo que se escribieron dos métodos nativos, *up* y *down*, que efectúan las llamadas al sistema *up* y *down*, respectivamente. Si invocamos estos métodos con enteros ordinarios como parámetros, podremos expresar el uso de semáforos en programas escritos en Java.

La figura 6-28 muestra cómo puede eliminarse la condición de competencia mediante el uso de semáforos. Se añaden dos semáforos a la clase *m*, *disponible*, que inicialmente es 100 (el tamaño del buffer), y *lleno*, que inicialmente es 0. El productor comienza a ejecutarse en P1 en la figura 6-28, y el consumidor comienza a ejecutarse en C1 igual que antes. La llamada *down* con *lleno* detiene el proceso consumidor de inmediato. Cuando el productor encuentra el primer número primo, invoca a *down* con *disponible* como parámetro, lo que decrementa *disponible* a 99. En P5, el productor invoca a *up* con *lleno* como parámetro, lo que incrementa *lleno* a 1. Esta acción libera al consumidor, que ahora puede finalizar su llamada *down* suspendida. En este momento, *lleno* es 0 y ambos procesos están en ejecución.

Volvamos a examinar ahora la condición de competencia. En cierto momento, *in* = 22, *out* = 21, el productor está en P1 y el consumidor está en C5. El consumidor termina lo que estaba haciendo y pasa a C1 donde invoca *down* con *filled*, que tenía el valor 1 antes de la llamada y 0 después. Luego, el consumidor saca el último número del buffer e invoca *up* con *disponible*, que lo incrementa a 100. El consumidor imprime el número y pasa a C1. Justo antes de que el consumidor pueda invocar a *down*, el productor encuentra el siguiente primo y en una secuencia rápida ejecuta los enunciados P2, P3 y P4.

En este punto, *lleno* es 0. El productor está a punto de aplicarle *up* y el consumidor está a punto de invocar a *up*. Si el consumidor ejecuta su instrucción primero, se suspenderá hasta que el productor lo libere (llamando a *up*). Por otra parte, si el productor actúa primero, el semáforo se pondrá en 1 y el consumidor no se suspenderá. En ninguno de los casos hay pérdida de una señal de despertar. Claro que esto era lo que buscábamos al introducir los semáforos.

La propiedad fundamental de las operaciones con semáforos es que son indivisibles. Una vez que se ha iniciado una operación con un semáforo, ningún otro proceso puede usar el semáforo hasta que el primer proceso haya finalizado la operación o bien haya quedado suspendido en el intento. Además, con los semáforos no se pierden señales de despertar. En contraste, los enunciados *if* de la figura 6-25 son indivisibles. Entre la evaluación de la condición y la ejecución del enunciado seleccionado otro proceso podría enviar una señal de despertar.

El problema de la sincronización de procesos se ha eliminado efectivamente declarando que las llamadas al sistema *up* y *down* que efectúan *up* y *down* son indivisibles. Para que estas operaciones sean indivisibles, el sistema operativo debe prohibir que dos o más procesos usen el mismo semáforo al mismo tiempo. Como mínimo, una vez efectuada una llamada al sistema *up* o *down*, no se ejecutará ningún otro código de usuario hasta que la llamada se haya llevado a cabo. Es común implementar los semáforos inhabilitando las interrupciones durante las operaciones con semáforos.

La sincronización con semáforos es una técnica que funciona con un número arbitrario de procesos. Varios procesos pueden estar dormidos tratando de finalizar una llamada al sistema *down* con el mismo semáforo. Cuando algún otro proceso por fin ejecuta *up* con ese semáforo, se permite que uno de los procesos en espera finalice su *down* y continúe su ejecución. El valor del semáforo seguirá siendo 0 y los demás procesos seguirán esperando.

Quizá podamos aclarar la naturaleza de los semáforos con una analogía. Imagine un día de campo con 20 equipos de volibol divididos en 10 juegos (procesos) cada uno de los cuales juega en su propia cancha, y una canasta grande (el semáforo) para los balones de volibol. Lo malo es que sólo hay siete balones. En un instante dado, hay entre cero y siete balones en la canasta (el semáforo tiene un valor entre 0 y 7). Colocar un balón en la canasta es un *up* porque incrementa el valor del semáforo. Sacar un balón de la canasta es un *down* porque decrementa el valor del semáforo.

Al principio del día de campo, cada cancha envía un jugador a la canasta para conseguir un balón. Siete de ellos logran conseguir un balón (llevan a cabo el *down*); tres se ven obligados a esperar un balón (es decir, no logran finalizar el *down*). Sus juegos se suspenden temporalmente. Tarde o temprano, uno de los otros juegos termina y coloca su balón en la canasta (ejecuta *up*). Esta operación permite a uno de los tres jugadores que esperan junto a la canasta obtener un balón (finalizar un *down* inconcluso), lo que permite a un juego continuar. Los otros dos juegos siguen suspendidos hasta que se coloquen otros dos balones en la canasta. Cuando se devuelven dos balones (se ejecutan otros dos *up*), los dos últimos juegos pueden continuar.

```

public class m {
    final public static int TAM_BUFFER = 100;           // buffer va de 0 a 99
    final public static long MAX_PRIMO = 1000000000000L; // parar aquí
    public static int in = 0, out = 0;                 // apuntadores a los datos
    public static long buffer[] = new long[TAM_BUFFER]; // los primos se guardan aquí
    public static productor p;                         // nombre del productor
    public static consumidor c;                       // nombre del consumidor
    public static int lleno = 0, disponible = 100;     // semáforos

    public static void main(String args[] ) {          // clase principal
        p = new productor( );                         // crear el productor
        c = new consumidor( );                       // crear el consumidor
        p.start( );                                   // iniciar el productor
        c.start( );                                   // iniciar el consumidor
    }
    // Función utilitaria para incrementar circularmente in y out
    public static int siguiente(int k) {if (k = TAM_BUFFER - 1) return (k + 1); else return(0);}
}

class productor extends Thread {                     // clase productor
    native void up(int s); native void down(int s);   // métodos para semáforos
    public void run( ) {                             // código de productor
        long primo = 2;                             // variable de borrador

        while (primo < m.MAX_PRIMO) {
            primo = sig_primo(primo);                // enunciado P1
            down(m.disponible);                       // enunciado P2
            m.buffer[m.in] = primo;                   // enunciado P3
            m.in = m.siguiente(m.in);                 // enunciado P4
            up(m.lleno);                              // enunciado P5
        }
    }

    private long sig_primo(long primo){...}           // función que calcula
                                                    // el siguiente primo
}

class consumidor extends Thread {                    // clase consumidor
    native void up(int s); native void down(int s);   // métodos para semáforos
    public void run( ) {                             // código del consumidor
        long omirp = 1;                             // variable de borrador

        while (omirp < m.MAX_PRIMO){
            down(m.lleno);                            // enunciado C1
            omirp = m.buffer[m.out];                  // enunciado C2
            m.out = m.siguiente(m.out);               // enunciado C3
            up(m.disponible);                         // enunciado C4
            System.out.println(omirp);                // enunciado C5
        }
    }
}

```

Figura 6-28. Procesamiento en paralelo con semáforos.

6.4 EJEMPLOS DE SISTEMAS OPERATIVOS

En esta sección seguiremos examinando nuestros sistemas de ejemplo, el Pentium II y el UltraSPARC II. En cada caso veremos un sistema operativo empleado en ese procesador. Para el Pentium II usaremos Windows NT (que abreviaremos como NT en lo que sigue); para el UltraSPARC II usaremos UNIX. Puesto que UNIX es más sencillo y en muchos aspectos más elegante, comenzaremos con él. Además, UNIX se diseñó e implementó primero y tuvo una importante influencia sobre NT, por lo que este orden es más lógico que el opuesto.

6.4.1 Introducción

En esta sección presentaremos una breve introducción a nuestros dos sistemas operativos de ejemplo, UNIX y NT, concentrándonos en su historia, estructura y llamadas al sistema.

UNIX

UNIX se creó en Bell Labs a principios de los años setenta. La primera versión fue escrita por Ken Thompson en ensamblador para la minicomputadora PDP-7. Pronto apareció una versión para la PDP-11, escrita en un nuevo lenguaje llamado C ideado e implementado por Dennis Ritchie. En 1974, Ritchie y Thompson publicaron un artículo fundamental sobre UNIX (Ritchie y Thompson, 1974). Por el trabajo descrito en ese artículo ellos recibieron posteriormente el prestigioso Premio Turing de la ACM (Ritchie, 1984; Thompson, 1984). La publicación de este artículo hizo que muchas universidades solicitaran a Bell Labs una copia de UNIX. Puesto que la compañía fundadora de Bell Labs, AT&T, era un monopolio regulado en ese entonces y no tenía permiso para entrar en el negocio de las computadoras, no tuvo reparo en otorgar licencias de UNIX a las universidades por una cuota modesta.

En una de esas coincidencias que a menudo moldean la historia, la PDP-11 era la computadora preferida por casi todos los departamentos de ciencias de la computación universitarios, y una opinión muy difundida tanto entre los profesores como los estudiantes era que los sistemas operativos que venían con la PDP-11 eran horribles. UNIX pronto llenó el vacío, en buena medida porque venía acompañado por el código fuente completo, lo que permitía a los profesores y estudiantes hacerle todas las adiciones y modificaciones que se les ocurrían.

Una de las muchas universidades que adquirieron UNIX desde el principio fue la University of California en Berkeley. Puesto que contaba con el código fuente completo, Berkeley pudo modificar el sistema considerablemente. Los cambios más notables fueron un puerto para la minicomputadora VAX y la adición de memoria virtual paginada, la extensión de los nombres de archivo de 14 a 255 caracteres, y la inclusión del protocolo de redes TCP/IP, que ahora se usa en Internet (en gran medida por el hecho de que estaba en Berkeley UNIX).

Mientras Berkeley estaba efectuando todos estos cambios, AT&T siguió desarrollando UNIX, y sacó el System III en 1982 y el System V en 1984. Para fines de los años ochenta se usaban ampliamente dos versiones diferentes e incompatibles de UNIX: Berkeley UNIX y System V. Esta situación en el mundo UNIX, junto con el hecho de que no había estándares para los

formatos de programas binarios, inhibieron considerablemente el éxito comercial de UNIX porque era imposible para los proveedores de software escribir y empaquetar programas UNIX con la expectativa de que funcionarían en cualquier sistema UNIX (como se hacía todo el tiempo con MS-DOS). Después de muchas peleas y discusiones, la IEEE Standards Board creó un estándar llamado **POSIX (Portable Operating System IX)**. POSIX también se conoce por su número de norma IEEE, P1003, y posteriormente se convirtió en un estándar internacional.

El estándar POSIX se divide en muchas partes, cada una de las cuales cubre un área diferente de UNIX. La primera parte, P1003.1, define las llamadas al sistema; la segunda, P1003.2, define los programas utilitarios básicos, etc. El estándar P1003.1 define unas 60 llamadas al sistema que todos los sistemas que cumplan con la norma deben apoyar. Éstas son las llamadas básicas para leer y escribir archivos, crear procesos nuevos, y demás. Casi todos los sistemas UNIX actuales apoyan las llamadas al sistema P1003.1, pero muchos reconocen también llamadas adicionales, sobre todo las definidas por System V y/o Berkeley UNIX. En general, esto representa unas 100 llamadas al sistema además del conjunto POSIX. El sistema operativo para el UltraSPARC II se basa en System V y se llama **Solaris**. Éste también reconoce muchas de las llamadas al sistema de Berkeley.

En la figura 6-29 se presenta una clasificación aproximada de las llamadas al sistema de Solaris. Las llamadas para gestión de archivos y directorios son las categorías más grandes e importantes. Casi todas provienen de P1003.1, y una fracción relativamente grande de las otras se deriva de System V.

Categoría	Ejemplos
Gestión de archivos	Abrir, leer, escribir, cerrar y poner candados a archivos
Gestión de directorios	Crear y eliminar directorios; trasladar archivos
Gestión de procesos	Engendrar, terminar, rastrear y señalar procesos
Gestión de memoria	Compartir memoria entre procesos; proteger páginas
Obtener/fijar parámetros	Obtener ID de usuario, grupo o proceso; fijar prioridad
Fechas y horas	Fijar tiempos de acceso a archivos; usar temporizador de intervalo, ejecutar perfiles
Trabajo con redes	Establecer/aceptar conexión; enviar/recibir mensaje
Diversas	Habilitar contabilización; manipular cuotas de disco; reiniciar el sistema

Figura 6-29. Clasificación a grandes rasgos de las llamadas al sistema de UNIX.

Un área que se debe en su mayor parte a Berkeley UNIX y no a System V es la de trabajo con redes. Berkeley inventó el concepto de **socket**, que es el punto terminal de una conexión de red. Los contactos de pared con cuatro terminales a los que pueden conectarse los teléfonos sirvieron como modelo para este concepto. Un proceso UNIX puede crear un **socket**, vincularse a él y establecer una conexión con un **socket** de una máquina distante. Luego el proceso puede intercambiar datos en ambas direcciones por esta conexión, casi siempre empleando el protocolo TCP/IP. Puesto que la tecnología de trabajo con redes ha estado en UNIX desde hace

décadas y es muy estable y madura, una fracción considerable de los servidores de Internet ejecutan UNIX.

Dado que hay muchas implementaciones de UNIX, es difícil decir mucho acerca de la estructura del sistema operativo, ya que cada una presenta varias diferencias respecto a todas las otras. No obstante, en general, la figura 6-30 se aplica a casi todas ellas. En la parte baja hay una capa de *drivers* de dispositivos que aíslan al sistema de archivos del hardware desnudo. Originalmente, cada *driver* de dispositivo se escribió como entidad independiente, distinta de todas las demás. Este sistema dio pie a duplicación de trabajo, ya que muchos *drivers* deben ocuparse del control de flujo, manejo de errores, prioridades, separación entre datos y control, etc. Esta observación llevó a Dennis Ritchie a crear un esquema llamado **streams** para escribir *drivers* de forma modular. Con un *stream*, es posible establecer una conexión bidireccional entre un proceso usuario y un dispositivo de hardware e insertar uno o más módulos en el camino. El proceso de usuario mete datos en el *stream*, que luego es procesado o transformado por cada módulo hasta llegar al hardware. Los datos entrantes experimentan el procesamiento inverso.

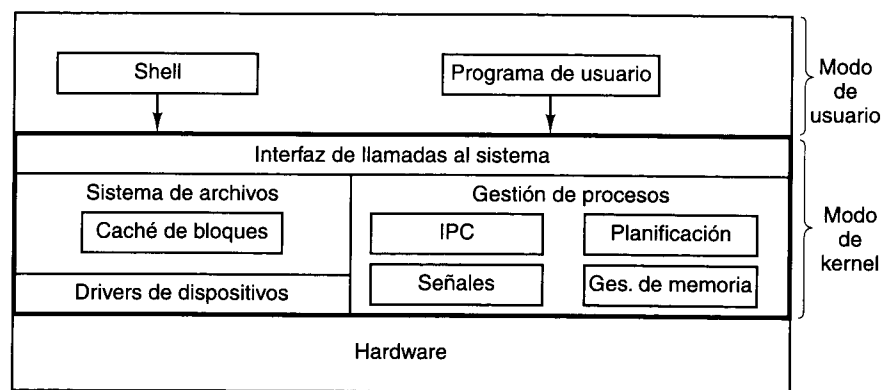


Figura 6-30. Estructura de un sistema UNIX representativo.

Encima de los *drivers* de dispositivos está el sistema de archivos, que administra nombres de archivo, directorios, asignación de bloques en disco, protección y mucho más. Parte del sistema de archivos es una **caché de bloques** en el que se retienen los bloques que se leyeron del disco más recientemente, por si se vuelven a necesitar pronto. Se han utilizado diversos sistemas de archivos con el paso de los años, incluido el sistema de archivos rápido Berkeley (McKusick *et al.*, 1984) y sistemas de archivos con estructura de bitácora (Rosenblum y Ousterhout, 1991; Seltzer *et al.*, 1993).

La otra parte del kernel de UNIX es la porción de gestión de procesos. Entre sus diversas funciones, esta parte maneja la comunicación entre procesos (IPC, *InterProcess Communication*), que permite a los procesos comunicarse entre sí y sincronizarse para evitar condiciones de competencia. Se proporcionan diversos mecanismos. El código de gestión de procesos

también se encarga de la planificación de procesos, que se basa en prioridades. Las señales, que son una forma de interrupción (asíncronica) por software, también se gestionan aquí. Por último, la gestión de memoria también se efectúa en esta porción. Casi todos los sistemas UNIX apoyan una memoria virtual paginada por demanda, a veces con unas cuantas funciones extra, como la posibilidad de que varios procesos compartan regiones de espacio de direcciones comunes.

Desde su nacimiento, UNIX ha tratado de ser un sistema pequeño, a fin de mejorar la confiabilidad y el desempeño. Las primeras versiones de UNIX se basaban exclusivamente en texto, empleando terminales capaces de exhibir 24 o 25 líneas de 80 caracteres ASCII. La interfaz con el usuario era manejada por un programa de nivel de usuario llamado **shell**, que ofrecía una interfaz de línea de comandos. Puesto que el *shell* no formaba parte del kernel, era fácil añadir nuevos *shells*, y con el tiempo se inventaron varios, cada uno más sofisticado que el anterior.

Más adelante, cuando aparecieron las terminales gráficas, se creó en MIT, un sistema de ventanas para UNIX llamado **X Windows**. Tiempo después, se colocó encima de Windows X una **GUI (interfaz gráfica con el usuario, Graphical User Interface)** más robusta, llamada **Motif**. En congruencia con la filosofía UNIX de tener un kernel pequeño, casi todo el código de X Windows y Motif se ejecuta en modo de usuario, fuera del kernel.

Windows NT

Cuando la IBM PC original salió al mercado en 1981, estaba equipada con un sistema operativo de 16 bits, en modo real, monousuario, orientado a línea de comandos, llamado MS-DOS 1.0. Este sistema operativo consistía en 8 KB de código residente en la memoria. Dos años después apareció un sistema mucho más potente de 24 KB, MS-DOS 2.0, que contenía un procesador de línea de comandos (*shell*) con varias funciones copiadas de UNIX. Cuando IBM sacó la PC/AT basada en el 286 en 1984, venía equipada con MS-DOS 3.0, que para entonces ocupaba 36 KB. Con los años, MS-DOS siguió adquiriendo funciones nuevas, pero seguía siendo un sistema orientado a línea de comandos.

Inspirado por el éxito de la Apple Macintosh, Microsoft decidió añadir a MS-DOS una interfaz gráfica con el usuario a la que llamó **Windows**. Las primeras tres versiones de Windows, que culminaron con Windows 3.x, no eran verdaderos sistemas operativos, sino interfaces gráficas con el usuario montadas encima de MS-DOS, que seguía teniendo el control de la máquina. Todos los programas se ejecutaban en el mismo espacio de direcciones, y un error en cualquiera de ellos podía parar en seco a toda la máquina.

Con el lanzamiento de Windows 95 en 1995 no llegó a eliminarse a MS-DOS, aunque introdujo una nueva versión, 7.0. Juntos, Windows 95 y MS-DOS 7.0 contenían casi todas las características de un sistema operativo más robusto, incluidas memoria virtual, gestión de procesos y multiprogramación. Sin embargo, Windows 95 no era un programa cabal de 32 bits; contenía grandes fragmentos de código viejo de 16 bits (además de un poco de código de 32 bits) y seguía usando el sistema de archivos de MS-DOS, con casi todas sus limitaciones. El único cambio importante al sistema de archivos fue la adición de nombres de archivo largos en lugar de los nombres de 8 + 3 caracteres permitidos en MS-DOS.

Incluso con la llegada de Windows 98 en 1998, MS-DOS seguía ahí (ahora en su versión 7.1) y seguía ejecutando código de 16 bits. Aunque un poco más de funcionalidad migró de la parte MS-DOS a la parte Windows, y se adoptó como norma una organización de disco apropiada para discos mayores, en su corazón Windows 98 no era muy diferente de Windows 95. La principal diferencia fue la interfaz con el usuario, que integró el escritorio, Internet y la televisión de forma más estrecha. Fue precisamente esta integración la que atrajo la atención del Departamento de Justicia de Estados Unidos, que entonces demandó a Microsoft alegando que era un monopolio ilegal.

Mientras ocurrían todas estas cosas, Microsoft también estaba enfrascado en la creación de un sistema operativo de 32 bits totalmente nuevo que se estaba escribiendo desde cero. Este nuevo sistema se llamó **Windows New Technology**, o **Windows NT**. En un principio, NT se anunció como el reemplazo de todos los demás sistemas operativos para PC basadas en Intel, pero su adopción fue un tanto lenta y posteriormente se reorientó hacia el extremo superior del mercado, donde encontró un nicho. Ahora también se está popularizando en el extremo inferior.

NT se vende en dos versiones: servidor y estación de trabajo. Estas dos versiones son casi idénticas y se generan a partir del mismo código fuente. La versión para servidor se diseñó para máquinas que operan como servidores de archivos e impresoras basadas en LAN y tiene funciones de administración más completas que la versión para estación de trabajo, que está dirigida a la computación de escritorio monousuario. La versión para servidor tiene una variante (empresa) pensada para sitios grandes. Las diversas versiones se afinan de forma distinta, y cada una se optimiza para su entorno esperado. Fuera de estas diferencias menores, todas las versiones son prácticamente iguales. De hecho, casi todos los archivos ejecutables son idénticos para todas las versiones. NT por sí mismo descubre qué versión está examinando una variable de una estructura de datos interna (el Registro). La licencia prohíbe a los usuarios modificar esta variable y así convertir la versión para estación de trabajo (de bajo costo) en las versiones para servidor o empresa (mucho más costosas). Aquí no haremos más distinciones entre estas versiones.

MS-DOS y todas las versiones previas de Windows eran sistemas monousuario. NT, en cambio, apoya la multiprogramación, así que varios usuarios pueden trabajar en la misma máquina al mismo tiempo. Por ejemplo, un servidor de red podría tener varios usuarios conectados simultáneamente por una red, cada uno de los cuales accede a sus propios archivos de forma protegida.

NT es un verdadero sistema operativo de 32 bits con multiprogramación. NT Apoya múltiples procesos de usuario, cada uno de los cuales tiene un espacio de direcciones virtual completo de 32 bits paginado por demanda. Además, el sistema mismo se escribió como código de 32 bits exclusivamente.

Una de las mejoras originales de NT respecto a Windows 95 fue su estructura modular que consistía en un núcleo más o menos pequeño que operaba en modo de kernel más varios procesos servidores que operaban en modo de usuario. Los procesos de usuario interactuaban con los procesos servidores empleando el modelo cliente-servidor: un cliente envía un mensaje de solicitud a un servidor; el servidor efectúa el trabajo y devuelve el resultado al cliente en un segundo mensaje. Tal estructura modular facilitó transportar el sistema a varias computadoras además de la línea Intel, incluidas la Alpha de DEC, PowerPC de IBM y MIPS

de SGI. Sin embargo, por razones de desempeño, a partir de NT 4.0 prácticamente todo el sistema se volvió a poner en el kernel.

Podríamos hablar por mucho tiempo acerca de la estructura interna de NT y el aspecto de su interfaz de llamadas al sistema. Puesto que lo que nos interesa primordialmente aquí es la máquina virtual que presentan los diversos sistemas operativos (es decir, las llamadas al sistema), daremos un breve resumen de la estructura del sistema y luego nos dedicaremos a la interfaz de llamadas al sistema.

La estructura de NT se ilustra en la figura 6-31. Consiste en varios módulos estructurados en capas que colaboran para implementar el sistema operativo. Cada módulo tiene una función en particular y una interfaz bien definida con los otros módulos. Casi todos los módulos están escritos en C, aunque una parte de la interfaz con dispositivos gráficos está escrita en C++ y una porción muy pequeña de las capas más bajas está escrita en lenguaje ensamblador.

En la parte inferior está una capa delgada llamada **capa de abstracción del hardware**. La tarea de esta capa es presentar al resto del sistema operativo dispositivos de hardware abstractos, carentes de las peculiaridades e idiosincrasias tan abundantes en el hardware real. Entre los dispositivos que se modelan están las cachés externas al chip, temporizadores, buses de E/S, controladores de interrupciones y controladores de DMA. Al exponer éstos al resto del sistema en una forma idealizada, se facilita el traslado de NT a otras plataformas de hardware, ya que casi todas las modificaciones requeridas están concentradas en un solo lugar.

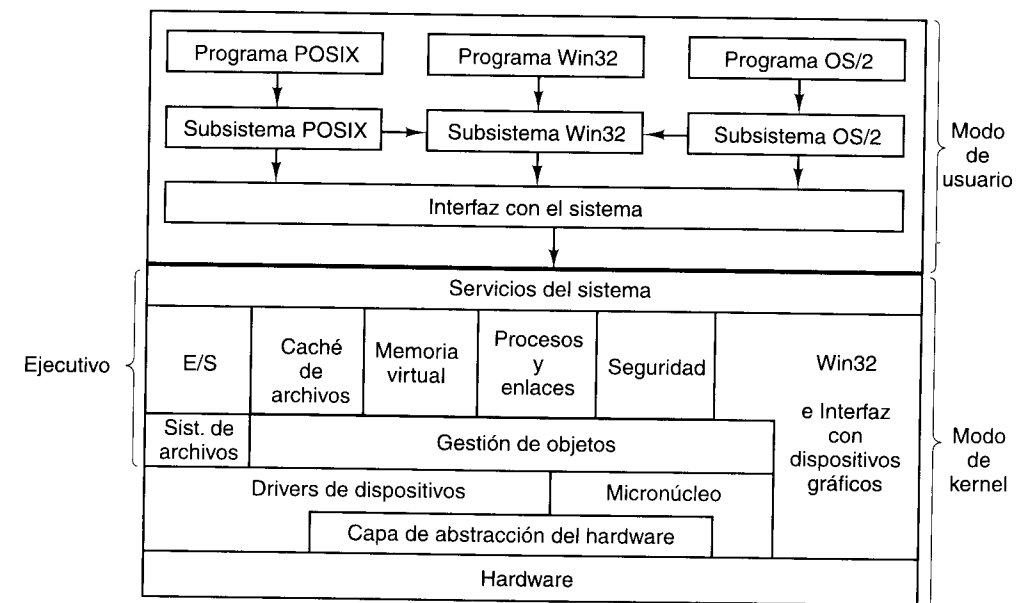


Figura 6-31. Estructura de Windows NT.

Arriba de la capa de abstracción del hardware está una capa que contiene el micronúcleo y los *drivers* de dispositivos. El micronúcleo y todos los *drivers* tienen acceso directo al hardware cuando lo necesitan, pues contienen código dependiente del hardware.

El **micronúcleo** apoya los objetos primitivos del núcleo, el manejo de interrupciones, trampas y excepciones, la planificación y sincronización de procesos, la sincronización de multiprocesadores y la gestión del tiempo. El propósito de esta capa es lograr que el resto del sistema operativo sea totalmente independiente del hardware, y por tanto muy portátil. El micronúcleo reside todo el tiempo en la memoria principal y no puede ser desalojado, aunque puede ceder temporalmente el control para atender las interrupciones de E/S.

Cada **driver de dispositivo** puede controlar uno o más dispositivos de E/S, pero un *driver* también puede hacer cosas no relacionadas con un dispositivo específico, como cifrar un flujo de datos o simplemente proporcionar acceso a estructuras de datos del núcleo. Puesto que los usuarios pueden instalar *drivers* de dispositivos nuevos, pueden afectar al núcleo y corromper el sistema. Por esta razón, los *drivers* se deben escribir con mucho cuidado.

Arriba del micronúcleo y los *drivers* de dispositivos está la porción superior del sistema operativo, llamada **ejecutivo**. El ejecutivo es independiente de la arquitectura y se puede trasladar a otras máquinas con relativamente poco esfuerzo. Esta parte abarca tres capas.

La capa más baja contiene los sistemas operativos y el gestor de objetos. Los **sistemas de archivos** apoyan el uso de archivos y directorios. El **gestor de objetos** maneja objetos que el núcleo conoce. Éstos incluyen procesos, enlaces (procesos ligeros dentro de un espacio de direcciones), archivos, directorios, semáforos, dispositivos de E/S, temporizadores y muchos más. El gestor de objetos también administra un espacio de nombres en el que se pueden colocar los objetos recién creados para poder hacer referencia a ellos después.

La siguiente capa consta de seis partes principales, como se muestra en la figura 6-31. El **gestor de E/S** proporciona un ámbito para administrar dispositivos de E/S y ofrece servicios de E/S genéricos. Este componente utiliza los servicios del sistema de archivos, que a su vez usa los *drivers* de dispositivos, además de los servicios del gestor de objetos.

El **gestor de cachés** mantiene los bloques de disco más recientemente usados en la memoria para agilizar el acceso a ellos en (el probable) caso de que se necesiten otra vez. Su tarea es determinar cuáles bloques es probable que se vayan a volver a necesitar y cuáles no. Es posible configurar NT con varios sistemas de archivos, en cuyo caso el gestor de caché dará servicio a todos ellos, con lo que se evita que cada uno tenga que realizar su propia administración de caché. Cuando se necesita un bloque, se le pide al gestor de caché que lo proporcione. Si el caché no tiene el bloque, el gestor se dirige al sistema de archivos apropiado para conseguirlo. Puesto que se puede establecer una correspondencia entre archivos y los espacios de direcciones de los procesos, el caché debe interactuar con el administrador de memoria virtual para establecer la consistencia requerida.

El **administrador de memoria virtual** implementa la arquitectura de memoria virtual paginada por demanda de NT. Este componente controla la correspondencia entre las páginas virtuales y los marcos de página físicos, y al hacerlo obliga al cumplimiento de las reglas de protección que restringen el acceso de cada proceso exclusivamente a las páginas que pertenecen a su espacio de direcciones y no a los de otros procesos (excepto en circunstancias especiales). El administrador de memoria también maneja ciertas llamadas al sistema relacionadas con la memoria virtual.

El **gestor de procesos y enlaces** maneja los procesos y los enlaces, lo que incluye su creación y destrucción; se ocupa de los mecanismos que permiten controlarlos, más que de las políticas de uso de procesos y enlaces.

El **gestor de seguridad** hace cumplir el complejo mecanismo de seguridad de NT, que cumple con los requisitos C2 del Libro Naranja del Departamento de la Defensa de Estados Unidos. El Libro Naranja especifica un gran número de reglas a las que debe ajustarse un sistema certificado, desde la verificación de autenticidad de los ingresos al sistema (*logins*) hasta el manejo del control de acceso y el hecho de que las páginas virtuales deben llenarse de ceros antes de reutilizarse.

La **interfaz con dispositivos gráficos** se encarga de la gestión de imágenes para el monitor y las impresoras, y cuenta con llamadas al sistema que permiten a los programas de usuario escribir en el monitor o las impresoras de forma independiente del dispositivo. Esta interfaz también contiene el administrador de ventanas y *drivers* de dispositivos de hardware. En versiones de NT previas a la 4.0, este componente estaba en el espacio de usuario pero su desempeño era muy bajo, por lo que Microsoft lo pasó al núcleo para hacerlo más rápido. El modelo Win32 también maneja la mayor parte de las llamadas del sistema. También estaba originalmente en el espacio de usuario pero fue movido al núcleo para mejorar su rendimiento.

Encima del ejecutivo hay una capa delgada llamada **servicios del sistema**. Su función es proporcionar una interfaz con el ejecutivo, la cual acepta las verdaderas llamadas al sistema de NT e invoca otras partes del ejecutivo para que las lleve a cabo.

Afuera del núcleo están los programas de usuario y los subsistemas del entorno. Se incluyen **subsistemas del entorno** porque no es recomendable que los programas de usuario efectúen llamadas al sistema directamente (aunque técnicamente sí pueden hacerlo). En vez de ello, cada subsistema de entorno exporta un conjunto (distinto) de llamadas a funciones que los programas de usuario pueden utilizar. En la figura 6-31 se muestran tres subsistemas de entorno: Win32 (para programas NT o Windows 95/98), POSIX (para programas UNIX que han sido trasladados) y os/2 (para programas os/2 que han sido trasladados).

Las aplicaciones Windows usan las funciones Win32 y se comunican con el subsistema Win32 para efectuar llamadas al sistema. El **subsistema Win32** acepta las llamadas a las funciones Win32 y usa el módulo de biblioteca de **interfaz con el sistema** (que en realidad es un archivo DLL; véase el capítulo 7) para efectuar las verdaderas llamadas al sistema NT requeridas para ejecutarlas.

También hay un **subsistema posix** que proporciona el apoyo mínimo para aplicaciones UNIX. Este subsistema reconoce sólo la funcionalidad P1003.1 y casi nada más. Se trata de un subsistema cerrado, lo que implica que sus aplicaciones no pueden usar los recursos del subsistema Win32, y esto restringe considerablemente su campo de acción. En la práctica, trasladar cualquier programa UNIX real a NT utilizando este subsistema es casi imposible. Se incluyó sólo porque algunas dependencias del gobierno estadounidense exigen que los sistemas operativos para computadoras del gobierno cumplan con P1003.1. Este subsistema no es autosuficiente y utiliza el subsistema Win32 para efectuar algunas de sus tareas, pero sin exportar la interfaz Win32 completa a sus programas de usuario.

El subsistema os/2 tiene limitaciones similares de su funcionalidad y es probable que se omita en alguna versión futura. También usa el subsistema Win32. Además, hay un subsistema de entorno MS-DOS que no se muestra en la figura.

Habiendo examinado brevemente la estructura de NT, pasaremos a lo que principalmente nos interesa, que son los servicios que ofrece. Esta interfaz es la principal conexión

del programador con el sistema. Lamentablemente, Microsoft nunca ha publicado la lista completa de llamadas al sistema de NT, y además las modifica de una versión a otra. En tales condiciones, escribir programas que efectúen llamadas al sistema directamente es casi imposible.

Lo que Microsoft hizo fue definir un conjunto de llamadas que recibe el nombre de **interfaz para programación de aplicaciones (API, Application Programming Interface) Win32**, y que se conoce públicamente. Estas llamadas son procedimientos de biblioteca que efectúan llamadas al sistema para realizar el trabajo o bien, en algunos casos, realizan el trabajo en el mismo procedimiento de biblioteca del espacio de usuario o en el subsistema Win32. Las llamadas de la API Win32 no cambian al cambiar las versiones, a fin de promover la estabilidad. Sin embargo, también hay llamadas de la API de NT que sí pueden cambiar de una versión a otra. Aunque no todas las llamadas de la API Win32 son llamadas al sistema de NT, es mejor concentrarnos en ellas aquí más que en las verdaderas llamadas al sistema NT porque las llamadas de la API Win32 están bien documentadas y son más estables.

La filosofía de la API Win32 es totalmente distinta de la filosofía de UNIX. En esta última, todas las llamadas al sistema se conocen públicamente y forman una interfaz mínima: eliminar siquiera una de ellas reduciría la funcionalidad del sistema operativo. La filosofía Win32 es ofrecer una interfaz muy completa, a veces con tres o cuatro formas de hacer la misma cosa, e incluir muchas funciones que a todas luces no deberían ser llamadas al sistema (y que no lo son), como una llamada API para copiar todo un archivo.

Muchas llamadas de la API Win32 crean objetos del núcleo de un tipo u otro, incluidos archivos, procesos, enlaces, filas de procesamiento, etc. Cada llamada que crea un objeto del núcleo devuelve al invocador un resultado llamado **asa o manija (handle)**. Esta manija se usa después para realizar operaciones con el objeto. Las manijas son específicas para el proceso que creó el objeto al que se hace referencia con la manija; no se pueden pasar directamente a otro proceso y usarse en él (así como los descriptores UNIX no pueden pasarse a otros procesos y usarse ahí). Sin embargo, en ciertas circunstancias es posible duplicar una manija y pasarla a otros procesos de forma protegida, permitiéndole acceder de forma controlada a objetos que pertenecen a otros procesos. Además, cada objeto tiene asociado un descriptor de seguridad que indica con detalle quién puede y quién no puede realizar qué tipos de operaciones con el objeto.

A veces se dice que NT está orientado a objetos porque la única forma de manipular objetos del núcleo es invocar métodos (funciones API) con sus manijas. Por otra parte, NT carece de algunas de las propiedades más básicas de los sistemas orientados a objetos, como herencia y polimorfismo.

La API Win32 también está disponible en Windows 95/98 (así como en el sistema operativo para aparatos electrónicos de consumo, Windows CE), con unas cuantas excepciones. Por ejemplo, Windows 95/98 no tiene seguridad, por lo que las llamadas API relacionadas con la seguridad simplemente devuelven códigos de error a Windows 95/98. Además, los nombres de archivo de NT usan el conjunto de caracteres Unicode, que no está disponible en Windows 95/98. También hay diferencias en los parámetros a algunas llamadas de funciones API. En NT, por ejemplo, todas las coordenadas de pantalla dadas en las funciones de gráficos son verdaderos números de 32 bits; en Windows 95/98 sólo se usan los 16 bits de orden bajo (por compatibilidad hacia atrás con Windows 3.1). La existencia de la API Win32 en varios

sistemas operativos distintos facilita el traslado de programas entre ellos pero también hace ver más claramente que está un tanto desacoplada de las llamadas al sistema reales. En la figura 6-32 se resumen algunas de las diferencias entre Windows 95/98 y NT.

Característica	Windows 95/98	NT 5.0
¿API Win32?	Sí	Sí
¿Sistema cabal de 32 bits?	No	Sí
¿Seguridad?	No	Sí
¿Correspondencias de archivos protegidas?	No	Sí
¿Espacio de direcciones privado para cada programa MS-DOS?	No	Sí
¿Opera de inmediato, sin tener que configurarse?	Sí	Sí
¿Reconoce UNICODE?	No	Sí
Se ejecuta en	Intel 80x86	80x86, Alpha
¿Apoyo a multiprocesadores?	No	Sí
¿Código reentrante dentro del sistema operativo?	No	Sí
¿El usuario puede escribir algunos datos críticos para el sistema operativo?	Sí	No

Figura 6-32. Algunas diferencias entre versiones de Windows.

6.4.2 Ejemplos de memoria virtual

En esta sección examinaremos la memoria virtual tanto en UNIX como en NT. En general, desde el punto de vista del programador hay muchas similitudes.

Memoria virtual en UNIX

El modelo de memoria de UNIX es sencillo. Cada proceso tiene tres segmentos: código, datos y pila, como se ilustra en la figura 6-33. En una máquina con un solo espacio de direcciones lineal, el código generalmente se coloca cerca de la base de la memoria, seguida de los datos. La pila se coloca en la parte alta de la memoria. El tamaño del código es fijo, pero los datos y la pila pueden crecer: la pila hacia abajo y los datos hacia arriba. Este modelo es fácil de implementar en casi cualquier máquina y es el modelo empleado por Solaris.

Además, si la máquina tiene paginación, se puede paginar todo el espacio de direcciones sin que los programas de usuario siquiera tengan conocimiento de ello; lo único que observan es que está permitido tener programas más grandes que la memoria física de la máquina. Los sistemas UNIX que no tienen paginación en general intercambian procesos enteros entre la memoria y el disco para poder tener un número arbitrariamente grande de procesos en tiempo compartido.

En el caso del Berkeley UNIX, la descripción anterior (memoria virtual paginada por demanda) cubre prácticamente todos los aspectos. En cambio, System V (y Solaris) incluye varias funciones que permiten a los usuarios administrar su memoria virtual de maneras más

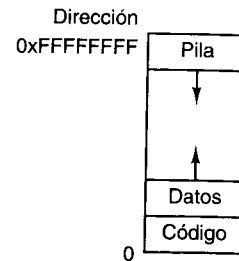


Figura 6-33. Espacio de direcciones de un solo proceso UNIX.

sofisticadas. La más importante de éstas es la capacidad de un proceso para hacer corresponder (una porción de) un archivo con una parte de su espacio de direcciones. Por ejemplo, si un archivo de 12 KB se hace corresponder con la dirección virtual 144K, una lectura de la palabra que está en la dirección 144K leerá la primera palabra del archivo. Esto permite efectuar E/S con archivos sin efectuar llamadas al sistema. Puesto que algunos archivos podrían exceder el tamaño del espacio de direcciones virtual, también es posible mapear sólo una porción de un archivo en lugar de todo. La correspondencia se establece abriendo primero el archivo y obteniendo un descriptor de archivo, *da*, que sirve para identificar el archivo por mapear. Luego el proceso efectúa una llamada

`dir_fisica = mmap(dir_virtual, longitud, prot, banderas, da, dist_arch)`

que hace corresponder *longitud* bytes que inician a *dist_arch* dentro del archivo, con el espacio de direcciones virtual a partir de *dir_virtual*. Como alternativa, se puede ajustar el parámetro *banderas* para pedir al sistema que escoja una dirección virtual, que entonces devolverá como *dir_fisica*. La región mapeada debe ser un número entero de páginas y estar alineada con una frontera de página. El parámetro *prot* puede especificar cualquier combinación de permisos de lectura, escritura o ejecución. La correspondencia puede eliminarse después con `unmap`.

Varios procesos pueden establecer una correspondencia con el mismo archivo al mismo tiempo. Se ofrecen dos opciones para compartir. En la primera se comparten todas las páginas, de modo que las escrituras realizadas por un proceso sean visibles para todos los demás. Esta opción ofrece un camino de comunicación entre procesos con gran ancho de banda. La otra opción comparte las páginas en tanto ningún proceso las modifique. Apenas un proceso intente escribir en una página, obtendrá un fallo de protección, lo que hará que el sistema operativo le entregue una copia privada de la página para que escriba en ella. Este esquema, llamado **copiar al escribir**, se usa cuando cada proceso necesita creer que es el único que tiene una correspondencia con el archivo.

Memoria virtual en Windows NT

En NT cada proceso de usuario tiene su propio espacio de direcciones virtual. Las direcciones virtuales son de 32 bits, así que cada proceso tiene 4 GB de espacio de direcciones virtual. Los 2 GB inferiores están disponibles para el código y los datos del proceso. Los 2 GB superiores ofrecen acceso (limitado) a la memoria del núcleo, excepto en las versiones para

empresa de NT en las que la división puede ser 3 GB para el usuario y 1 GB para el núcleo o kernel. El espacio de direcciones virtual se pagina por demanda, con tamaño de página fijo (4 KB en el Pentium II).

Cada página virtual puede estar en uno de tres estados: libre, reservada o confirmada. Una **página libre** no se está usando actualmente y una referencia a ella causa un fallo de página. Cuando un proceso se inicia, todas sus páginas están en el estado libre hasta que el programa y los datos iniciales se hacen corresponder con su espacio de direcciones. Una vez que se ha hecho corresponder código o datos con una página, se dice que la página está **confirmada**. Una referencia a una página confirmada se mapea usando el hardware de memoria virtual y tiene éxito si la página está en la memoria principal. Si no es así, ocurre un fallo de página y el sistema operativo busca la página en el disco y la trae a la memoria. Una página virtual también puede estar en el estado **reservado**, lo que significa que no está disponible para mapearse hasta que la reserva se elimine explícitamente. Además de los atributos de libre, reservada y comprometida, las páginas tienen otros atributos, como poderse leer, escribir y ejecutar. Los 64 KB superiores y los 64 KB inferiores de la memoria siempre están libres, a fin de atrapar errores de apuntador (los apuntadores no inicializados suelen valer 0 o -1).

Cada página confirmada tiene una página sombra en el disco, donde se guarda cuando no está en la memoria principal. Las páginas libres y reservadas no tienen páginas sombra, por lo que las referencias a ellas causan fallos de página (el sistema no puede traer una página del disco si no existe en el disco). Las páginas sombra en el disco se organizan en uno o más archivos de paginación. El sistema operativo se mantiene al tanto de cuál página virtual corresponde a qué parte de cuál archivo de paginación. En el caso de texto de programa (sólo para ejecución), el archivo binario ejecutable contiene las páginas sombra; en el caso de páginas de datos, se usan archivos de paginación especiales.

NT, al igual que System V, permite hacer corresponder archivos directamente con regiones de los espacios de direcciones virtuales (es decir, trabaja con páginas consecutivas). Una vez que se ha hecho corresponder un archivo con un espacio de direcciones, se puede leer o escribir usando referencias a la memoria ordinarias.

Los archivos con correspondencia en la memoria se implementan de la misma manera que otras páginas confirmadas, sólo que las páginas sombra pueden estar en el archivo de disco en lugar del archivo de paginación. Por ello, cuando un archivo se mapea en la memoria la versión que está en la memoria podría no ser idéntica a la versión en disco (por escrituras recientes en el espacio virtual de direcciones). Sin embargo, cuando el archivo no está mapeado, o cuando se borra explícitamente, la versión en disco se actualiza.

NT permite explícitamente a dos o más procesos mapear el mismo archivo al mismo tiempo, posiblemente en diferentes direcciones virtuales. Mediante la lectura y escritura de palabras en la memoria, los procesos pueden comunicarse entre sí y enviarse datos con gran ancho de banda, ya que no es necesario copiar. Diferentes procesos podrían tener diferentes permisos de acceso. Puesto que todos los procesos que usan un archivo mapeado comparten las mismas páginas, los cambios hechos por uno de ellos son visibles de inmediato para todos los demás, aunque todavía no se haya actualizado el archivo de disco.

La API Win32 contiene varias funciones que permiten a un proceso administrar explícitamente su memoria virtual. Las más importantes de estas funciones se enumeran en la figura

6-34. Todas ellas operan sobre una región que consiste en una sola página o bien en una sucesión de dos o más páginas que son consecutivas en el espacio de direcciones virtual.

Función API	Significado
VirtualAlloc	Reservar o confirmar una región
VirtualFree	Liberar o desconfirmar una región
VirtualProtect	Cambiar la protección leer/escribir/ejecutar de una región
VirtualQuery	Consultar la situación de una región
VirtualLock	Hacer a una región residente en la memoria (es decir, inhabilitar su paginación)
VirtualUnlock	Hacer una región paginable de la forma normal
CreateFileMapping	Crear un objeto de mapeo de archivo y (opcionalmente) asignarle un nombre
MapViewOfFile	Mapear (parte de) un archivo en el espacio de direcciones
UnmapViewOfFile	Quitar un archivo mapeado del espacio de direcciones
OpenFileMapping	Abrir un objeto de mapeo de archivo creado previamente

Figura 6-34. Principales funciones de la API para administrar la memoria virtual en Windows NT.

Las primeras cuatro funciones de la API no requieren explicación. Las dos que siguen confieren a un proceso la capacidad de fijar hasta 30 páginas en memoria de modo que no se intercambien a disco, y revocar esta propiedad. Un programa en tiempo real podría necesitar esta propiedad, por ejemplo. El sistema operativo establece un límite para evitar que los procesos se vuelvan acaparadores de los recursos . Aunque no se muestran en la figura 6-34, NT también tiene funciones que permiten a un proceso acceder a la memoria virtual de un proceso distinto sobre el cual se le ha concedido control (es decir, para el cual tiene un mango).

Las últimas cuatro funciones de la lista son para administrar archivos con correspondencia en la memoria. Para mapear un archivo, primero hay que crear un objeto de mapeo de archivo con **CreateFileMapping**. Esta función devuelve un mango para el objeto de mapeo de archivo y opcionalmente asienta en el sistema de archivos un nombre para ese objeto a fin de que otro proceso pueda usarlo. Las dos funciones que siguen establecen y anulan la correspondencia de archivos, respectivamente. Un proceso puede usar la última para mapear un archivo que actualmente también está mapeado por un proceso distinto. De este modo, dos o más procesos pueden compartir regiones de sus espacios de direcciones.

Estas funciones de la API son las fundamentales sobre las que se construye el resto del sistema de administración de memoria. Por ejemplo, hay funciones API que asignan espacio a estructuras de datos en uno o más montículos, y que liberan ese espacio. Se usan montículos para almacenar estructuras de datos que se crean y destruyen dinámicamente. No se aplica recolección de basura a los montículos, así que es responsabilidad del software usuario liberar los bloques de memoria virtual que ya no se usan. (La recolección de basura es la eliminación automática de estructuras de datos no utilizadas, por parte del sistema.) El uso de montículos en NT es similar al uso de la función *malloc* en sistemas UNIX, excepto que ahí puede haber múltiples montículos administrados de forma independiente.

6.4.3 Ejemplos de E/S virtual

La tarea fundamental de cualquier sistema operativo es proveer servicios a los programas de usuario, principalmente servicios de E/S como leer y escribir archivos. Tanto UNIX como NT ofrecen una amplia variedad de servicios de E/S a los programas de usuario. Para casi todas las llamadas al sistema de UNIX, NT tiene una llamada equivalente, pero al contrario no se cumple, pues NT tiene muchas más llamadas y cada una es mucho más complicada que su contraparte en UNIX.

E/S virtual en UNIX

Gran parte de la popularidad del sistema UNIX se debe directamente a su sencillez, que a su vez es un resultado directo de la organización del sistema de archivos. Un archivo ordinario es una sucesión de bytes de 8 bits que comienzan en 0 y llegan hasta un máximo de $2^{32} - 1$ bytes. El sistema operativo en sí no impone una estructura de registros a los archivos, aunque muchos programas de usuario consideran a los archivos de texto ASCII como secuencias de líneas, cada una de las cuales termina con un salto de línea.

Cada archivo abierto tiene asociado un apuntador al siguiente byte que se leerá o escribirá. Las llamadas al sistema **read** y **write** leen y escriben datos a partir de la posición en el archivo indicada por dicho apuntador. Ambas llamadas adelantan el apuntador después de la operación en una cantidad igual al número de bytes transferidos. Sin embargo, se puede acceder aleatoriamente a los archivos asignando explícitamente un valor específico al apuntador del archivo.

Además de los archivos ordinarios, el sistema UNIX también reconoce archivos especiales, que sirven para acceder a dispositivos de E/S. A cada dispositivo de E/S se le asigna por lo regular uno o más archivos especiales. Al leer del archivo especial asociado, y escribir en él, un programa puede leer del dispositivo de E/S o escribir en él. Los discos, impresoras, terminales y muchos otros dispositivos se manejan de esta manera.

Las principales llamadas al sistema de UNIX para archivos se enumeran en la figura 6-35. La llamada **creat** (sin la *e* final) se puede usar para crear un archivo nuevo. Hoy día esta llamada no es estrictamente necesaria, porque **open** también puede crear un archivo nuevo. **Unlink** elimina un archivo, siempre que el archivo sólo esté en un directorio.

Open sirve para abrir archivos existentes (y crear archivos nuevos). La bandera **modo** indica cómo se abrirá (para lectura, para escritura, etc.). La llamada devuelve un entero pequeño llamado **descriptor de archivo** que identifica al archivo en llamadas subsecuentes. La E/S real con archivos se efectúa con **read** y **write**, cada una de las cuales tiene un descriptor de archivo que indica cuál archivo debe usarse, un buffer para colocar los datos leídos o por escribir, y una cuenta de bytes que indica cuántos datos deben transmitirse. **Lseek** sirve para ubicar el apuntador de archivo, lo cual hace posible el acceso aleatorio a los archivos.

Stat devuelve información acerca de un archivo, incluido su tamaño, la hora del último acceso, el propietario y más. **Chmod** cambia el modo de protección de un archivo, por ejemplo, permitiendo o prohibiendo a usuarios distintos del propietario que lo lean. Por último, **fcntl** realiza varias operaciones diversas con un archivo, como ponerle o quitarle un candado.

La figura 6-36 ilustra el funcionamiento de las principales llamadas para E/S con archivos. Este código es mínimo y no incluye la verificación de errores necesaria. Antes de ingre-

Llamada al sistema	Significado
creat(name, mode)	Crear un archivo, <i>modo</i> especifica el modo de protección
unlink(name)	Eliminar un archivo (suponiendo que sólo hay un vínculo a él)
open(name, mode)	Abrir o crear un archivo y devolver un descriptor de archivo
close(fd)	Cerrar un archivo
read(fd, buffer, count)	Leer <i>cuenta</i> bytes y colocarlos en <i>buffer</i>
write(fd, buffer, count)	Escribir <i>cuenta</i> bytes de <i>buffer</i>
lseek(fd, offset, w)	Mover el apuntador de archivo según indican <i>distancia</i> y <i>w</i>
stat(name, buffer)	Devolver información acerca de un archivo
chmod(name, mode)	Cambiar el modo de protección de un archivo
fcntl(fd, cmd, ...)	Realizar diversas operaciones de control como poner un candado a (parte de) un archivo

Figura 6-35. Principales llamadas al sistema UNIX para archivos.

sar en el ciclo, el programa abre un archivo existente, *datos*, y crea un archivo nuevo, *anuevo*. Cada llamada devuelve un descriptor de archivo, *dain* y *daout*, respectivamente. Los segundos parámetros de las dos llamadas son bits de protección que especifican que los archivos son para leerse y escribirse, respectivamente. Ambas llamadas devuelven un descriptor de archivo. Si falla *open* o *create*, se devuelve un descriptor de archivo negativo para indicar que la llamada falló.

```
// Open the file descriptors
dain = open("datos", 0);
daout = creat("anuevo", ProtectionBits);

// Ciclo de copiado
do {
    cuenta = read(dain, buffer, bytes);
    if (cuenta > 0) write(daout, buffer, cuenta);
} while (cuenta > 0);

// Cerrar los archivos
close(dain);
close(daout);
```

Figura 6-36. Fragmento de programa para copiar un archivo usando las llamadas al sistema UNIX. Este fragmento está en C porque Java oculta las llamadas al sistema de bajo nivel y estamos tratando de exponerlas.

La llamada a *read* tiene tres parámetros: un descriptor de archivo, un buffer y una cuenta de bytes. La llamada trata de leer el número deseado de bytes del archivo indicado y colocarlos en el buffer. El número de bytes que realmente se leyeron se devuelve en *cuenta*, que será menor que *bytes* si el archivo era demasiado corto. La llamada *write* deposita los bytes recién leídos en el archivo de salida. El ciclo continúa hasta que el archivo de entrada se termina de leer, y entonces el ciclo termina y ambos archivos se cierran.

Los descriptores de archivo en UNIX son enteros pequeños (en general menores que 20). Los descriptores 0, 1 y 2 son especiales y corresponden a la **entrada estándar**, la **salida estándar** y el **error estándar**, respectivamente. Normalmente, éstas se refieren al teclado, el monitor y la pantalla, respectivamente, pero el usuario puede redirigirlas a archivos. Muchos programas UNIX reciben sus entradas de la entrada estándar y escriben los resultados procesados en la salida estándar. Tales programas se denominan **filtros**.

Algo estrechamente relacionado con el sistema de archivos es el sistema de directorios. Cada usuario puede tener varios directorios, y cada directorio puede contener tanto archivos como subdirectorios. Los sistemas UNIX normalmente se configuran con un directorio principal, llamado **directorio raíz**, que contiene los subdirectorios *bin* (para programas que se ejecutan con frecuencia), *dev* (para los archivos especiales de dispositivos de E/S), *lib* (para bibliotecas) y *usr* (para los directorios de los usuarios), como se muestra en la figura 6-37. En este ejemplo, el subdirectorio *usr* contiene los subdirectorios *ast* y *jim*. El directorio *ast* contiene dos archivos, *data* y *foo.p*, y un subdirectorio, *bin*, que contiene cuatro juegos.

Los archivos pueden nombrarse dando su **ruta** desde el directorio raíz. Un camino contiene una lista de todos los directorios por los que se pasa desde la raíz hasta el archivo, separando los nombres de directorio con diagonales. Por ejemplo, el nombre de camino absoluto de *game2* es */usr/ast/bin/game2*. Un camino que comienza en la raíz se llama **ruta absoluta**.

En todo momento, cada programa en ejecución tiene un **directorio de trabajo**. Los nombres de ruta también pueden ser relativos al directorio de trabajo, cuyo nombre no comienza con una diagonal, para distinguirlo de los nombres de ruta absolutos. Tales rutas se llaman **rutas relativas**. Si */usr/ast* es el directorio de trabajo, se puede acceder a *game3* usando el camino *bin/game3*. Un usuario puede crear un **vínculo** con un archivo de otra persona usando la llamada al sistema *link*. En el ejemplo anterior, */usr/ast/bin/game3* y */usr/jim/jotto* accesan el mismo archivo. A fin de evitar ciclos en el sistema de directorios, no se permiten vínculos con directorios. Las llamadas *open* y *creat* reciben nombres de ruta absolutos o bien relativos como argumentos.

Las principales llamadas al sistema UNIX para gestión de directorios se enumeran en la figura 6-38. *Mkdir* crea un directorio nuevo y *rmdir* elimina un directorio existente (vacío). Las siguientes tres llamadas sirven para leer entradas de directorio. La primera abre el directorio, la siguiente lee entradas de él y la última cierra el directorio. *Chdir* cambia el directorio de trabajo.

Link crea una entrada de directorio nueva que apunta a un archivo existente. Por ejemplo, la entrada */usr/jim/jotto* podría haberse creado con la llamada

```
link("/usr/ast/bin/game3", "/usr/jim/jotto")
```

o una llamada equivalente empleando nombres de camino relativos, dependiendo del directorio de trabajo del programa que efectúa la llamada. *Unlink* elimina una entrada de directorio. Si el archivo sólo tiene un vínculo, el archivo se elimina; si tiene dos o más vínculos, se conserva. No importa si el vínculo eliminado es el original o una copia creada posteriormente. Una vez creado un vínculo, es un ciudadano de primera clase, indistinguible del original. La llamada

```
unlink("/usr/ast/bin/game3")
```

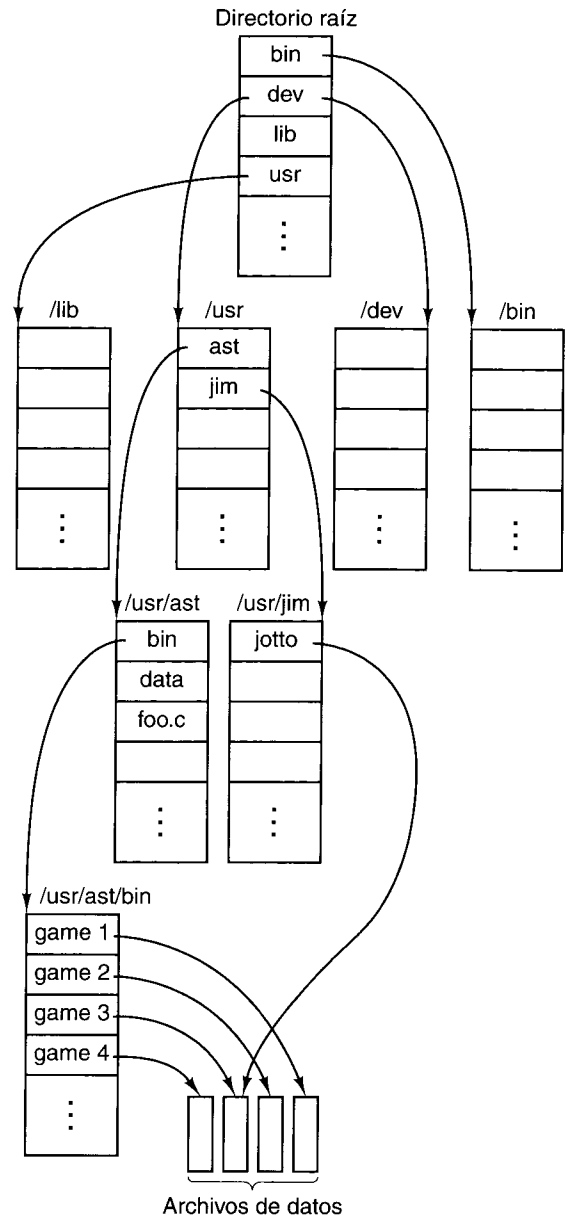


Figura 6-37. Parte de un sistema de directorios UNIX representativo.

hace que en adelante sólo se pueda acceder a *game3* por la ruta */usr/jim/jotto*. Podemos usar *link* y *unlink* de esta forma para “trasladar” archivos de un directorio a otro.

Cada archivo (incluidos los directorios, que también son archivos) está asociado a un mapa de bits que indica quién puede acceder al archivo. El mapa contiene tres campos RWX:

Llamada al sistema	Significado
<code>mkdir(name, mode)</code>	Create un directorio nuevo
<code>rmdir(name)</code>	Eliminar un directorio vacío
<code>opendir(name)</code>	Abrir un directorio para leerlo
<code>readdir(dirpointer)</code>	Leer la siguiente entrada de un directorio
<code>closedir(dirpointer)</code>	Cerrar un directorio
<code>chdir(dirname)</code>	Cambiar el directorio de trabajo a <i>nombredir</i>
<code>link(name1, name2)</code>	Crear una entrada de directorio <i>nombre2</i> que apunta a <i>nombre1</i>
<code>unlink(name)</code>	Eliminar <i>nombre</i> de su directorio

Figura 6-38. Principales llamadas para gestión de directorios en UNIX.

el primero controla los permisos de lectura (*Read*), escritura (*Write*) y ejecución (*eXecute*) del dueño; el segundo hace lo propio para otros del grupo del dueño; y el tercero, para el resto del mundo. Así, *RWX R-X-X* implica que el dueño puede leer, escribir y ejecutar el archivo (obviamente, se trata de un programa ejecutable, pues si no el permiso de ejecutar estaría desactivado), mientras que otros de su grupo pueden leerlo y ejecutarlo, y la gente extraña sólo puede ejecutarlo. Con estos permisos, los extraños pueden usar el programa pero no robárselo (copiarlo) porque no tienen permiso de lectura. La asignación de usuarios a grupos corre por cuenta del administrador del sistema, generalmente llamado **superusuario**. El superusuario también tiene el poder de supeditar el mecanismo de protección y leer, escribir o ejecutar cualquier archivo.

Veamos brevemente cómo se implementan los archivos y directorios en UNIX. Si quiere un tratamiento más completo, vea (Vahalia, 1996). Cada archivo (y cada directorio, porque un directorio también es un archivo) tiene asociado un bloque de información de 64 bytes llamado **nodo i** o **inodo**. El nodo i indica quién es el propietario del archivo, cuáles son los permisos, dónde están los datos y cosas así. Los nodos i de los archivos de cada disco se encuentran en secuencia numérica al principio del disco o bien, si el disco está dividido en grupos de cilindros, al principio de cada grupo de cilindros. Así, al recibir un número de nodo i, el sistema UNIX puede localizar el nodo i con sólo calcular su dirección en disco.

Una entrada de directorio consta de dos partes: un nombre de archivo y un número de nodo i. Cuando un programa ejecuta

```
open("foo.c", 0)
```

el sistema busca el nombre de archivo “foo.c” en el directorio de trabajo para encontrar el número de nodo i para ese archivo. Una vez que encuentra el número de nodo i, el sistema puede leer el nodo i, que le proporciona toda la información acerca del archivo.

Cuando se especifica un nombre de camino más largo, los pasos básicos que acabamos de bosquejar se repiten varias veces hasta que se analiza el camino completo. Por ejemplo, para encontrar el número de nodo i para */usr/ast/data*, el sistema primero busca en el directorio raíz una entrada *usr*. Una vez que encuentra el nodo i para *usr*, el sistema puede leer ese archivo (un directorio es un archivo en UNIX). En este archivo, el sistema busca una entrada *ast*, y así encuentra el número de nodo i del archivo */usr/ast*. Si lee */usr/ast*, el sistema puede

entonces encontrar la entrada para *data*, y por tanto el número de nodo *i* para */usr/ast/data*. Una vez que el sistema tiene el número de nodo *i* del archivo, puede averiguar todo acerca del archivo leyendo el nodo *i*.

El formato, contenido y organización de un nodo *i* varían un poco de un sistema a otro (sobre todo cuando se trabaja con redes), pero casi siempre se encuentran los siguientes elementos en cada nodo *i*.

1. El tipo de archivo, los nueve bits de protección RWX y unos cuantos bits más.
2. El número de vínculos con el archivo (número de entradas de directorio que apuntan a él).
3. La identidad del propietario.
4. El grupo del propietario.
5. La longitud del archivo en bytes.
6. Trece direcciones en disco.
7. La hora en que se leyó por última vez el archivo.
8. La hora en que se escribió por última vez el archivo.
9. La hora en que se modificó por última vez el nodo *i*.

El tipo de archivo distingue los archivos ordinarios, directorios y dos tipos de archivos especiales, para dispositivo de E/S estructurados en bloques y no estructurados, respectivamente. El número de vínculos y la identificación del propietario ya se explicaron. La longitud del archivo es un entero de 32 bits que da el byte más alto que tiene un valor. Es perfectamente válido crear un archivo, efectuar un *lseek* hasta la posición 1,000,000, y escribir un byte, lo que da una longitud de archivo de 1,000,001. Sin embargo, el archivo *no* solicitaría almacenamiento para todos los bytes “faltantes”.

Las primeras 10 direcciones en disco apuntan a bloques de datos. Con un tamaño de bloque de 1024 bytes, se pueden manejar de este modo archivos de hasta 10,240 bytes. La dirección 11 apunta a un bloque de disco, llamado **bloque indirecto**, que contiene 256 direcciones en disco. Los archivos de hasta $10,240 + 256 \times 1024 = 272,384$ bytes se manejan de esta manera. En el caso de archivos aún más grandes, la dirección 12 apunta a un bloque que contiene las direcciones de 256 bloques indirectos, con lo que se pueden manejar archivos de hasta $272,384 + 256 \times 256 \times 1024 = 67,381,248$ bytes. Si este esquema de **bloque indirecto doble** aún es demasiado pequeño, se usa la dirección en disco 13 que apunta a un **bloque indirecto triple** que contiene las direcciones de 256 bloques indirectos dobles. Si se usan las direcciones indirectas, directas, sencillas y dobles, es posible direccionar hasta 16,843,018 bloques que dan un tamaño de archivo máximo teórico de 17,247,250,432 bytes. Puesto que los apuntadores de archivo están limitados a 32 bits, el límite superior práctico es en realidad de 4,294,967,295 bytes. Los bloques de disco libres se mantienen en una lista enlazada. Cuando se necesita un bloque nuevo, se toma el siguiente bloque de la lista. El resultado es que los bloques de cada archivo están dispersos aleatoriamente por todo el disco.

A fin de hacer más eficiente la E/S de disco, cuando un archivo se abre su nodo *i* se copia en una tabla de la memoria principal y se guarda ahí para agilizar las referencias en tanto el archivo siga abierto. Además, se mantiene en la memoria una reserva de bloques de disco a los que recientemente se hizo referencia. Dado que la mayor parte de los archivos se lee en forma secuencial, es común que una referencia a un archivo requiera el mismo bloque de disco que la referencia anterior. Para reforzar este efecto, el sistema también trata de leer el *siguiente* bloque de un archivo, antes de que se haga referencia a él, a fin de agilizar el procesamiento. El usuario no percibe toda esta optimización; cuando un usuario emite una llamada *read*, el programa se suspende hasta que los datos solicitados están en el buffer.

Con estos antecedentes, podemos ver cómo funciona la E/S de archivos. *Open* hace que el sistema busque en los directorios el camino especificado. Si la búsqueda tiene éxito, se lee el nodo *i* y se coloca en una tabla interna. Las llamadas *read* y *write* hacen que el sistema calcule el número de bloque a partir de la posición actual en el archivo. Las direcciones en disco de los 10 primeros bloques siempre están en la memoria principal (en el nodo *i*); los bloques con números más grandes requieren la lectura de uno o más bloques indirectos primero. *Lseek* simplemente modifica el apuntador a la posición actual, sin efectuar E/S.

Ahora también es fácil entender *link* y *unlink*. *Link* busca su primer argumento para encontrar el número de nodo *i*; luego crea una entrada de directorio para el segundo argumento y coloca el número de nodo *i* del primer archivo en esa entrada; por último, incrementa en uno la cuenta de vínculos en el nodo *i*. *Unlink* elimina una entrada de directorio y decrementa la cuenta de vínculos en el nodo *i*. Si la cuenta es cero, el archivo se borra y todos los bloques se colocan en la lista libre.

E/S virtual en Windows NT

NT maneja varios sistemas de archivos, de los cuales los más importantes son NTFS (NT File System) y el sistema de archivos FAT (tabla de asignación de archivos, *File Allocation Table*). El primero es un sistema de archivos nuevo creado específicamente para NT; el segundo es el antiguo sistema de archivos de MS-DOS, que también se usa en Windows 95/98 (aunque con apoyo para nombres de archivo más largos). Puesto que el sistema de archivos FAT es básicamente obsoleto (aunque varios cientos de millones de personas siguen usándolo), estudiaremos NTFS a continuación. A partir de NT 5.0, también se reconoce el sistema de archivos FAT32 empleado en versiones posteriores de Windows 95 y en Windows 98.

Los nombres de archivo en NTFS pueden tener hasta 255 caracteres, y están en Unicode, lo que permite a personas de países que no usan el alfabeto latino (por ejemplo, Japón, India e Israel) escribir nombres de archivo en su lenguaje nativo. (De hecho, NT usa Unicode internamente; las versiones a partir de NT 5.0 tienen un solo binario que se puede usar en cualquier país y emplear el idioma local porque todos los menús, mensajes de error, etc., se guardan en archivos de configuración que dependen del país.) NTFS apoya plenamente nombres sensibles a la caja (de modo que *foo* es diferente de *FOO*). Lo malo es que la API Win32 no apoya plenamente la sensibilidad a la caja para los nombres de archivo, y en absoluto para los nombres de directorio, por lo que los programas que usan Win32 no tienen esta ventaja.

Al igual que en UNIX, un archivo no es más que una secuencia lineal de bytes, aunque hasta un máximo de $2^{64} - 1$. También existen apuntadores de archivo, como en UNIX, pero tienen capacidad de 64 bits, en lugar de 32, a fin de manejar la longitud de archivo máxima. Las llamadas a funciones de la API Win32 para manipular archivos y directorios son similares a grandes rasgos a sus contrapartes en UNIX, excepto que casi todas tienen más parámetros y el modelo de seguridad es distinto. La apertura de un archivo devuelve una manija, que entonces se usa para leer y escribir el archivo. Sin embargo, a diferencia de UNIX, las manijas no son enteros pequeños, y la entrada estándar, la salida estándar y el error estándar tienen que adquirirse explícitamente en lugar de estar predefinidos como 0, 1 y 2 (excepto en modo de consola, en el que se preabren). Las principales funciones de la API Win32 para gestión de archivos se enumeran en la figura 6-39.

Función API	UNIX	Significado
CreateFile	open	Crear un archivo o abrir un archivo existente; devolver una manija
DeleFile	unlink	Destruir un archivo existente
CloseHandle	close	Cerrar un archivo
ReadFile	read	Leer datos de un archivo
WriteFile	write	Escribir datos en un archivo
SetFilePointer	lseek	Hacer que el apuntador de archivo apunte a un lugar específico del archivo
GetFileAttributes	stat	Devolver las propiedades del archivo
LockFile	fcntl	Poner candado a una región del archivo para tener exclusión mutua
UnlockFile	fcntl	Quitar el candado puesto antes a una región del archivo

Figura 6-39. Principales funciones de la API Win32 para E/S con archivos. La segunda columna da el equivalente más cercano en UNIX.

Examinemos estas llamadas brevemente. `CreateFile` puede servir para crear un archivo nuevo y devolver una manija para él. Esta función API también sirve para abrir archivos existentes porque no existe una función API `open`. No enumeramos los parámetros para las funciones API de NT porque son muy voluminosos. Por ejemplo, `CreateFile` tiene siete parámetros, a saber:

1. Un apuntador al nombre del archivo que se creará o abrirá.
2. Banderas que indican si el archivo puede leerse, escribirse o ambas cosas.
3. Banderas que indican si varios procesos pueden abrir el archivo a la vez.
4. Un apuntador al descriptor de seguridad, que indica quién puede acceder al archivo.
5. Banderas que indican qué hacer si el archivo existe/no existe.
6. Banderas que se ocupan de atributos como archivado, compresión, etc.
7. La manija de un archivo cuyos atributos deberán clonarse para el nuevo archivo.

Las siguientes seis funciones API de la figura 6-39 son muy parecidas a las llamadas al sistema UNIX correspondientes. Las últimas dos permiten poner y quitar un candado a una región de un archivo para que un proceso pueda obtener exclusión mutua garantizada en su acceso a ella.

Con estas funciones API es posible escribir un procedimiento para copiar un archivo, análogo a la versión UNIX de la figura 6-36. En la figura 6-40 se muestra un procedimiento semejante (sin verificación de errores), diseñado para imitar la estructura de la figura 6-36. En la práctica no sería necesario escribir un programa para copiar archivos porque existe la función API `CopyFile` (que ejecuta algo parecido a este programa como procedimiento de biblioteca).

```
// Abrir archivos para entrada y salida.
mangoin = CreateFile("datos", GENERIC_READ, 0, NULL, OPEN_EXISTING, 0, NULL);
mangoout = CreateFile("anuevo", GENERIC_WRITE, 0, NULL, CREATE, ALWAYS,
FILE_ATTRIBUTE_NORMAL, NULL);
```

```
// Copiar el archivo.
do {
    s = ReadFile(mangoin, buffer, BUF_SIZE, &cuenta, NULL);
    if (s > 0 && cuenta > 0) WriteFile(mangoout, buffer, cuenta, &ocnt, NULL);
} while (s > 0 && cuenta > 0);
```

```
// Cerrar los archivos.
CloseHandle(mangoin);
CloseHandle (mangoout);
```

Figura 6-40. Fragmento de programa para copiar un archivo usando las funciones API de Windows NT. Este fragmento está en C porque Java oculta las llamadas al sistema de bajo nivel y estamos tratando de exponerlas.

NT maneja un sistema de archivos jerárquico similar al de UNIX. Sin embargo, el separador de nombres componentes es \ en lugar de /, un fósil heredado de MS-DOS. Existe el concepto de directorio de trabajo vigente y los nombres de ruta pueden ser relativos o absolutos. Una diferencia importante respecto a UNIX es que UNIX permite montar sistemas de archivos que están en diferentes discos y máquinas juntos en un solo árbol de nombres, lo que oculta la estructura de discos de todo el software. NT 4.0 no tiene esta propiedad, por lo que los nombres de archivo absolutos deben comenzar con una letra de unidad que indica de cuál disco lógico se trata, como en `C:\windows\system\foo.dll`. A partir de NT 5.0, se añadió el montaje de sistemas de archivos al estilo UNIX.

Las principales funciones API para gestión de directorios se dan en la figura 6-41, otra vez con sus equivalentes UNIX más cercanos. Creemos que las funciones no requieren explicación.

Cabe señalar que NT 4.0 no manejaba vínculos de archivos. En el nivel del escritorio gráfico se manejaban atajos, pero estas construcciones eran internas del escritorio y no tenían contraparte en el sistema de archivos mismo. Un programa de aplicación no podía incluir un archivo en un segundo directorio sin copiar todo el archivo. A partir de NT 5.0 se añadió la vinculación de archivos al sistema de archivos propiamente dicho.

Función API	UNIX	Significado
CreateDirectory	mkdir	Crear un directorio nuevo
RemoveDirectory	rmdir	Eliminar un directorio vacío
FindFirstFile	opendir	Inicializar para comenzar a leer las entradas de un directorio
FindNextFile	readdir	Leer la siguiente entrada del directorio
MoveFile		Pasar un archivo de un directorio a otro
SetCurrentDirectory	chdir	Cambiar el directorio de trabajo vigente

Figura 6-41. Principales funciones de la API Win32 para gestión de directorios. La segunda columna da el equivalente UNIX más cercano, si existe.

NT tiene un mecanismo de seguridad mucho más completo que el de UNIX. Aunque hay cientos de funciones API relacionadas con la seguridad, la breve descripción que sigue da la idea general. Cuando un usuario ingresa en el sistema, su proceso inicial recibe una **ficha de acceso** del sistema operativo. La ficha de acceso contiene el **identificador de seguridad (SID, Security ID)** del usuario, una lista de los grupos de seguridad a los que el usuario pertenece, privilegios especiales que estén disponibles, y unas cuantas cosas más. El motivo para tener una ficha de acceso es concentrar toda la información sobre seguridad en un solo lugar fácil de encontrar. Todos los procesos creados por este proceso heredan la misma ficha de acceso.

Uno de los parámetros que pueden proporcionarse cuando se crea cualquier objeto es su **descriptor de seguridad**, el cual contiene una lista de entradas llamada **lista de control de acceso (ACL, Access Control List)**. Cada entrada permite o prohíbe que algún SID o grupo lleve a cabo algún conjunto de las operaciones con el objeto. Por ejemplo, un archivo podría tener un descriptor de seguridad que especifique que Leonor no tiene ningún acceso al archivo, Carlos puede leer el archivo, Linda puede leer o escribir el archivo, y que todos los miembros del grupo XYZ pueden leer la longitud del archivo pero nada más.

Cuando un proceso trata de realizar una operación con un objeto usando la manija que obtuvo cuando creó el objeto, el gestor de seguridad obtiene la ficha de acceso del proceso y revisa la lista de entradas de la ACL en orden. Tan pronto como se encuentra una entrada que coincide con el SID del invocador o uno de los grupos del invocador, el acceso que se encuentra ahí se toma como definitivo. Por esta razón, se acostumbra colocar en la ACL las entradas que niegan acceso adelante de las que otorgan acceso, para que un usuario al que se niega específicamente el acceso no pueda introducirse por una “puerta trasera” al ser miembro de un grupo al que se le permite el acceso. El descriptor de seguridad también contiene información que sirve para auditar los accesos al objeto.

Veamos ahora con brevedad cómo se implementan los archivos y directorios en NT. Cada disco se divide estáticamente en volúmenes autosuficientes, que equivalen a las particiones de disco en UNIX. Cada volumen contiene archivos, directorios, mapas de bits y otras estructuras de datos para manejar su información. El volumen se organiza como una secuencia lineal de **cúmulos** cuyo tamaño es fijo para cada volumen y varía entre 512 bytes y 64 KB, dependiendo del tamaño de volumen. Se hace referencia a los cúmulos por su distancia al principio del volumen empleando números de 64 bits.

La principal estructura de datos de cada volumen es la **tabla maestra de archivos (MFT, Master File Table)**, que tiene una entrada para cada archivo y directorio del volumen. Estas entradas son análogas a los nodos *i* en UNIX. La MFT en sí es un archivo, y como tal puede colocarse en cualquier lugar del volumen, lo que elimina el problema que UNIX tiene con bloques de disco defectuosos en medio de los nodos *i*.

En la figura 6-42 se muestra la MFT, que comienza con una cabecera que contiene información acerca del volumen, como (apuntadores a) el directorio raíz, el archivo de arranque, el archivo de bloques defectuosos, la administración de lista libre, etc. Después viene una entrada por cada archivo o directorio, de 1 KB excepto cuando el tamaño de cúmulo es de 2 KB o más. Cada entrada contiene todos los metadatos (información administrativa) acerca del archivo o directorio. Se permiten varios formatos, y en la figura 6-42 se muestra uno de ellos.

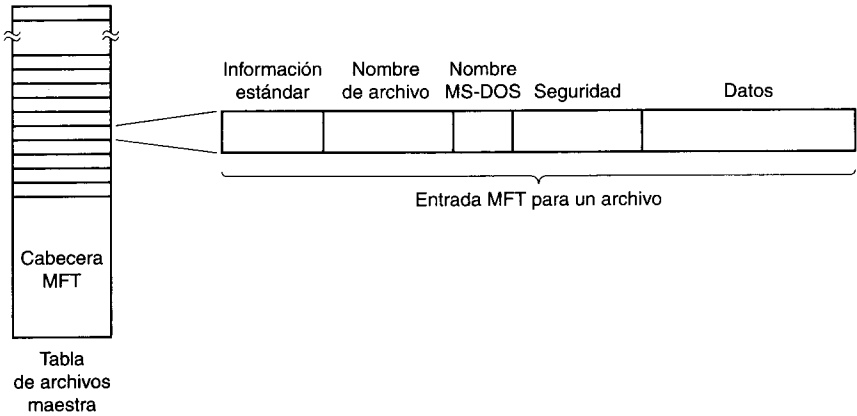


Figura 6-42. La tabla maestra de archivos de Windows NT.

El campo de información estándar contiene información como las marcas de tiempo que necesita POSIX, el número de vínculos a disco duro, los bits de sólo lectura y almacenamiento, etc. Se trata de un campo de longitud fija y siempre está presente. El nombre de archivo tiene una longitud variable, hasta 255 caracteres Unicode. Para que tales archivos sean accesibles a programas viejos de 16 bits, los archivos pueden tener también un nombre MS-DOS, que consiste en ocho caracteres alfanuméricos seguidos opcionalmente de un punto y una extensión de hasta tres caracteres alfanuméricos. Si el archivo ya se ajusta a la regla de nombres 8 + 3 de MS-DOS, no se usa un nombre MS-DOS secundario.

Luego viene la información de seguridad. En versiones hasta NT 4.0 inclusive, el campo de seguridad contenía el descriptor de seguridad real. A partir de NT 5.0 toda la información de seguridad se centralizó en un solo archivo, y el campo de seguridad simplemente apunta a la parte pertinente de ese archivo.

En el caso de archivos pequeños, los datos del archivo mismos están contenidos en la entrada de MFT, lo que ahorra un acceso a disco para obtenerla. Esta idea se llama **archivo inmediato** (Mullender y Tanenbaum, 1987). En el caso de archivos un poco más grandes, este campo contiene apuntadores a los cúmulos que contienen los datos o, lo que es más

común, a series de cúmulos consecutivos, por lo que un solo número de cúmulo y una longitud pueden representar una cantidad arbitraria de datos del archivo. Si una sola entrada de MFT no basta para contener toda la información que se supone debe contener, se pueden encadenar a ella una o más entradas adicionales.

El tamaño de archivo máximo es de 2^{64} bytes. Para tener una idea de cuán grande es un archivo de 2^{64} bytes, imagine que el archivo se escribe en binario y cada 0 o 1 ocupa 1 mm de espacio. El listado de 2^{67} mm tendría 15 años luz de longitud, y podría salir del Sistema Solar, llegar a Alpha Centauri, y regresar.

El sistema de archivos NTFS tiene muchas otras propiedades interesantes que incluyen compresión de datos y tolerancia de fallos empleando transacciones atómicas. Se puede encontrar información adicional al respecto en (Solomon, 1998).

6.4.4 Ejemplos de gestión de procesos

Tanto UNIX como NT permiten dividir un trabajo en varios procesos que se pueden ejecutar en (pseudo)paralelo y comunicarse entre sí, como en el ejemplo del productor y el consumidor que vimos antes. En esta sección explicaremos cómo se controlan los procesos en ambos sistemas. Además, los dos sistemas apoyan el paralelismo dentro de un solo procesador empleando enlaces, así que también hablaremos de esto.

Gestión de procesos en UNIX

En cualquier momento, un proceso UNIX puede crear un subproceso que es una réplica exacta de sí mismo, ejecutando la llamada de sistema denominada **fork**. El proceso original se llama **padre** y el nuevo se llama **hijo**. Inmediatamente después de la **fork**, los dos procesos son idénticos e incluso comparten los mismos descriptores de archivo. En adelante, cada uno sigue su propio camino y se comporta con independencia del otro.

En muchos casos, el proceso hijo manipula los descriptores de archivo de ciertas maneras y luego ejecuta la llamada de sistema **exec**, que sustituye su programa y datos por los programas y datos que se encuentran en un archivo ejecutable que se especifica como parámetro de la llamada **exec**. Por ejemplo, cuando un usuario teclea un comando *xyz* en una terminal, el intérprete de comandos (*shell*) ejecuta **fork** para crear un proceso hijo. Luego, este proceso hijo emite **exec** para ejecutar el programa *xyz*.

Los dos procesos se ejecutan en paralelo (con o sin **exec**), a menos que el padre desee esperar a que el hijo termine antes de continuar. En tal caso, el padre emite una de dos llamadas de sistema, **wait** o **waitpid**, que hacen que el padre se suspenda hasta que el hijo termine emitiendo **exit**. Una vez que el hijo termina, el padre continúa.

Los procesos pueden ejecutar **fork** cuantas veces quieran, y esto da pie a un árbol de procesos. En la figura 6-43, por ejemplo, el proceso *A* emitió **fork** dos veces y creó dos hijos, *B* y *C*. Luego *B* emitió **fork** dos veces, y *C* lo hizo una vez, para dar el árbol final de seis procesos.

Los procesos en UNIX pueden comunicarse entre ellos mediante una estructura llamada **fila** (*pipe*). Una fila es una especie de buffer en el que un proceso puede escribir un flujo de

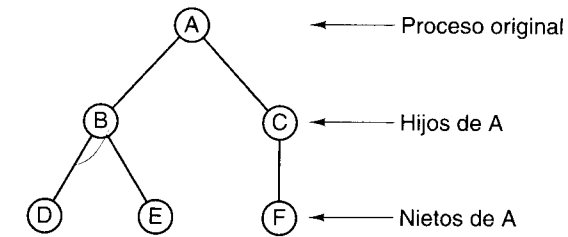


Figura 6-43. Árbol de procesos en UNIX.

datos y otro puede sacar el flujo de él. Los bytes siempre se recuperan de una fila en el orden en que se escribieron; no puede haber acceso aleatorio. Las filas no preservan las fronteras de los mensajes, de modo que si un proceso efectúa cuatro escrituras de 128 bytes y el otro efectúa una lectura de 512 bytes, el lector obtendrá todos los datos juntos, sin indicio de que se escribieron en varias operaciones.

En System V y Solaris, otro mecanismo de comunicación entre procesos es el uso de **colas de mensajes**. Un proceso puede crear una cola de mensajes o abrir una cola ya existente usando **msgget**. Mediante una cola de mensajes, un proceso puede enviar mensajes con **msgsnd** y recibirlos con **msgrcv**. Los mensajes así enviados difieren de los datos dentro de una fila en varios sentidos. Primero, las fronteras de los mensajes se preservan, mientras que una fila no es más que un flujo de datos. Segundo, los mensajes tienen prioridades, de modo que los urgentes pueden adelantarse a otros menos importantes. Tercero, los mensajes tienen tipos, y un **msgrcv** puede especificar un tipo dado si lo desea.

Otro mecanismo de comunicación es la capacidad de dos o más procesos para compartir una región de sus espacios de direcciones respectivos. UNIX maneja esta memoria compartida haciendo corresponder las mismas páginas con el espacio de direcciones virtual de todos los procesos que las comparten. El resultado es que una escritura efectuada por un proceso en la región compartida es visible de inmediato para los demás procesos. Este mecanismo ofrece un camino de comunicación con gran ancho de banda entre los procesos. Las llamadas de sistema que se usan para manejar memoria compartida tienen nombres como **shmat** y **shmop**.

Otra característica de System V y Solaris es la disponibilidad de semáforos. Éstos funcionan básicamente como se describió en el ejemplo del productor-consumidor que se dio en el texto.

Otro recurso que proporcionan todos los sistemas UNIX que cumplen con POSIX es la capacidad para tener varios enlaces de control dentro de un solo proceso. Estos enlaces de control, generalmente llamados sólo **enlaces** (*threads*) son como procesos ligeros que comparten un mismo espacio de direcciones y todo lo asociado con dicho espacio, como descriptores de archivo, variables de entorno y temporizadores pendientes. Sin embargo, cada enlace tiene su propio contador de programa, sus propios registros y su propia pila. Cuando un enlace se bloquea (es decir, se detiene temporalmente hasta que termina una operación de E/S u ocurre algún otro suceso), otros enlaces del mismo proceso pueden seguir ejecutándose. Dos enlaces del mismo proceso que operan como productor y consumidor son similares, pero no idénticos, a dos procesos de un solo enlace cada uno que comparten un segmento de

memoria que contiene un buffer. Las diferencias tienen que ver con el hecho de que en el segundo caso cada proceso tiene sus propios descriptores de archivo, etc., mientras que en el primero todas esas cosas son compartidas. Ya vimos el uso de enlaces Java en nuestro ejemplo del productor-consumidor. Es común que el sistema de tiempo de ejecución de Java use un enlace del sistema operativo para cada uno de sus enlaces, pero esto no es obligatorio.

Como ejemplo de caso en el que los enlaces podrían ser útiles, considere un servidor de la World Wide Web. Un servidor así podría mantener una caché de páginas Web de uso común en la memoria principal. Si una solicitud hace referencia a una página que está en la caché, la página Web se devuelve de inmediato; si no, se trae del disco. Lo malo es que la espera mientras llega la página del disco es larga (digamos 20 ms), y durante este tiempo el proceso está bloqueado y no puede atender ninguna solicitud entrante, ni siquiera las que piden páginas Web que están en la caché.

La solución es tener varios enlaces dentro del proceso servidor, todos los cuales comparten la caché común de páginas Web. Cuando un enlace se bloquea, otros enlaces pueden atender solicitudes nuevas. Para evitar el bloqueo sin enlaces, podríamos tener varios procesos servidores, pero es probable que ello implicaría duplicar la caché, lo que desperdiciaría memoria valiosa.

El estándar UNIX para enlaces se llama **threads** y está definido por POSIX (P1003.1C). El estándar contiene llamadas para gestionar y sincronizar enlaces. No está definido si los enlaces están bajo el control del núcleo o totalmente en el espacio de usuario. Las llamadas para enlaces de uso más común se enumeran en la figura 6-44.

Llamada para enlaces	Significado
<code>pthread_create</code>	Crear un enlace nuevo en el espacio de direcciones del invocador
<code>pthread_exit</code>	Terminar el enlace invocado
<code>pthread_join</code>	Esperar que un enlace termine
<code>pthread_mutex_init</code>	Crear un mutex nuevo
<code>pthread_mutex_destroy</code>	Destruir un mutex
<code>pthread_mutex_lock</code>	Cerrar un mutex
<code>pthread_mutex_unlock</code>	Abrir un mutex
<code>pthread_cond_init</code>	Crear una variable de condición
<code>pthread_cond_destroy</code>	Destruir una variable de condición
<code>pthread_cond_wait</code>	Esperar por una variable de condición
<code>pthread_cond_signal</code>	Liberar un enlace que espera por una variable de condición

Figura 6-44. Principales llamadas de POSIX para enlaces.

Examinemos brevemente las llamadas para enlaces que se muestran en la figura 6-44. La primera llamada, `pthread_create`, crea un enlace nuevo. Si la llamada se lleva a cabo con éxito, se estará ejecutando un enlace más en el espacio de direcciones que los que se estaban ejecutando antes de la llamada. Un enlace que terminó su trabajo y quiere terminar invoca `pthread_exit`. Un enlace puede esperar que otro enlace termine invocando `pthread_join`. Si el enlace que se desea esperar ya terminó, `pthread_join` termina de inmediato; de lo contrario, se bloquea.

Los enlaces pueden sincronizarse empleando candados llamados **mutexes**. Por lo regular, un mutex protege a algún recurso, como un buffer compartido por dos enlaces. Para asegurarse de que sólo un enlace a la vez accese al recurso compartido, se espera que los enlaces cierren el mutex antes de tocar el recurso y lo abran cuando hayan terminado. En tanto todos los enlaces obedezcan este protocolo, podrán evitarse condiciones de competencia. Los mutexes son como semáforos binarios, es decir, semáforos que sólo pueden adoptar los valores 0 y 1. El nombre "mutex" proviene de su uso para asegurar la mutua exclusión en el uso de algún recurso.

Los mutexes se pueden crear y destruir con las llamadas `pthread_mutex_init` y `pthread_mutex_destroy`, respectivamente. Un mutex puede estar en uno de dos estados: cerrado o abierto. Cuando un enlace trata de cerrar un mutex abierto (usando `pthread_mutex_lock`), el candado se cierra y el enlace continúa. En cambio, si un enlace trata de cerrar un mutex que ya está cerrado, se bloquea. Cuando el enlace que cerró el mutex termina de usar el recurso compartido, se espera que abra el mutex correspondiente invocando `pthread_mutex_unlock`.

Los mutexes se diseñaron para bloqueo a corto plazo, como la protección de una variable compartida, no para una sincronización a largo plazo, como esperar que una unidad de cinta se desocupe. Para este fin se cuenta con **variables de condición**, las cuales se crean y se destruyen con llamadas a `pthread_cond_init` y `pthread_cond_destroy`, respectivamente.

Usamos una variable de condición haciendo que un enlace la espere y que otro la señalice. Por ejemplo, si descubre que la unidad de cinta que necesita está ocupada, un enlace invocaría `pthread_cond_wait` con una variable de condición que todos los enlaces han acordado asociar a la unidad de cinta. Cuando el enlace que está usando la unidad de cinta ya no la necesita (tal vez horas después), usa `pthread_cond_signal` para liberar exactamente un enlace de los que están esperando esa variable de condición (si los hay). Si ningún enlace está esperando, la variable se pierde. Las variables de condición no cuentan como los semáforos. También se definen otras operaciones con enlaces, mutexes y variables de condición.

Gestión de procesos en Windows NT

NT maneja múltiples procesos, que pueden comunicarse y sincronizarse. Cada proceso contiene por lo menos un enlace, que a su vez contiene al menos una fibra (enlace ligero). Juntos, los procesos, enlaces y fibras proveen un conjunto muy general de herramientas para gestionar paralelismo, tanto en uniprosesores (máquinas con una sola CPU) como en multiprosesores (máquinas con varias CPU).

Se crean procesos nuevos empleando la función API `CreateProcess`. Esta función tiene 10 parámetros, cada uno de los cuales tiene muchas opciones. Este diseño obviamente es mucho más complicado que el de UNIX, en el que `fork` no tiene parámetros, y `exec` sólo tiene tres: apuntadores al nombre del archivo que se ejecutará, el arreglo de parámetros de la línea de comandos (analizados sintácticamente) y las cadenas de entorno. A grandes rasgos, los 10 parámetros de `CreateProcess` son los siguientes:

1. Un apuntador al nombre del archivo ejecutable.
2. La línea de comandos (sin analizar sintácticamente).
3. Un apuntador a un descriptor de seguridad para el proceso.
4. Un apuntador a un descriptor de seguridad para el enlace inicial.
5. Un bit que indica si el nuevo proceso hereda o no las manijas de su creador.
6. Diversas banderas (por ejemplo, modo de error, prioridad, depuración, consolas).
7. Un apuntador a las cadenas de entorno.
8. Un apuntador al nombre del directorio de trabajo vigente del nuevo proceso.
9. Una apuntador a una estructura que describe la ventana inicial en la pantalla.
10. Un apuntador a una estructura que devuelve 18 valores al invocador.

NT no obliga a establecer una jerarquía padre-hijo ni de ningún otro tipo. Todos los procesos se crean iguales. Sin embargo, dado que uno de los 18 parámetros que se devuelven al proceso creador es un mango para el proceso nuevo (que permite tener un control considerable sobre este último), existe una jerarquía implícita en términos de quién tiene una manija para quién. Aunque estas manijas no pueden pasarse directamente a otros procesos, existe una forma en que un proceso puede hacer una manija apropiada para otro proceso y luego dársela, así que la jerarquía de procesos implícita podría no durar mucho.

Cada proceso NT se crea con un solo enlace, pero un proceso puede crear más enlaces posteriormente. La creación de enlaces es más sencilla que la de procesos: `CreateThread` sólo tiene seis parámetros en lugar de 10: el descriptor de seguridad, el tamaño de pila, la dirección inicial, un parámetro definido por el usuario, el estado inicial del enlace (listo o bloqueado) y el identificador del enlace. El núcleo se encarga de crear los enlaces, por los que tiene un conocimiento claro de ellos (es decir, no se implementan únicamente en el espacio de usuario, como en algunos otros sistemas).

Cuando el núcleo efectúa la planificación, no sólo escoge el proceso que se ejecutará a continuación, sino también el enlace dentro de ese proceso. Esto implica que el núcleo siempre sabe cuáles enlaces están listos y cuáles están bloqueados. Puesto que los enlaces son objetos del núcleo, tienen descriptors de seguridad y manijas. Dado que una manija para un enlace se puede pasar a otro proceso, es posible hacer que un proceso controle los enlaces de un proceso distinto. Esta característica es útil para los depuradores, por ejemplo.

Los enlaces NT son relativamente costosos porque la conmutación de un enlace requiere ingresar en el núcleo y después salir de él. A fin de ofrecer un pseudoparalelismo muy ligero, NT cuenta con **fibras**, que son parecidas a los enlaces sólo que son planificados en el espacio de usuario por el programa que las crea (o su sistema de tiempo de ejecución). Cada enlace puede tener varias fibras, así como un proceso puede tener varios enlaces, excepto que cuando una fibra se bloquea lógicamente se coloca a sí misma en la cola de fibras bloqueadas y selecciona otra fibra que se ejecutará en el contexto de su enlace. El núcleo no tiene conocimiento de esta transición porque el enlace se sigue ejecutando, aunque primero podría estar

ejecutando una fibra y después otra. El núcleo sólo administra procesos e enlaces, no fibras. Las fibras son útiles, por ejemplo, cuando programas que administran sus propios enlaces se trasladan a NT.

Los procesos pueden comunicarse de muy diversas maneras, incluidas filas, filas con nombre, buzones, *sockets*, llamadas a procedimientos remotos y archivos compartidos. Las filas tienen dos modos, de bytes y de mensajes, que se seleccionan en el momento en que se crean. Las filas en modo de bytes funcionan igual que en UNIX. Las filas en modo de mensajes son parecidos pero preservan las fronteras de los mensajes, de modo que cuatro escrituras de 128 bytes se leerán como cuatro mensajes de 128 bytes, no un mensaje de 512 bytes, como ocurriría con filas en modo de bytes. También existen filas con nombre y tienen los mismos dos modos que las filas normales. Las filas con nombres también pueden usarse en una red, cosa que no es posible con los normales.

Los **buzones** (*mailslots*) son una característica de NT que no está presente en UNIX. Los buzones son parecidos a las filas en ciertos aspectos, pero no en todos. En primer lugar, son unidireccionales, mientras que las filas son bidireccionales. Los buzones también pueden usarse en una red pero no garantizan la entrega. Por último, los buzones permiten al proceso transmisor difundir un mensaje a varios receptores, no sólo a uno.

Los *sockets* son como filas, excepto que normalmente conectan procesos de diferentes máquinas; sin embargo, también pueden servir para conectar procesos en la misma máquina. En general, casi nunca representa una ventaja usar una conexión de *socket* en lugar de una fila o fila con nombre para la comunicación dentro de la máquina.

Las llamadas a procedimientos remotos permiten al proceso *A* pedir al proceso *B* que invoque un procedimiento en el espacio de direcciones de *B* a nombre de *A*, y que devuelva el resultado a *A*. Existen diversas restricciones de los parámetros. Por ejemplo, no tiene sentido pasar un apuntador a un proceso distinto.

Por último, los procesos pueden compartir memoria estableciendo una correspondencia con el mismo archivo al mismo tiempo. Así, todas las escrituras efectuadas por un proceso aparecen en los espacios de direcciones de los otros procesos. Con este mecanismo es fácil implementar el buffer compartido que usamos en nuestro ejemplo de productor-consumidor.

Así como NT ofrece varios mecanismos de comunicación entre procesos, también provee numerosos mecanismos de sincronización, incluidos semáforos, mutexes, secciones críticas y eventos. Todos estos mecanismos funcionan con enlaces, no con procesos, de modo que cuando un enlace se bloquea en un semáforo los demás enlaces de ese proceso (si los hay) no resultan afectados y pueden continuar su ejecución.

Se crea un semáforo con la función API `CreateSemaphore`, que puede asignarle un valor inicial dado y definir también un valor máximo. Los semáforos son objetos del núcleo y por ende tienen descriptors de seguridad y manijas. La manija de un semáforo puede duplicarse con `DuplicateHandle` y pasarse a otro proceso para que varios procesos puedan sincronizarse con el mismo semáforo. Hay llamadas para *up* y *down*, aunque tienen los nombres un tanto extraños de `ReleaseSemaphore` (*up*) y `WaitForSingleObject` (*down*). También es posible asignar a `WaitForSingleObject` un plazo de tiempo que permita liberar el enlace invocador tarde o temprano, aunque el semáforo siga siendo 0 (aunque los temporizadores vuelven a introducir competencias).

Los mutexes también son objetos del núcleo que se usan para sincronización, pero son más sencillos que los semáforos porque no tienen contadores. Los mutexes son esencialmente candados, con funciones API para cerrarse (`WaitForSingleObject`) y abrirse `ReleaseMutex`. Al igual que las manijas de semáforos, las manijas de mutexes pueden duplicarse y pasarse de un proceso a otro para que enlaces de diferentes procesos puedan acceder al mismo mutex.

El tercer mecanismo de sincronización se basa en **secciones críticas**, que son similares a los mutexes sólo que son locales respecto al espacio de direcciones del enlace que las crea. Puesto que las secciones críticas no son objetos del núcleo, no tienen manijas ni descriptores de seguridad y no pueden transferirse entre procesos. El cierre y la apertura se efectúan con `EnterCriticalSection` y `LeaveCriticalSection`, respectivamente. Puesto que estas funciones API se efectúan totalmente en el espacio de usuario, son mucho más rápidas que los mutexes.

El último mecanismo de sincronización emplea objetos del núcleo llamados **eventos**. Un enlace puede esperar que ocurra un evento con `WaitForSingleObject`. Un enlace puede liberar un enlace individual que esté esperando un evento con `SetEvent` o bien liberar todos los enlaces que esperan un evento, con `PulseEvent`. Hay varios tipos de eventos y además tienen diversas opciones.

Los eventos, mutexes y semáforos pueden recibir nombres y guardarse en el sistema de archivos como los tubos con nombre. Dos o más procesos pueden sincronizarse abriendo el mismo evento, mutex o semáforo, en lugar de hacer que uno de ellos cree el objeto y luego produzca copias del mango para los demás, aunque ésta no deja de ser una opción.

6.5 RESUMEN

El sistema operativo puede verse como un intérprete de ciertas características arquitectónicas que no se encuentran en el nivel ISA. Las principales de ellas son la memoria virtual, las instrucciones de E/S virtuales y los recursos para procesamiento en paralelo.

La memoria virtual es una característica arquitectónica cuyo propósito es permitir a los programas usar un espacio de direcciones mayor que la memoria física de la máquina, o proporcionar un mecanismo coherente y flexible para proteger y compartir la memoria. La memoria virtual se puede implementar como paginación pura, segmentación pura y una combinación de las dos. En la paginación pura el espacio de direcciones se divide en páginas virtuales de tamaño uniforme. Algunas de éstas se hacen corresponder con marcos de página físicos, pero otras no tienen correspondencia. La MMU traduce una referencia a una página con correspondencia en la dirección física correcta. Una referencia a una página sin correspondencia causa un fallo de página. Tanto el Pentium II como el UltraSPARC II tienen unas MMU avanzadas que manejan tanto memoria virtual como paginación.

La abstracción de E/S más importante en este nivel es el archivo. Un archivo consiste en una secuencia de bytes o registros lógicos que se pueden leer y escribir sin saber cómo funcionan los discos, cintas y otros dispositivos de E/S. El acceso a los archivos puede ser secuencial, aleatorio por número de registro o aleatorio por clave. Los directorios sirven para agrupar archivos. Los archivos se pueden almacenar en sectores consecutivos o dispersarse por todo el disco. En el segundo caso, normal en los discos duros, se requieren estructuras de

datos para localizar todos los bloques de un archivo. El espacio de disco libre puede contabilizarse con una lista o un mapa de bits.

Es común que se apoye el procesamiento en paralelo y se implemente simulando múltiples procesadores operando una sola CPU en tiempo compartido. La interacción no controlada entre procesos puede dar pie a condiciones de competencia. Para resolver este problema se introducen primitivas de sincronización, de las cuales los semáforos son un ejemplo sencillo. Con la ayuda de semáforos, los problemas de productor-consumidor se pueden resolver de forma sencilla y elegante.

Dos ejemplos de sistemas operativos avanzados son UNIX y NT. Ambos apoyan la paginación y archivos con correspondencia en la memoria, así como sistemas de archivos jerárquicos, con archivos que consisten en secuencias de bytes. Por último, ambos manejan procesos y enlaces y proporcionan formas de sincronizarlos.

PROBLEMAS

1. ¿Por qué un sistema operativo interpreta sólo algunas de las instrucciones de nivel 3, mientras que un microprograma interpreta todas las instrucciones de nivel ISA?
2. Una máquina tiene un espacio de direcciones virtual de 32 bits direccionable por bytes. El tamaño de página es de 8 KB. ¿Cuántas páginas de espacio de direcciones virtual existen?
3. Una memoria virtual tiene un tamaño de página de 1024 palabras, ocho páginas virtuales y cuatro marcos de página físicos. La tabla de páginas tiene este aspecto:

Página virtual	Marcos de página
0	3
1	1
2	no en la memoria principal
3	no en la memoria principal
4	4
5	no en la memoria principal
6	0
7	no en la memoria principal

- a. Haga una lista de todas las direcciones virtuales que causarán fallos de página.
 - b. ¿Qué dirección física tienen 0, 3728, 1023, 1024, 1025, 7800 y 4096?
4. Una computadora tiene 16 páginas de espacio de direcciones virtual pero sólo cuatro marcos de página. Inicialmente, la memoria está vacía. Un programa hace referencia a las páginas virtuales en el orden
0, 7, 2, 7, 5, 8, 9, 2, 4
a. ¿Cuáles referencias causan un fallo de página con LRU?
b. ¿Cuáles referencias causan un fallo de página con FIFO?